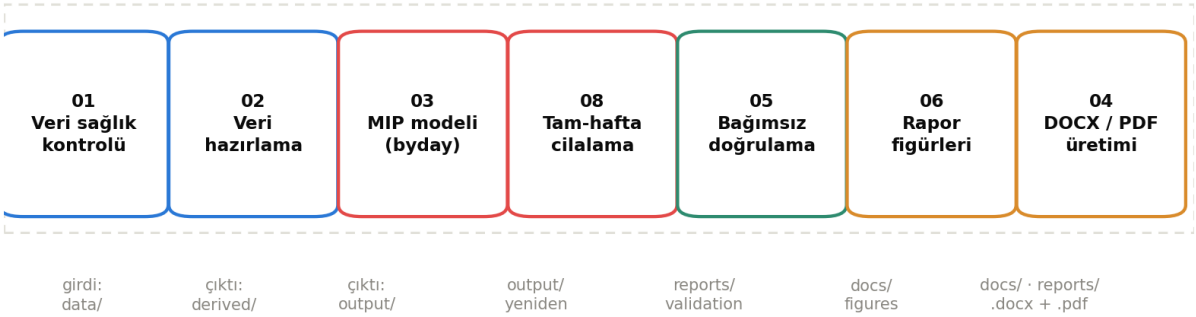
TEKNOFEST · Yapay Zekâ Destekli Havayolu Optimizasyonu

KOD REHBERİ

src/ klasöründeki her modülün, blok blok anlatımı

Takım: Büke

run_all.py — tek komutla uçtan uca koşu



Bir adım hata verirse akış AÇIK hata ile durur — doğrulanmamış çözüm teslim edilmez.

Dosya	Satır	Rolü	Bölüm
`01_data_health_check.py`	232	Veri sağlık kontrolü — teşhis raporu	1
`02_data_prep.py`	321	Veri hazırlama — `derived/` katmanı	2
`03_model.py`	1357	MIP modeli, çözüm ve çıktı (**çekirdek**)	3
`08_fullweek_polish.py`	596	Tam-hafta warm-start cilalaması	4
`05_validate_solution.py`	470	Bağımsız doğrulayıcı (çift kayıt)	5
`06_make_figures.py`	172	Rapor figürleri (PNG)	6
`07_sensitivity.py`	133	Duyarlılık analizi (λ, Wdev, τ)	7
`04_export_docs.py`	83	Markdown → DOCX/PDF dönüşümü	8
`run_all.py`	56	Tek komutla uçtan uca koşu	9

Toplam 3.364 satır Python · Pyomo + HiGHS · tüm açıklamalar Türkçe

İçindekiler

#	Bölüm	İçerik
0	Kuşbakışı — proje ne yapıyor, kod nasıl bölünmüş?	problem · sözlük · boru hattı · kısıt haritası
1	`01_data_health_check.py` — Veri Sağlık Kontrolü	teşhis raporu · veri tuzakları
2	`02_data_prep.py` — Veri Hazırlama	de-wrap · hafta normalizasyonu · `derived/`
3	`03_model.py` — MIP Modeli (çekirdek)	A1–G kısıtları · amaç · çözücü · byday
4	`08_fullweek_polish.py` — Tam-Hafta Cilalama	warm-start · global MIP-gap · bariyer
5	`05_validate_solution.py` — Bağımsız Doğrulayıcı	çift kayıt · 12 kontrol · PASS/FAIL
6	`06_make_figures.py` — Rapor Figürleri	4 PNG · palet · stil
7	`07_sensitivity.py` — Duyarlılık Analizi	λ · Wdev · τ taramaları
8	`04_export_docs.py` — Doküman Dışa Aktarma	pandoc · Word COM
9	`run_all.py` — Uçtan Uca Koşu	adım sırası · hata yönetimi
A	Ek A — Hızlı başvuru	sabitler · CFG · komutlar · çıktı dosyaları
B	Ek B — Kodla ilgili muhtemel jüri soruları	12 soru ve cevabı

0 Kuşbakışı — Bu proje ne yapıyor, kod nasıl bölünmüş?

0.1 Problem, beş cümlede

İstanbul (IST) bir **aktarma merkezidir**: yolcu bir uçuşla IST'e gelir, bir süre bekler, ikinci uçuşla nihai varış noktasına gider. Bu ikili birleşime **bağlantı** (itinerary) denir ve satılabilir bir üründür. Bir bağlantının gerçekten satılabilmesi için IST'teki bekleme süresinin ne çok kısa (yetişemezsiniz) ne de çok uzun (kimse tercih etmez) olması gerekir — brief bu pencereyi **60–300 dakika** olarak sabitler. Aynı O–D pazarında rakip havayolları da bağlantı satar; kim daha kısa toplam yolculuk süresi (J) sunuyorsa sıralamada önde olur. **Bu projenin görevi: THY'nin IST kalkış/varış saatlerini ±60 dakikalık bir pencerede, 10 dakikalık ızgarada oynatarak, fiziksel ve operasyonel kısıtları hiç bozmadan, gelir-ağırlıklı rekabetçilik puanını en yükseğe çıkarmak.**

> **Tek cümlelik özet**
>
> Kod, dört Excel dosyasını okur → tutarlı bir veri katmanı üretir → bunu bir **Karışık Tamsayılı Programlama (MIP)** modeline çevirir → HiGHS çözücüsüyle çözer → sonucu bağımsız bir ikinci kodla doğrular → rapor, figür ve teslim dosyalarını yazar.

0.2 Kod nasıl organize edilmiş? ("class" meselesi)

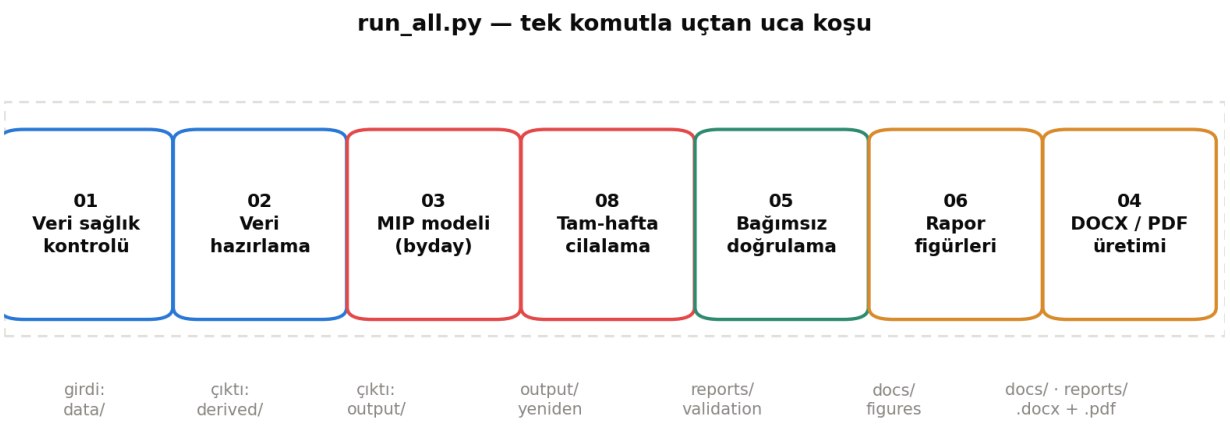
`src/` altındaki sekiz dosya birer **modül**dür; projenin organizasyon birimi sınıf değil, **modüldür**. Bunun bilinçli bir nedeni var: her modül tek bir sorumluluk taşır, kendi başına çalıştırılabilir (`python src/02_data_prep.py`) ve diskteki dosyalar üzerinden bir sonrakine bağlanır. Bu, akademik/yarışma projelerinde tercih edilen **boru hattı (pipeline) mimarisidir**: bir adım çöktüğünde hangi adımın hangi dosyayı bozduğu tek bakışta anlaşılır.

Projede yalnızca **iki gerçek sınıf** vardır ve ikisi de bilinçli olarak çok sadedir:

Sınıf	Dosya	Görevi
`Instance`	`03_model.py` (satır 205–207)	Modelin tüketeceği ön-işlenmiş alt-örneği taşıyan **boş veri kabı**. Gövdesi yalnız `pass`'tır; alanları dışarıdan (`build_instance`) atanır. Amaç: 15+ değişkeni fonksiyonlar arasında tek tek taşımak yerine tek nesnede toplamak.
`Checks`	`05_validate_solution.py` (satır 87–102)	Doğrulama sonuçlarını biriktiren **PASS/FAIL toplayıcı**. `add()` ile kontrol ekler, ekrana basar ve rapor satırını saklar; bir tane bile FAIL varsa `failed` bayrağını kaldırır.

Yani "sınıfları öğrenmek" istediğinizde aslında öğrenmeniz gereken şey modüllerin ne yaptığıdır — bu doküman tam olarak bunu, blok blok anlatır.

0.3 Boru hattı: hangi dosya ne zaman çalışır?



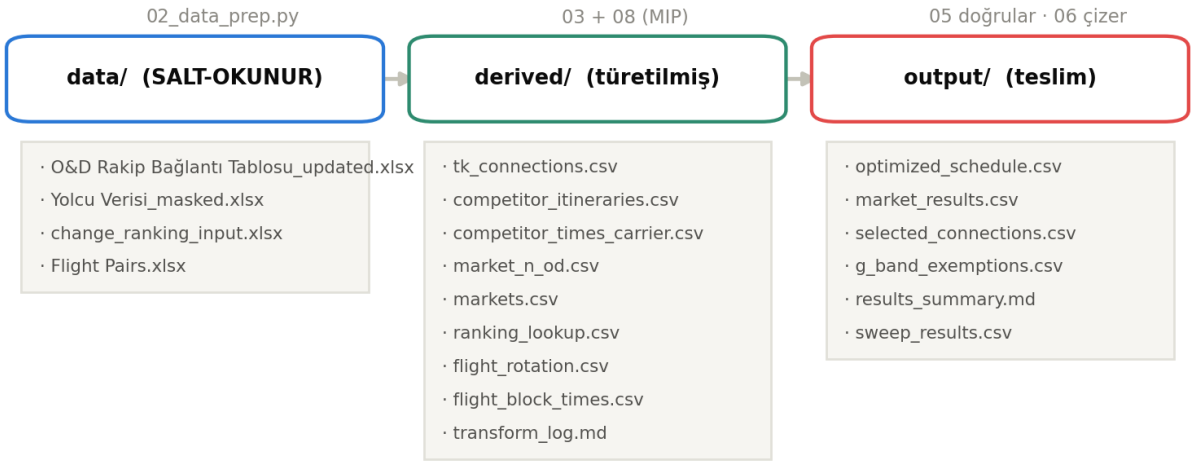
Bir adım hata verirse akış AÇIK hata ile durur — doğrulanmamış çözüm teslim edilmez.

Şekil 0.1 — `run_all.py` adımları çalışma sırasıyla koşturur. Numaralandırma dosya adına göredir; çalışma sırası 01 → 02 → 03 → 08 → 05 → 06 → 04'tür.

Dikkat edilecek iki nokta:

- **08, 03'ten sonra gelir** (numarası büyük ama aşama-2'dir): 03 haftanın günlerini ayrı ayrı çözer, 08 aynı çözümü haftanın tamamını kapsayan tek modele başlangıç noktası olarak verip küresel olarak iyileştirir.
- **05 en sonda değil, ortadadır**: doğrulama başarısız olursa figür ve doküman üretilmeden koşu durur. Doğrulanmamış bir çözüm için rapor üretmek, hatayı cilalamak demektir.

0.4 Veri katmanları: hangi klasör neyi tutar?



Altın kural: yarışmanın orijinal Excel dosyalarına ASLA yazılmaz; her dönüşüm derived/transform_log.md içine kaydedilir.

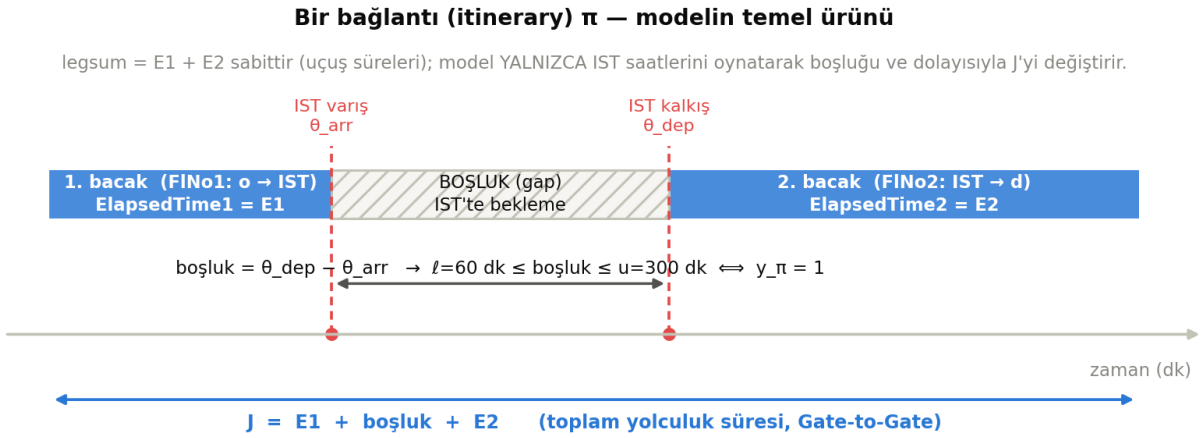
Şekil 0.2 — Tek yönlü veri akışı. Sol taraf asla değişmez; sağa doğru her katman bir öncekinden türetilir.

Klasör	İçerik	Kim yazar?	Kim okur?
`data/`	Yarışmanın orijinal 4 Excel dosyası	**Hiç kimse** (salt-okunur)	01, 02

Klasör	İçerik	Kim yazar?	Kim okur?
`derived/`	Temizlenmiş, türetilmiş CSV veri katmanı	02	03, 05, 07, 08
`output/`	Çözüm dosyaları (teslim #3)	03, 08	05, 06, 08
`reports/`	Sağlık raporu, doğrulama raporu, duyarlılık analizi	01, 05, 07, 08	04 (docx'e çevirir)
`docs/`	Model formülasyonu + figürler	06	04

0.5 Modelin temel nesnesi: bir bağlantı (π)

Kodun her yerinde karşınıza çıkacak dört büyüklüğü baştan oturtalım. Bunları bilmeden 03_model.py okunmaz:



Şekil 0.3 — Bağlantının anatomisi. Model uçuş sürelerini değiştiremez; yalnızca iki kırmızı çizgiyi (IST varış ve IST kalkış saatlerini) oynatabilir.

Simge	Koddaki adı	Ne demek?
θ (theta)	`theta_expr(...)`, `x[eid,v]`	Bir uçuş olayının IST saati (dakika). Modelin **karar değişkeni** budur.
boşluk (gap)	`gap_min`, `m._gap[pi]`	θ_kalkış − θ_varış. Yolcunun IST'te bekleme süresi.
legsum	`legsum_min`	İki bacağın uçuş süreleri toplamı (E1 + E2). **Sabittir**, model değiştiremez.
J	`J_min`, `J_dk`	Toplam yolculuk süresi = legsum + boşluk. Rakiple kıyaslanan büyüklük budur.

0.6 Kısıt haritası — kod boyunca A1, B1, E2 gibi etiketler göreceksiniz

Kısıtlar `docs/model_formulation.md` ile aynı harflerle etiketlenmiştir. Bu doküman boyunca her kısıt kodda geçtiği yerde ayrıntılı açıklanır; burası sadece haritadır:

Kod	Adı	Ne garanti eder?	Nerede kurulur?
A1	Tekil zaman ataması	Her uçuş olayı ±60 dk pencerede, 10 dk ızgarada **tam bir** saate atanır	03: 372–375
A2	Uçak rotasyonu	Dönüş varışı ≥ gidiş kalkışı + T_OB + τ + T_IB (aynı uçak gidip döner)	03: 492–566
B1	Bağlantı uygunluğu	$y_\pi = 1 \Leftrightarrow 60 \le$ boşluk ≤ 300 dk	03: 386–393

Kod	Adı	Ne garanti eder?	Nerede kurulur?
B2	Zorunlu sunum	Uygun her bağlantı sunulmalı — **kapalı** (E1 ile çatışıyor)	03: 394–400
C	Azalan getiri	Bir pazardaki j. bağlantının ödülü (½)^(j−1); monoton doldurma	03: 402–418
D	Rakip yenme + sıralama	J ≤ T_comp ise rakip yenilir; yenilen sayısı sıralama ödülüne çevrilir	03: 420–475
E1	Yönsel sayı dengesi	|n_od − n_do| ≤ 0.20·(n_od + n_do)	03: 477–490
E2	En iyi süre dengesi	|J_min(o→d) − J_min(d→o)| ≤ 30 dk — tam ölçekte **kapalı**	03: 649–684
F	IST hub kapasitesi	Her 10 dk kovada kalkış ≤ 10, varış ≤ 15	03: 686–725
G	Tarife düzenliliği	Aynı uçuş no'nun saat-içi zamanı günler arası ±15 dk bandında	03: 590–647
P	Tarife-değişim cezası	Amaçtan −λ·Σ|θ−cur| düşülür (λ = 0.3)	03: 734–748

0.7 Terimler sözlüğü (kodda geçtiği hâliyle)

Terim	Anlamı
O–D (pazar)	Origin–Destination, yönlü pazar. Örn. `"AGP-KCM"`. Kodda `market`.
itinerary	Bir aktarmalı yolculuk seçeneği. TK'nınkine `conns`, rakibinkine `competitor_itineraries` denir.
ρ (rho)	Pazarın gelir/önem ağırlığı (`OD importance factor`). Amaç fonksiyonundaki çarpan — büyük pazarda kazanılan puan daha değerlidir.
N_od	O–D pazarındaki rakip **taşıyıcı** sayısı (`n_od_def = "carrier"`).
T_comp	Bir rakip taşıyıcının o pazardaki **en iyi** (en kısa) J değeri.
b_od	Pazarın **başlangıç sırası** — mevcut tarifedeki performansa göre hesaplanır. Sıralama ödülü b_od'den s'ye geçişin ödülüdür.
s	Son sıra = clip(N − yenilen, 1, N). Küçük s = daha iyi sıralama.
baseline	Mevcut (hiç dokunulmamış) tarifenin amaç değeri. Tüm iyileşme buna göre ölçülür.
MIP	Mixed-Integer Programming. Bazı değişkenleri tam sayı (burada 0/1) olan doğrusal optimizasyon.
Pyomo / HiGHS	Pyomo modeli Python'da kurar, HiGHS açık-kaynak çözücü onu çözer.
ikili değişken	0 veya 1 değeri alan karar değişkeni. `x`, `y`, `z`, `w`, β, γ hepsi ikilidir.
big-M	"Şu koşul aktifse şu eşitsizlik geçerli olsun" mantığını doğrusal hâle getiren büyük sabit. Kodda `Mlo`, `Mhi`, `Mw`, `MJ` isimleriyle geçer.
warm-start / MIP-start	Çözücüye "işte elimde çalışan bir çözüm, buradan başla" demek. 08'in tüm yöntemi budur.
MIP-gap	Bulunan çözüm ile teorik üst sınır arasındaki bağıl fark. %0.1 gap = "en fazla binde bir daha iyisi olabilir".
byday	Güne-göre ayrıştırma: 7 günü ayrı ayrı çözme stratejisi (`03_model.py byday`).
λ (lambda)	Tarife-değişim cezasının ağırlığı. Büyük λ = "uçuşları kımıldatma".
Wdev / Xdev / τ / Δ	Sırasıyla: uçuş penceresi (±60 dk), günler-arası düzenlilik bandı (15 dk), minimum yer süresi (45 dk), ızgara adımı (10 dk).

0.8 Bu dokümanı nasıl okumalısınız?

- Her bölüm bir dosyayı anlatır. Bölüm başında **"Bu dosya ne yapar"** özeti, girdi/çıktı tablosu ve çalıştırma komutu vardır.

- Ardından dosya **baştan sona, blok blok** ilerler. Her blokta önce **gerçek kod** (satır numaralarıyla, kaynaktan birebir), sonra **ne yaptığının açıklaması** gelir.
- Gri kutular bağlam/uyarı, mavi kutular önemli fikirler, yeşil kutular "neden böyle yapılmış" gerekçeleridir.
- Sadece belirli bir kısıtı anlamak istiyorsanız **0.6'daki haritadan** doğrudan ilgili satır aralığına gidin.

1 01_data_health_check.py — Veri Sağlık Kontrolü

Bu dosya ne yapar?

Dört girdi Excel'ini okur ve **hiçbirini değiştirmeden** "bu veriyle modelleme yapılabilir mi?" sorusunu yanıtlar. Kaç satır var, hangi sütunlar eksik, saatler tutarlı mı, kaç bağlantı zaten uygun pencerede, hangi pazarların gelir ağırlığı yok — hepsini ekrana basar ve `reports/data_health_report.md` dosyasına yazar. **Tek bir karar üretmez; sadece teşhis koyar.**

	Değer
Girdi	`data/` — 4 Excel dosyası (salt-okunur)
Çıktı	`reports/data_health_report.md`
Çalıştırma	`python src/01_data_health_check.py` (proje kökünden)
Bağımlılık	yalnız `pandas`
Süre	birkaç saniye

Neden ilk adım bu? Çünkü **bir optimizasyon modelinin en tehlikeli hatası, sessiz veri hatasıdır.** Model çalışır, sayı üretir, ama üretilen sayı yanlış varsayıma dayanır. Bu dosya o varsayımları koşmadan önce açığa çıkarır.

1.1 Modül başlığı ve kurulum

`src/01_data_health_check.py` · satır 1–20 · docstring + import'lar

```
"""
TEKNOFEST Havayolu Optimizasyonu — Veri Sağlık Kontrolü (1. adım)

4 girdi Excel dosyasını okur, tutarlı bir veri katmanına bağlar ve
modellemeden önce bilmemiz gereken her şeyi bir 'sağlık raporu' olarak çıkarır.

Çalıştırma:  python src/01_data_health_check.py   (proje kökünden)
Girdi:       data/   (yarışmanın orijinal dosyalarının kopyası — değiştirilmez)
Çıktı:       reports/data_health_report.md
"""

import os
import datetime as dt
import warnings

import pandas as pd

warnings.filterwarnings("ignore")
pd.set_option("display.width", 200)
pd.set_option("display.max_columns", 60)
```

Üstteki üç tırnaklı metin **modül docstring'idir**: dosyanın ne yaptığını, nasıl çalıştırıldığını, girdi ve çıktısını tek bakışta verir. Bu projedeki her dosya bu kalıbı izler — jüri veya yeni bir geliştirici dosyayı açtığında ilk 10 satırda bağlamı alır.

- `warnings.filterwarnings("ignore")` — openpyxl'in Excel biçim uyarıları raporu kirletmesin diye susturulur. Rapor okunabilirliği önemlidir.
- `pd.set_option("display.width", 200)` ve `max_columns 60` — pandas'ın geniş tabloları `...` ile kırpmasını engeller; sağlık raporunda tüm sütun adlarını görmek isteriz.

1.2 Yol sabitleri — "proje kökü neresi?" problemi

`src/01_data_health_check.py` · satır 22–27 · yolların dosyanın konumundan türetilmesi

```python
# proje kökü = bu dosyanın bulunduğu src/ klasörünün üstü
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
DATA = os.path.join(ROOT, "data")
REPORT_DIR = os.path.join(ROOT, "reports")
os.makedirs(REPORT_DIR, exist_ok=True)
```

Bu üç satır küçük görünür ama projenin **her yerden çalışabilmesini** sağlar:

1. `__file__` bu dosyanın yolu; `os.path.abspath` onu mutlak yola çevirir.
2. `os.path.dirname(...)` → `HERE` = `.../Buke_cozum/src`
3. Bir kez daha `dirname` → `ROOT` = `.../Buke_cozum` (proje kökü).

> **Neden `os.getcwd()` kullanılmadı?**
>
> Çünkü çalışma dizini, betiği nereden çağırdığınıza göre değişir. `__file__` tabanlı türetme ile betik **hangi klasörden çağrılırsa çağrılsın** doğru `data/` ve `reports/` klasörlerini bulur. `os.makedirs(REPORT_DIR, exist_ok=True)` da klasör yoksa oluşturur, varsa hata vermez.

1.3 `_find_conn_file()` — hangi bağlantı tablosunu kullanacağız?

`src/01_data_health_check.py` · satır 29–40 · dayanıklı dosya bulma

```python
def _find_conn_file():
    """Bağlantı tablosunu bul: güncellenmiş sürüm (ElapsedTime'lı) öncelikli."""
    for name in ["O&D Rakip Bağlantı Tablosu_updated.xlsx",
                 "O&D Rakip Bağlantı Tablosu.xlsx"]:
        p = os.path.join(DATA, name)
        if os.path.exists(p):
            return p
    import glob as _glob
    hits = sorted(_glob.glob(os.path.join(DATA, "O&D*.xlsx")))
    if hits:
        return hits[0]
    raise FileNotFoundError("data/ altında O&D bağlantı tablosu bulunamadı")
```

Yarışma sürecinde bağlantı tablosunun **iki sürümü** dolaştı: orijinal ve `_updated` (bacak sürelerini, yani `ElapsedTime1/2` sütunlarını içeren) sürüm. Güncellenmiş sürüm çok daha değerlidir — nedeni 2. bölümde açıklanacak — bu yüzden fonksiyon **öncelik sırasıyla** arar:

1. Önce `..._updated.xlsx` var mı? Varsa onu döndür.
2. Yoksa orijinal `O&D Rakip Bağlantı Tablosu.xlsx`'i dene.
3. O da yoksa `glob` ile `O&D*.xlsx` kalıbına uyan **her şeyi** ara, alfabetik ilkini al.
4. Hiçbiri yoksa `FileNotFoundError` fırlat — sessizce devam etme.

> **Bu neden önemli?**
>
> Gizli test verisi farklı bir dosya adıyla gelebilir. `glob` yedeği sayesinde kod, adı tam bilmediği bir dosyayı da bulabilir; bulamazsa **açık ve okunabilir bir hatayla** durur. Sessiz çökme yerine anlaşılır hata — bu proje boyunca tekrar eden bir tasarım ilkesidir.

`src/01_data_health_check.py` · satır 42–47 · dosya sabitleri ve bağlantı penceresi

```python
F_CONN = _find_conn_file()
F_PAX = os.path.join(DATA, "Yolcu Verisi_masked.xlsx")
F_RANK = os.path.join(DATA, "change_ranking_input.xlsx")
F_PAIR = os.path.join(DATA, "Flight Pairs.xlsx")

L_MIN, U_MIN = 60, 300  # bağlantı penceresi (dk)
```

`L_MIN, U_MIN = 60, 300` — brief'in verdiği bağlantı penceresi. Bir yolcunun IST'te en az 60, en çok 300 dakika beklemesi gerekir. Bu iki sayı projedeki en sık kullanılan sabitlerdir ve **her dosyada yeniden tanımlanır** (05 doğrulayıcısında bile) — çünkü modülleri birbirinden bağımsız tutmak, doğrulamanın da bağımsız olmasını sağlar.

1.4 Rapor tamponu: `out`, `h1`, `h2`

`src/01_data_health_check.py` · satır 49–56 · hem ekrana bas hem dosyaya biriktir

```
49 _buf = []
50 def out(*args):
51     line = " ".join(str(a) for a in args)
52     print(line)
53     _buf.append(line)
54
55 def h1(t): out("\n" + "=" * 90); out(t); out("=" * 90)
56 def h2(t): out("\n" + "-" * 70); out(t); out("-" * 70)
```

Basit ama etkili bir hile: `out()` fonksiyonu aldığı her şeyi metne çevirir, **hem `print` eder hem de `_buf` listesine ekler**. Böylece betiğin sonunda `_buf`'ı dosyaya yazmak yeterlidir — ekranda gördüğünüz rapor ile dosyadaki rapor **birebir aynıdır**. `h1` ve `h2` ise `=` ve `-` karakterlerinden başlık çizgileri üretir; düz metin raporda görsel hiyerarşi böyle kurulur.

1.5 `time_to_min()` — Excel'in saat kâbusu

`src/01_data_health_check.py` · satır 59–67 · saat/süre → dakika

```
59 def time_to_min(v):
60     """time-of-day veya timedelta -> dakika."""
61     if pd.isna(v):
62         return None
63     if isinstance(v, dt.time):
64         return v.hour * 60 + v.minute + v.second / 60.0
65     if isinstance(v, (pd.Timedelta, dt.timedelta)):
66         return v.total_seconds() / 60.0
67     return None
```

Excel bir hücredeki "06:19" değerini duruma göre **saat-of-day** (`datetime.time`) ya da **süre** (`timedelta`) olarak verir. Bu fonksiyon ikisini de dakikaya çevirir; tanımadığı bir şey gelirse `None` döndürür (sessizce sıfır varsaymaz).

> **İleride büyüyecek bir sorunun ilk işareti**
>
> `dt.time` için formül `saat*60 + dakika`'dır — yani **24 saati aşan bir süre bu biçimde temsil edilemez**. 30 saatlik bir yolculuk Excel'de "06:00" olarak görünür. Bu dosya sorunu sadece *tespit eder*; gerçek çözüm `02_data_prep.py`'nin `dur_min()` fonksiyonundadır.

1.6 `main()` — dört dosyayı oku

`src/01_data_health_check.py` · satır 70–77 · raporun açılışı ve okuma

```
70 def main():
71     h1("TEKNOFEST VERİ SAĞLIK RAPORU")
72     out("Üretim tarihi:", dt.datetime.now().strftime("%Y-%m-%d %H:%M"))
73
74     conn = pd.read_excel(F_CONN, sheet_name=0)
75     pax = pd.read_excel(F_PAX, sheet_name=0)
76     rank = pd.read_excel(F_RANK, sheet_name=0)
77     pair = pd.read_excel(F_PAIR, sheet_name=0)
```

`sheet_name=0` her dosyanın **ilk sayfasını** alır. Dosya adları değişebilir ama veri hep ilk sayfadadır. Bu noktadan sonra dört DataFrame elimizdedir: `conn` (bağlantı tablosu), `pax` (yolcu/gelir ağırlıkları), `rank` (sıralama ödül tablosu), `pair` (uçak rotasyon eşleri).

1.7 Bölüm 1 — Bağlantı tablosunun anatomisi

`src/01_data_health_check.py` · satır 79–92 · boyut, sütunlar, ElapsedTime var mı?

```
79      # ============================================================
80      h1(f"1) BAĞLANTI TABLOSU  ({os.path.basename(F_CONN)})")
81      out("Boyut:", conn.shape)
82      out("Kolonlar:", list(conn.columns))
83      has_elapsed = {"ElapsedTime1", "ElapsedTime2"}.issubset(conn.columns)
84      out("ElapsedTime1/2 (bacak süreleri) mevcut:", "EVET -> J kesin hesaplanabilir"
85          if has_elapsed else "HAYIR -> Gate-to-Gate wrap riski (de-wrap sezgiseli gerekir)")
86      if "ML2" in conn.columns:
87          out("ML2 (taşıyıcı tipi, M=Mainline/L=Low-cost) dağılımı:",
88              conn["ML2"].value_counts(dropna=False).to_dict())
89
90      out("\nArr Time aralığı:", conn["Arr Time"].min(), "->", conn["Arr Time"].max())
91      out("Dep Time aralığı:", conn["Dep Time"].min(), "->", conn["Dep Time"].max())
92      out("Gün değerleri:", sorted(conn["Gün"].dropna().unique().tolist()))
```

Buradaki en kritik satır **83–85**'tir:

> **`ElapsedTime1/2` neden "J kesin hesaplanabilir" demek?**
>
> Bu iki sütun varsa her bacağın uçuş süresi doğrudan bilinir ve **J = E1 + boşluk + E2** olarak *hesaplanır*. Yoksa J'yi yalnız `Gate-to-Gate` sütunundan okumak zorunda kalırız — o sütun ise 24 saatte sarabildiği için belirsizdir. Rapor bu durumu tek satırda "EVET → J kesin" / "HAYIR → wrap riski" olarak bildirir.

`ML2` sütunu varsa taşıyıcı tipi dağılımı (M = Mainline / L = Low-cost) da yazılır. Brief'in sıralama tanımı taşıyıcı tipi ayrımı yapmadığı için bu bilgi modele girmez, sadece raporlanır.

`src/01_data_health_check.py` · satır 94–103 · taşıyıcı dağılımı ve TK/rakip ayrımı

```
94      h2("Taşıyıcı dağılımı (Cr1)")
95      vc = conn["Cr1"].value_counts()
96      out("Toplam taşıyıcı:", vc.shape[0])
97      out(vc.head(15).to_string())
98      is_tk = conn["Cr1"] == "TK"
99      out("\nTK satır:", int(is_tk.sum()), " | Rakip satır:", int((~is_tk).sum()))
100
101     tk = conn[is_tk].copy()
102     out("\nTK Arr1 (gelen bacağın varışı) benzersiz:", tk["Arr1"].unique().tolist()[:5])
103     out("TK Dep2 (giden bacağın kalkışı) benzersiz:", tk["Dep2"].unique().tolist()[:5])
```

`conn["Cr1"] == "TK"` maskesi tabloyu ikiye böler: **TK satırları** (bizim optimize edeceğimiz bağlantılar) ve **rakip satırları** (kıyaslayacağımız referanslar). Bu ayrım kodun tamamında `is_tk` adıyla yaşar.

`src/01_data_health_check.py` · satır 105–117 · tutarlılık ve pazar kapsamı kontrolleri

```
105     conn["_od_from_cols"] = conn["Dep1"].astype(str) + "-" + conn["Arr2"].astype(str)
106     mism = (conn["_od_from_cols"] != conn["O&D"]).sum()
107     out("\nO&D kolonu, (Dep1-Arr2) ile uyuşmayan satır sayısı:", int(mism))
108
109     out("\nBenzersiz O&D pazarı (yönlü):", conn["O&D"].nunique())
110     out("Benzersiz (O&D, Gün) anahtarı:", conn.groupby(['O&D', 'Gün']).ngroups)
111
112     tk_markets = set(tk["O&D"].unique())
113     all_markets = set(conn["O&D"].unique())
114     comp_markets = set(conn[~is_tk]["O&D"].unique())
115     out("\nTK bağlantısı OLAN pazar sayısı:", len(tk_markets))
116     out("TK bağlantısı OLMAYAN (sadece rakip) pazar sayısı:", len(all_markets - tk_markets))
117     out("Rakip bağlantısı OLMAYAN (sadece TK) pazar sayısı:", len(all_markets - comp_markets))
```

- **105–107:** `O&D` sütunu gerçekten `Dep1-Arr2` mi? Uyuşmayan satır sayısı 0 olmalı — değilse pazar anahtarına güvenilemez.
- **109–110:** Kaç benzersiz pazar, kaç benzersiz (pazar, gün) çifti var? Bu sayılar modelin **büyüklüğünü** belirler.
- **112–117:** Üç kümenin karşılaştırması: TK'nın bağlantısı olan pazarlar, hiç TK bağlantısı olmayanlar (buralarda yapabileceğimiz bir şey yok), rakibi olmayanlar (buralarda sıralama terimi tanımsızdır).

1.8 Bölüm 2 — Mevcut tarifedeki bağlantılar gerçekten geçerli mi?

`src/01_data_health_check.py` · satır 119–134 · boşluk hesabı ve pencere kontrolü

```
119     # ============================================================
120     h1("2) TK BAĞLANTI GEÇERLİLİĞİ  (mevcut tarifede)")
121     tk["gap_min"] = (tk["Dep Time"] - tk["Arr Time"]).dt.total_seconds() / 60.0
122     tk["j_min"] = tk["Gate-to-Gate Uçuş Süresi"].map(time_to_min)
123     tk["legsum_min"] = tk["j_min"] - tk["gap_min"]
124
125     out("Bağlantı boşluğu (dk): min/medyan/max =",
126         round(tk["gap_min"].min(), 1), "/",
127         round(tk["gap_min"].median(), 1), "/",
128         round(tk["gap_min"].max(), 1))
129     in_window = tk["gap_min"].between(L_MIN, U_MIN)
130     out(f"[{L_MIN},{U_MIN}] penceresinde olan TK bağlantısı:",
131         int(in_window.sum()), f"/ {len(tk)}  ({100*in_window.mean():.1f}%)")
132     out("Negatif boşluk (veri hatası?) sayısı:", int((tk["gap_min"] < 0).sum()))
133     out("Boşluk < L (kısa) :", int((tk["gap_min"] < L_MIN).sum()),
134         " | Boşluk > U (uzun):", int((tk["gap_min"] > U_MIN).sum()))
```

Üç türetme, üç satırda:

1. `gap_min` = kalkış − varış (dakika). Yolcunun IST'te beklediği süre.
2. `j_min` = Gate-to-Gate süresi (dakikaya çevrilmiş). Toplam yolculuk.
3. `legsum_min` = J − boşluk. Yani **iki bacağın uçuş süreleri toplamı** — model boyunca sabit kalacak büyüklük.

Ardından gelen istatistikler doğrudan projenin **iyileştirme potansiyelini** gösterir: kaç bağlantı zaten [60, 300] penceresinde? Kaçı çok kısa, kaçı çok uzun? **Negatif boşluk sayısı** ise saf bir veri hatası sinyalidir (kalkış varıştan önce olamaz).

`src/01_data_health_check.py` · satır 136–152 · 24 saat sarması ve bacak ayrıştırma fizibilitesi

```
136     out("\nGate-to-Gate (J) dakika: min/medyan/max =",
137         round(tk["j_min"].min(), 1), "/",
138         round(tk["j_min"].median(), 1), "/",
139         round(tk["j_min"].max(), 1))
140     out("J >= 24h (time formatında temsil edilemez!) sayısı:",
141         int((tk["j_min"] >= 24 * 60 - 1).sum()),
142         "  -> uzun itineraryler 'time' alanında kırpılmış olabilir, KONTROL ET")
143
144     h2("Bacak blok süresi ayrıştırma fizibilitesi")
145     g = tk.groupby(["FlNo1", "Gün"])["Arr Time"].nunique()
146     out("Aynı (FlNo1,Gün) için farklı 'Arr Time' sayısı >1 olan grup:", int((g > 1).sum()),
147         " (0 olmalı: bir gelen uçuşun tek IST varışı var)")
148     g2 = tk.groupby(["FlNo2", "Gün"])["Dep Time"].nunique()
149     out("Aynı (FlNo2,Gün) için farklı 'Dep Time' sayısı >1 olan grup:", int((g2 > 1).sum()),
150         " (0 olmalı: bir giden uçuşun tek IST kalkışı var)")
151     out("Benzersiz gelen TK uçuş örneği (FlNo1,Gün):", tk.groupby(["FlNo1", "Gün"]).ngroups)
152     out("Benzersiz giden TK uçuş örneği (FlNo2,Gün):", tk.groupby(["FlNo2", "Gün"]).ngroups)
```

140–142 satırları az önce bahsedilen tuzağı sayar: J ≥ 24 saat olan kaç satır var? Eğer `time` biçiminde tutuluyorsa bunlar **kırpılmıştır** ve uzun pazarların J değeri olduğundan çok küçük görünür — yani model, olmayan bir üstünlüğü ödüllendirir.

144–152 ise ileriye dönük bir fizibilite testidir: aynı `(FlNo1, Gün)` çifti için birden fazla farklı varış saati var mı? Olmamalı — bir uçuşun bir gündeki IST varışı tektir. Bu sayı 0 değilse, aynı fiziksel uçuş modelde birden çok karar olayına bölünür ve tutarsız kaydırma alabilir. (02 bunu **hafta normalizasyonu** ile çözer.)

1.9 Bölüm 3 — Rakipler

`src/01_data_health_check.py` · satır 154–170 · N_od tanımı ve rakip süreleri

```
154     # ============================================================
155     h1("3) RAKİP SÜRELERİ ve RAKİP SAYISI (N_od)")
156     comp = conn[~is_tk].copy()
157     comp["j_min"] = comp["Gate-to-Gate Uçuş Süresi"].map(time_to_min)
158     by_market = comp.groupby(["O&D", "Gün"])
159     n_itin = by_market.size()
160     n_carr = by_market["Cr1"].nunique()
161     out("Rakip itinerary sayısı / (O&D,Gün): min/medyan/max =",
```

```
162          int(n_itin.min()), "/", int(n_itin.median()), "/", int(n_itin.max()))
163      out("Farklı rakip taşıyıcı / (O&D,Gün): min/medyan/max =",
164          int(n_carr.min()), "/", int(n_carr.median()), "/", int(n_carr.max()))
165      out("Rakip J (dk): min/medyan/max =",
166          round(comp["j_min"].min(), 1), "/", round(comp["j_min"].median(), 1), "/", round(comp["j_min"].max(),
1))
167      out("\nNOT: N_od (rakip sayısı) tanımı netleştirilmeli — farklı itinerary mi, farklı taşıyıcı mı?")
168      out("Ranking lookup 'Rakip Sayısı' aralığı:", int(rank['Rakip Sayısı'].min()), "-", int(rank['Rakip
Sayısı'].max()))
169      out("Itinerary-bazlı rakip sayısı 40'ı aşan (O&D,Gün) sayısı:", int((n_itin > 40).sum()),
170          " | taşıyıcı-bazlı 40'ı aşan:", int((n_carr > 40).sum()))
```

Buradaki soru masumca görünür ama **ödül tablosunu doğrudan etkiler**: bir pazarda "kaç rakip var" derken neyi sayıyoruz?

Tanım	Ne sayar?	Sonuç
itinerary bazlı	Rakiplerin sunduğu **her bağlantı seçeneğini**	Sayı 40'ı aşabilir → ödül tablosunun dışına çıkar
taşıyıcı bazlı	Farklı **havayolu şirketi** sayısını	Veride en fazla 17 → ödül tablosu (max N = 40) **tam kapsar ✓**

Satır 168–170 bu iki tanımı sayısal olarak karşılaştırır ve `change_ranking_input.xlsx`'in kapsadığı N aralığını yazar. 02_data_prep bu bulguya dayanarak `n_od_def = "carrier"` seçer.

1.10 Bölüm 4 — Gelir ağırlığı ρ

src/01_data_health_check.py · satır 172–184 · ρ kapsamı ve eksikler

```
172      # ============================================================
173      h1("4) YOLCU VERİSİ / GELİR AĞIRLIĞI (ρ_od)")
174      out("Boyut:", pax.shape, " | Kolonlar:", list(pax.columns))
175      out("ρ (önem) aralığı:", int(pax["OD importance factor"].min()), "-", int(pax["OD importance
factor"].max()))
176      pax["_od"] = pax["Orig Airport Code"].astype(str) + "-" + pax["Dest Airport Code"].astype(str)
177      dup = pax["_od"].duplicated().sum()
178      out("Tekrarlı O-D satırı:", int(dup), " | Benzersiz O-D:", pax["_od"].nunique())
179      pax_set = set(pax["_od"])
180      missing_rho = [m for m in all_markets if m not in pax_set]
181      out("Bağlantı tablosundaki pazar sayısı:", len(all_markets))
182      out("ρ değeri OLMAYAN pazar sayısı:", len(missing_rho))
183      if missing_rho:
184          out("  Örnek eksikler:", missing_rho[:10])
```

`OD importance factor`, bir pazarın THY için ne kadar değerli olduğunu söyler — amaç fonksiyonundaki **çarpandır**. Bu blok iki şeyi ölçer: ρ değerleri hangi aralıkta ve **bağlantı tablosundaki kaç pazarın ρ'su yok?**

> **ρ'su olmayan pazar ne anlama gelir?**
>
> Amaç fonksiyonu ρ ile çarpıldığı için, ρ'su olmayan bir pazar için "bu iyileştirme ne kadar değerli?" sorusunun cevabı yoktur. 0 varsaymak o pazarı önemsiz saymak, ortalama atamak ise **veri uydurmak** olur. Proje üçüncü yolu seçer: bu pazarları amaç evreninden **dışlar** ama uçuşlarını hub kapasitesinde (F kısıtı) saymaya devam eder. Bu ayrım 03'te `extra_use` mekanizmasıyla hayata geçer.

1.11 Bölüm 5 — Sıralama ödül tablosu

src/01_data_health_check.py · satır 186–197 · lookup tablosunun iç tutarlılığı

```
186      # ============================================================
187      h1("5) RANKING ÖDÜL TABLOSU (change_ranking_input.xlsx)")
188      out("Boyut:", rank.shape, " | Kolonlar:", list(rank.columns))
189      out("Weight aralığı:", round(rank["Weight"].min(), 4), "-", round(rank["Weight"].max(), 4))
190      cov = rank.groupby("Rakip Sayısı").size()
191      out("\nN -> satır sayısı (ilk birkaç):")
192      for n in sorted(rank["Rakip Sayısı"].unique())[:6]:
193          out(f"  N={n}: {int(cov[n])} satır (beklenen N*N={n*n})")
194      bad = (rank["Son Rank"] > rank["İlk Rank"]).sum()
```

```
    out("Son Rank > İlk Rank olan satır (sıralama kötüleşmesi):", int(bad))
    bad2 = (rank["İlk Rank"] > rank["Rakip Sayısı"]).sum()
    out("İlk Rank > Rakip Sayısı olan (tutarsız) satır:", int(bad2))
```

`change_ranking_input.xlsx` şu soruya yanıt verir: "*N rakip varken b. sıradan s. sıraya çıkarsam kaç puan kazanırım?*" Blok üç şeyi doğrular:

- Her N için satır sayısı **N×N** mi? (tüm b→s kombinasyonları kapsanmış mı)
- `Son Rank > İlk Rank` olan satır var mı? — sıralamanın *kötüleştiği* durumlar tabloda tanımlı mı?
- `İlk Rank > Rakip Sayısı` gibi mantıksız satır var mı?

1.12 Bölüm 6 — Uçak rotasyonu (Flight Pairs)

src/01_data_health_check.py · satır 199–212 · rotasyon eşleri ve kapsam

```
    # ============================================================
    h1("6) FLIGHT PAIRS / ROTASYON (Flight Pairs.xlsx)")
    out("Boyut:", pair.shape, " | Kolonlar:", list(pair.columns))
    out("Benzersiz Pair:", pair["Pair"].nunique(), " | Benzersiz FlNo:", pair["FlNo"].nunique())
    psize = pair.groupby("Pair").size()
    out("Pair başına uçuş sayısı dağılımı:", psize.value_counts().to_dict())
    out("IST'ten kalkan (Orig=IST) uçuş:", int((pair["Orig"] == "IST").sum()),
        " | IST'e gelen (Dest=IST):", int((pair["Dest"] == "IST").sum()))
    tk_flnos = set(tk["FlNo1"]).union(set(tk["FlNo2"]))
    pair_flnos = set(pair["FlNo"])
    out("\nBağlantı tablosundaki benzersiz TK uçuş no:", len(tk_flnos))
    out("Flight Pairs'teki uçuş no:", len(pair_flnos))
    out("TK uçuş no'su olup Flight Pairs'te OLMAYAN:", len(tk_flnos - pair_flnos))
    out("Flight Pairs'te olup bağlantı tablosunda kullanılmayan uçuş no:", len(pair_flnos - tk_flnos))
```

`Flight Pairs.xlsx` hangi gidiş uçuşunun hangi dönüş uçuşuyla **aynı uçağı** paylaştığını söyler. Bu, A2 kısıtının ham maddesidir. Kritik satır **211**'dir:

> **"Flight Pairs'te olmayan TK uçuşu" ne demek?**
>
> Rotasyon partneri bilinmeyen bir uçuşu kaydırmak risklidir — hangi uçağın hangi tura bağlı olduğunu bilmeden saatini değiştirirsek fiziksel olarak uygulanamaz bir tarife üretebiliriz. Proje bu uçuşları **"ayarlanamaz / sabit"** kabul eder (03'te `fixed_flnos`). İhtiyatlı ve savunulabilir bir tercih: bilmediğimiz şeye dokunmuyoruz.

1.13 Bölüm 7 — Açık sorular ve raporun yazılması

src/01_data_health_check.py · satır 214–228 · modellemeden önce karara bağlanacaklar + dosyaya yazma

```
    # ============================================================
    h1("7) ÖZET / AÇIK SORULAR (modellemeden önce karara bağlanmalı)")
    out("- N_od tanımı: rakip = farklı itinerary mi, farklı taşıyıcı mı? (ranking ödülünü doğrudan etkiler)")
    out("- Gate-to-Gate 'time' formatı 24h'i aşan itineraryleri kırpıyor olabilir — uzun pazarları doğrula.")
    out("- T_IB / T_OB bacak blok sürelerini connection'dan ayrıştırma stratejisi (J - boşluk = sabit).")
    out("- Ayar pencereleri (Standard: sınır yok) — gün-içi düzenlilik (X_dev=15dk) ve hub kovaları (10dk) ile etkileşim.")
    out("- ρ eksik pazarlar için varsayım (0 mı, ortalama mı, ele mi?).")
    out("- Constraint H (slot pencereleri) sadece sunumda — dahil edilecek mi netleştir.")

    report_path = os.path.join(REPORT_DIR, "data_health_report.md")
    with open(report_path, "w", encoding="utf-8") as f:
        f.write("# TEKNOFEST Veri Sağlık Raporu\n\n```\n")
        f.write("\n".join(_buf))
        f.write("\n```\n")
    out("\n[OK] Rapor yazıldı:", report_path)
```

Bu blok, sağlık kontrolünün **çıktısı olan altı karar sorusunu** listeler. Hepsi 02 ve 03'te bilinçli birer tercihle kapanır:

Açık soru	Nerede karara bağlandı?
N_od tanımı: itinerary mi taşıyıcı mı?	02 — `CFG["n_od_def"] = "carrier"` (veri 17 ≤ 40)

Açık soru	Nerede karara bağlandı?
Gate-to-Gate 24 saat sarması	02 — `dur_min()` + `ElapsedTime` ile kesin J
T_IB / T_OB bacak sürelerini ayrıştırma	02 — `flight_block_times.csv` (A2 için)
Ayar pencereleri ve gün-içi düzenlilik	03 — `wdev_min = 60`, `xdev_min = 15`, `delta_min = 10`
ρ'su olmayan pazarlar	02 — `missing_rho = "exclude"`; 03 — `extra_use` ile F'de sayılır
Slot pencereleri (Kısıt H)	Kapsam dışı bırakıldı; F kova kapasitesi uygulanır

Son olarak `_buf` içeriği bir markdown kod bloğuna sarılarak `reports/data_health_report.md` dosyasına yazılır — böylece rapor hem düz metin okunabilirliğini korur hem de markdown araçlarıyla DOCX/PDF'e çevrilebilir.

src/01_data_health_check.py · satır 231–232 · standart giriş noktası koruması

```
231 if __name__ == "__main__":
232     main()
```

`if __name__ == "__main__"` kalıbı: dosya doğrudan çalıştırıldığında `main()` koşar, ama başka bir modül tarafından `import` edildiğinde koşmaz. Bu proje bu kalıba her dosyada uyar (07 ve 08, 03'ü tam da bu sayede import edebiliyor).

2 02_data_prep.py — Veri Hazırlama

Bu dosya ne yapar?

Ham Excel'leri modelin doğrudan tüketebileceği **temiz CSV katmanına** çevirir. Yolculuk sürelerini kesinleştirir, 24 saat sarmasını düzeltir, hafta kopyalarını birleştirir, rakip sürelerini ve pazar ağırlıklarını hesaplar. Yaptığı **her dönüşümü** `derived/transform_log.md` içine yazar — yani hiçbir değişiklik gizli kalmaz.

	Değer
Girdi	`data/` — 4 Excel (yine salt-okunur)
Çıktı	`derived/` — 9 CSV + `transform_log.md`
Çalıştırma	`python src/02_data_prep.py`
Bağımlılık	`pandas`, `numpy`
Süre	yaklaşık 10–20 saniye

2.1 Politika: "minimal + belgeli"

`src/02_data_prep.py` · satır 1–14 · dosyanın anayasası

```
"""
TEKNOFEST Havayolu Optimizasyonu — Veri Hazırlama (2. adım)

POLİTİKA: "Minimal + belgeli"
  * Orijinal Excel dosyalarına ASLA dokunulmaz (sadece okunur).
  * Hiçbir değer uydurulmaz.
  * J (Gate-to-Gate) yalnızca FİZİKSEL OLARAK İMKÂNSIZ satırlarda (J < boşluk) +24sa düzeltilir.
  * Belirsiz durumlarda nötr/ihtiyatlı varsayım + açık işaret.
  * Her dönüşüm derived/transform_log.md içine yazılır; ham haliyle de izlenebilir.

Çalıştırma:  python src/02_data_prep.py   (proje kökünden)
Girdi:       data/      (yarışmanın orijinal dosyalarının kopyası — değiştirilmez)
Çıktı:       derived/   (yeni dosyalar)
"""
```

Bu docstring bir kullanım kılavuzu değil, bir **taahhüttür**. Dört maddesi de yarışma bağlamında savunulabilirliğin temelidir:

İlke	Ne demek?	Neden?
Orijinale dokunma	`data/` yalnız `pd.read_excel` ile okunur, asla yazılmaz	Jüri istediği an ham veriyi kontrol edebilir
Değer uydurma	Eksik bir hücre tahminle doldurulmaz	Uydurulan her değer sonucu şişirir ve savunulamaz
Minimal düzeltme	J yalnız **fiziksel olarak imkânsız** satırlarda düzeltilir	"Sanırım hatalıydı" değil, "matematiksel olarak olamaz"
Her şeyi logla	Tüm dönüşümler `transform_log.md`'ye yazılır	İzlenebilirlik = güvenilirlik

2.2 Kurulum — 01 ile aynı kalıp

`src/02_data_prep.py` · satır 16–49 · import'lar, yollar ve dosya bulma

```
import os
import datetime as dt
import warnings
```

```
19
20 import numpy as np
21 import pandas as pd
22
23 warnings.filterwarnings("ignore")
24
25 # proje kökü = bu dosyanın bulunduğu src/ klasörünün üstü
26 HERE = os.path.dirname(os.path.abspath(__file__))
27 ROOT = os.path.dirname(HERE)
28 DATA = os.path.join(ROOT, "data")
29 DERIVED = os.path.join(ROOT, "derived")
30 os.makedirs(DERIVED, exist_ok=True)
31
32 def _find_conn_file():
33     """Bağlantı tablosunu bul: güncellenmiş sürüm (ElapsedTime'lı) öncelikli."""
34     candidates = ["O&D Rakip Bağlantı Tablosu_updated.xlsx",
35                   "O&D Rakip Bağlantı Tablosu.xlsx"]
36     for name in candidates:
37         p = os.path.join(DATA, name)
38         if os.path.exists(p):
39             return p
40     import glob as _glob
41     hits = sorted(_glob.glob(os.path.join(DATA, "O&D*.xlsx")))
42     if hits:
43         return hits[0]
44     raise FileNotFoundError(f"data/ altında O&D bağlantı tablosu bulunamadı: {candidates}")
45
46 F_CONN = _find_conn_file()
47 F_PAX = os.path.join(DATA, "Yolcu Verisi_masked.xlsx")
48 F_RANK = os.path.join(DATA, "change_ranking_input.xlsx")
49 F_PAIR = os.path.join(DATA, "Flight Pairs.xlsx")
```

Bu 34 satır, `01_data_health_check.py`'nin 12–47 satırlarının neredeyse birebir aynısıdır: aynı `__file__` tabanlı yol türetmesi, aynı `_find_conn_file()` önceliklendirmesi, aynı dosya sabitleri. Fark yalnız çıktı klasörüdür — 01 `reports/`, 02 `derived/` üretir.

> **Kod tekrarı bir kusur değil mi?**
>
> Burada bilinçli bir tercih. Her modülün **tek başına** çalışabilmesi ve birbirine bağımlı olmaması isteniyor: 01 çökerse 02 yine koşabilmeli. Paylaşılan bir `utils.py`, modüller arasında gizli bağımlılık yaratırdı ve en kritik yerde — bağımsız doğrulayıcı 05'te — bu bağımlılık **doğrulamayı geçersiz kılardı** (bkz. Bölüm 5.2). Yaklaşık 30 satırlık tekrar, bu bağımsızlığın makul bedelidir.

2.3 Yapılandırma sözlüğü (CFG) ve log

`src/02_data_prep.py` · satır 51–63 · tek yerden yönetilen parametreler

```
51 # ---------------------------- yapılandırma (hepsi belgeli, değiştirilebilir)
52 CFG = {
53     "L_min": 60, "U_min": 300,        # bağlantı penceresi
54     "tau_min": 45,                    # minimum yer süresi
55     "dewrap": "minimal",              # "minimal" (yalnız J<boşluk) | "off"
56     "n_od_def": "carrier",            # "carrier" (veri 17<=40) | "itinerary"
57     "missing_rho": "exclude",         # eksik gelir ağırlığı olan pazarı dışla
58 }
59
60 LOG = []
61 def log(*a):
62     line = " ".join(str(x) for x in a)
63     print(line); LOG.append(line)
```

Sihirli sayılar kodun içine dağıtılmamış, tek bir sözlükte toplanmıştır. Bu, birinin "peki bağlantı penceresini 45–360 yapsak?" sorusuna tek satır değiştirerek cevap verebilmesi demektir. Beş anahtarın anlamı:

Anahtar	Değer	Anlamı
`L_min` / `U_min`	60 / 300	Bağlantı boşluğu penceresi (dakika) — brief'ten
`tau_min`	45	Minimum yer süresi τ; A2 rotasyon kısıtında kullanılır
`dewrap`	"minimal"	24 saat sarması düzeltmesi: yalnız imkânsız satırlarda ("off" = hiç yapma)

Anahtar	Değer	Anlamı
`n_od_def`	"carrier"	Rakip sayısı **taşıyıcı** bazlı sayılır (01'in bulgusu: max 17 ≤ 40)
`missing_rho`	"exclude"	Gelir ağırlığı olmayan pazarlar amaç evreninden çıkarılır

`log()` fonksiyonu 01'deki `out()` ile aynı fikri taşır: hem ekrana bas hem `LOG` listesine biriktir, sonunda dosyaya yaz.

2.4 `dur_min()` — projenin en kritik 27 satırı

`src/02_data_prep.py` · satır 65–98 · tmin ve dur_min

```
65  def tmin(v):
66      """time-of-day -> dakika (saat:dk:sn)."""
67      if isinstance(v, dt.time):
68          return v.hour * 60 + v.minute + v.second / 60.0
69      return np.nan
70
71
72  def dur_min(v):
73      """Süre -> dakika. Excel'in süreyi temsil edebileceği tüm biçimleri kabul eder:
74      time-of-day, 1899-12-30 epoch'lu datetime (24sa+ süreler gün kısmında taşınır),
75      'dd.mm.yyyy HH:MM:SS' / 'HH:MM[:SS]' metni, timedelta, gün-kesri sayısı."""
76      if v is None or (isinstance(v, float) and np.isnan(v)):
77          return np.nan
78      if isinstance(v, dt.datetime):
79          return (v - dt.datetime(1899, 12, 30)).total_seconds() / 60.0
80      if isinstance(v, dt.time):
81          return v.hour * 60 + v.minute + v.second / 60.0
82      if isinstance(v, (pd.Timedelta, dt.timedelta)):
83          return v.total_seconds() / 60.0
84      if isinstance(v, (int, float)):
85          return float(v) * 1440.0  # Excel serisi: 1.0 = 1 gün
86      if isinstance(v, str):
87          s = v.strip()
88          if not s:
89              return np.nan
90          if "." in s.split(" ")[0] and " " in s:  # '30.12.1899 06:19:00'
91              d = dt.datetime.strptime(s, "%d.%m.%Y %H:%M:%S")
92              return (d - dt.datetime(1899, 12, 30)).total_seconds() / 60.0
93          parts = s.split(":")
94          if len(parts) in (2, 3) and all(p.strip().isdigit() for p in parts):
95              h, m = int(parts[0]), int(parts[1])
96              sec = int(parts[2]) if len(parts) == 3 else 0
97              return h * 60 + m + sec / 60.0
98      return np.nan
```

İki fonksiyon var. `tmin()` (65–69) yalnız *saat-of-day* değerlerini çevirir — basit hâl. Asıl iş `dur_min()`'de:

Bu fonksiyon neden bu kadar uzun? Çünkü Excel bir "süre" değerini **altı farklı şekilde** verebilir ve hangisinin geleceği dosyanın nasıl kaydedildiğine bağlıdır:

Gelen tip	Örnek	Dönüşüm	Satır
`datetime`	`1899-12-30 06:19:00`	1899-12-30 epoch'undan fark — **24 saati aşan süreler gün kısmında taşınır**	78–79
`time`	`06:19:00`	saat×60 + dakika + saniye/60	80–81
`timedelta`	`0 days 06:19:00`	`total_seconds()/60`	82–83
`int` / `float`	`0.2632`	Excel seri numarası: 1.0 = 1 gün → ×1440	84–85
`str` (nokta biçim)	`"30.12.1899 06:19:00"`	`strptime` ile ayrıştır, epoch farkı al	90–92
`str` (saat biçim)	`"06:19"` / `"06:19:00"`	`:` ile böl, sayısal doğrula, dakikaya çevir	93–97

Neden bu kadar önemli?

J (toplam yolculuk süresi) rakiple karşılaştırılan **tek** büyüklüktür. J'yi yanlış hesaplarsanız model yanlış rakibi "yenilmiş" sayar, sıralama ödülünü hak etmeden alır ve tüm sonuç geçersiz olur. Bu 27 satır, sonucun doğruluğunu taşıyan

temeldir. Tanımadığı bir biçim gelirse `np.nan` döner — ve bir sonraki blok o NaN'ı yakalayıp **açık hatayla** koşuyu durdurur.

2.5 `main()` ve şema doğrulaması

src/02_data_prep.py · satır 101–111 · yapılandırmanın loglanması ve dört dosyanın okunması

```
101 def main():
102     log("# TEKNOFEST Veri Hazırlama — dönüşüm logu")
103     log("Üretim:", dt.datetime.now().strftime("%Y-%m-%d %H:%M"), "| Politika: minimal+belgeli")
104     log("Yapılandırma:", CFG)
105 
106     conn = pd.read_excel(F_CONN, sheet_name=0)
107     pax = pd.read_excel(F_PAX, sheet_name=0)
108     rank = pd.read_excel(F_RANK, sheet_name=0)
109     pair = pd.read_excel(F_PAIR, sheet_name=0)
110     log(f"\nOkundu (salt-okunur): bağlantı={conn.shape} [{os.path.basename(F_CONN)}], "
111         f"yolcu={pax.shape}, ranking={rank.shape}, pair={pair.shape}")
```

`main()` önce CFG'yi loglar — böylece `transform_log.md` hangi ayarlarla üretildiğini kendi içinde taşır — sonra dört Excel'i okuyup boyutlarını yazar.

src/02_data_prep.py · satır 113–122 · beklenen sütunlar gerçekten var mı?

```
113     # === şema doğrulaması (gizli veride sütun farklıysa kriptik KeyError yerine net hata) ===
114     def need(df, cols, name):
115         miss = [c for c in cols if c not in df.columns]
116         if miss:
117             raise ValueError(f"[ŞEMA] '{name}' beklenen sütun(lar) eksik: {miss}\n"
118                              f"        mevcut sütunlar: {list(df.columns)}")
119     need(conn, ["Dep1", "Arr2", "Cr1", "Dep Time", "Arr Time", "Gate-to-Gate Uçuş Süresi",
120                 "O&D", "Gün", "FlNo1", "FlNo2", "Carrier Name"], "O&D Rakip Bağlantı Tablosu")
121     need(pax, ["Orig Airport Code", "Dest Airport Code", "OD importance factor"], "Yolcu Verisi")
122     need(pair, ["FlNo", "Pair", "Orig", "Dest"], "Flight Pairs")
```

`need()` üç satırlık bir yardımcıdır ama işlevi büyüktür: gizli test verisinde bir sütun adı değişmişse, kod 200 satır sonra anlaşılmaz bir `KeyError: 'Dep1'` vermek yerine **hemen burada** şunu der: "'*O&D Rakip Bağlantı Tablosu*' *beklenen sütun(lar) eksik: ['Dep1'] — mevcut sütunlar: [...]*". Hata mesajı, çözümü de içinde taşır.

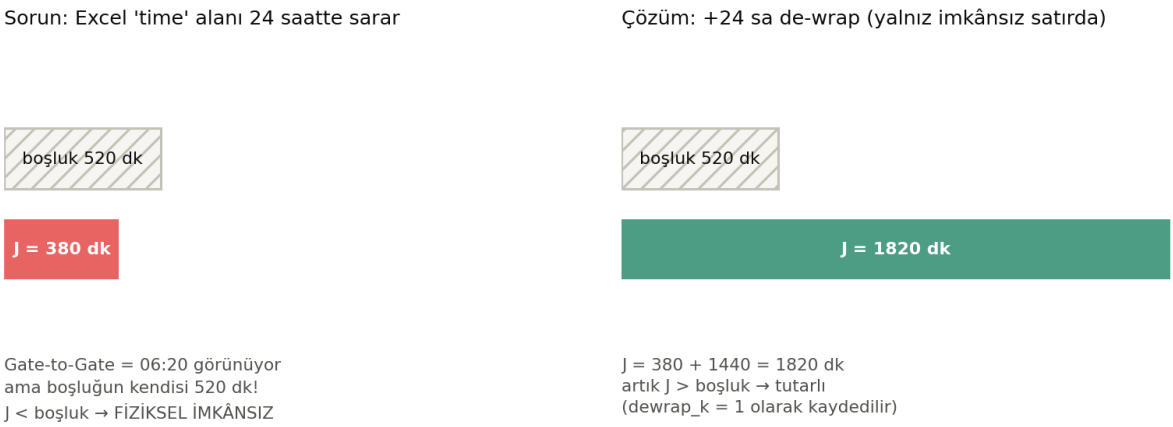
2.6 Ortak türetmeler

src/02_data_prep.py · satır 124–130 · o, d, is_tk, gap_min, J_wrap_min

```
124     # === ortak türetmeler (uydurma yok; tarih/saatten kesin hesap) ============
125     conn = conn.copy()
126     conn["o"] = conn["Dep1"].astype(str)
127     conn["d"] = conn["Arr2"].astype(str)
128     conn["is_tk"] = conn["Cr1"].eq("TK")
129     conn["gap_min"] = (conn["Dep Time"] - conn["Arr Time"]).dt.total_seconds() / 60.0
130     conn["J_wrap_min"] = conn["Gate-to-Gate Uçuş Süresi"].map(dur_min)
```

- `o` ve `d` — pazarın kalkış ve varış havalimanı kodları.
- `is_tk` — bu satır THY'ye mi ait? Tablo boyunca kullanılacak maske.
- `gap_min` — IST'teki bekleme süresi, doğrudan zaman damgalarından **kesin** hesaplanır (burada belirsizlik yok).
- `J_wrap_min` — Gate-to-Gate sütunundan okunan J. "wrap" adı, bu değerin 24 saatte sarmış olabileceğini hatırlatır.

2.7 J hesabı — iki yol, tek doğru sonuç



Şekil 2.1 — Sarma (wrap) problemi ve minimal de-wrap düzeltmesi.

Yol 1 (tercih edilen): ElapsedTime sütunları varsa

`src/02_data_prep.py` · satır 132–155 · J'nin kesin hesabı ve doğrulanması

```
132     # === J hesabı ============================================================
133     # Güncellenmiş veri (ElapsedTime1/2 sütunlu): J = E1 + boşluk + E2 KESİN olarak
134     # hesaplanır — Gate-to-Gate'in 24 saatte sarma (wrap) belirsizliği tamamen kalkar.
135     # Eski şema (ElapsedTime yok): J yalnız Gate-to-Gate'ten gelir; fiziksel imkânsız
136     # (J < boşluk) satırlara minimal +24sa de-wrap uygulanır.
137     has_elapsed = {"ElapsedTime1", "ElapsedTime2"}.issubset(conn.columns)
138     if has_elapsed:
139         conn["E1_min"] = conn["ElapsedTime1"].map(dur_min)
140         conn["E2_min"] = conn["ElapsedTime2"].map(dur_min)
141         n_bad = int(conn["E1_min"].isna().sum()) + int(conn["E2_min"].isna().sum())
142         if n_bad:
143             raise ValueError(f"[ŞEMA] ElapsedTime1/2 çözümlenemeyen hücre: {n_bad} "
144                              f"(örnek: {conn.loc[conn['E1_min'].isna(), 'ElapsedTime1'].head(3).tolist()})")
145         conn["legsum_min"] = conn["E1_min"] + conn["E2_min"]
146         conn["J_min"] = conn["legsum_min"] + conn["gap_min"]
147         conn["dewrap_k"] = np.round((conn["J_min"] - conn["J_wrap_min"]) / 1440.0).astype(int)
148 
149         # doğrulama: kesin J, sarılı Gate-to-Gate ile mod-24sa tutarlı olmalı
150         mod_off = ((conn["J_min"] - conn["J_wrap_min"]) % 1440.0)
151         inconsistent = ~(np.isclose(mod_off, 0, atol=1.01) | np.isclose(mod_off, 1440, atol=1.01))
152         log(f"\n[J KESİN] ElapsedTime1/2 mevcut -> J = E1 + boşluk + E2 (wrap belirsizliği YOK)")
153         log(f"  Gate-to-Gate ile mod-24sa tutarsız satır: {int(inconsistent.sum())} (0 beklenir)")
154         log(f"  24 saati aşan gerçek yolculuk: {int((conn['J_min'] >= 1440).sum())} satır"
155             f" (eski şemada bunlar sarılı görünüyordu)")
```

Güncellenmiş veride her bacağın süresi ayrı ayrı verildiği için J **hesaplanır**, okunmaz:

- `legsum_min = E1 + E2` — iki bacağın toplam uçuş süresi.
- `J_min = legsum_min + gap_min` — kesin toplam yolculuk. **Sarma belirsizliği tamamen ortadan kalkar.**
- `dewrap_k` — bu kesin J ile Gate-to-Gate sütunu arasındaki fark kaç tam gündür? (Belgeleme amaçlı saklanır.)

149–153 satırları bir **çapraz doğrulamadır**: kesin J ile sarılı Gate-to-Gate arasındaki fark tam olarak 24 saatin katı olmalı. Değilse iki kaynak birbiriyle çelişiyor demektir ve rapor bunu "tutarsız satır" olarak bildirir (beklenen değer 0).

`src/02_data_prep.py` · satır 156–162 · eski sezgiselin ne kaçıracağını ölçer

```
156         # eski minimal de-wrap sezgiseli neyi kaçırırdı? (belgeleme amaçlı)
157         imposs = conn["J_wrap_min"] < conn["gap_min"]
158         k_old = np.where(imposs, np.maximum(np.ceil((conn["gap_min"] - conn["J_wrap_min"]) / 1440.0), 1), 0)
159         j_old = conn["J_wrap_min"] + 1440 * k_old
160         missed = ~np.isclose(j_old, conn["J_min"], atol=1.01)
161         log(f"  Eski minimal de-wrap'in yanlış hesaplayacağı satır: {int(missed.sum())}"
162             f" (TK: {int((conn['is_tk'] & missed).sum())}, rakip: {int((~conn['is_tk'] & missed).sum())})")
```

Bu blok koda hiçbir şey **yapmaz** — sadece belgeler. Sorduğu soru: "*Eğer ElapsedTime olmasaydı ve eski minimal de-wrap sezgiselini kullansaydık kaç satırı yanlış hesaplardık?*" Bu, güncellenmiş veriyi kullanma kararının **sayısal gerekçesidir** ve teknik raporda doğrudan alıntılanabilir.

Yol 2 (yedek): eski şema — minimal de-wrap

`src/02_data_prep.py` · satır 163–182 · yalnız fiziksel olarak imkânsız satırlara +24 saat

```
    else:
        # === J de-wrap (YALNIZ fiziksel imkânsız: J_wrap < boşluk) — eski şema ===
        impossible = conn["J_wrap_min"] < conn["gap_min"]
        if CFG["dewrap"] == "minimal":
            k = np.ceil((conn["gap_min"] - conn["J_wrap_min"]) / 1440.0)
            k = np.where(impossible, np.maximum(k, 1), 0).astype(int)
        else:
            k = np.zeros(len(conn), dtype=int)
        conn["dewrap_k"] = k
        conn["J_min"] = conn["J_wrap_min"] + 1440 * conn["dewrap_k"]
        conn["legsum_min"] = conn["J_min"] - conn["gap_min"]

        n_fixed = int((conn["dewrap_k"] > 0).sum())
        log(f"\n[J de-wrap] ElapsedTime sütunları YOK (eski şema) -> minimal sezgisel.")
        log(f"  Fiziksel imkânsız (J<boşluk) satır: {int(impossible.sum())}  -> +24sa uygulanan: {n_fixed}")
        log(f"  TK düzeltilen: {int((conn['is_tk'] & (conn['dewrap_k']>0)).sum())}"
            f" | Rakip düzeltilen: {int((~conn['is_tk'] & (conn['dewrap_k']>0)).sum())}")
        suspicious = (~impossible) & (conn["J_wrap_min"] >= 1380)
        log(f"  [AÇIK İŞARET] 23s+ görünen ve gizli wrap riski taşıyan (DOKUNULMADI): {int(suspicious.sum())}"
            f"  -> teknik raporda belirtilecek.")
```

ElapsedTime yoksa J yalnız Gate-to-Gate'ten gelir. Bu durumda **tek güvenli düzeltme kuralı** şudur: J < boşluk ise bu satır fiziksel olarak imkânsızdır (toplam yolculuk, sadece bekleme süresinden bile kısa olamaz), demek ki değer sarmıştır. Sarma sayısı `k = ceil((boşluk − J) / 1440)` ile en az kaç gün eklenmesi gerektiğinden hesaplanır.

> **"Açık işaret" satırı (180–182)**
>
> 23 saatten uzun görünen ama J ≥ boşluk olduğu için imkânsız sayılmayan satırlar **dokunulmadan bırakılır** ve sayıları raporlanır. Bu, projedeki ihtiyat prensibinin somut örneğidir: emin olmadığımız yerde veriyi değiştirmiyoruz, sadece riski ilan ediyoruz.

2.8 Hafta normalizasyonu — aynı uçuşun iki kopyası

`src/02_data_prep.py` · satır 184–212 · TK adayları ve hafta kopyalarının birleştirilmesi

```
    # === TK bağlantı adayları ================================================
    tk = conn[conn["is_tk"]].copy()
    tk_out = tk[["o", "d", "O&D", "Gün", "FlNo1", "Arr Time", "FlNo2", "Dep Time",
                 "gap_min", "legsum_min", "J_min", "J_wrap_min", "dewrap_k"]].rename(
        columns={"O&D": "market", "Gün": "day", "Arr Time": "ist_arr", "Dep Time": "ist_dep"})

    # === hafta normalizasyonu ================================================
    # Veride AYNI operasyon günü (Gün=d) farklı pazarlarda FARKLI takvim haftalarına
    # yazılabiliyor (ör. gün 1 kimi satırda 4 Ağu, kiminde 11 Ağu Pazartesi). Aynı uçuş
    # no'nun aynı gündeki hafta-kopyaları modelde AYRI karar olaylarına dönüşür ve
    # tutarsız kaydırma alabilir (aynı fiziksel uçuşa iki farklı saat). Düzeltme:
    # satırın İKİ zaman damgası BİRLİKTE 7k gün geri çekilir (kalkış tarihi o günün
    # kanonik = en erken tarihine oturur) — boşluk ve J değişmez, kopyalar birleşir.
    # Gizli veri güvencesi: 7'nin katı olmayan fark görülürse satır DOKUNULMADAN bırakılır.
    ev_key = lambda df: set(zip(df["FlNo1"], df["day"], df["ist_arr"])) \
                        | set(zip(df["FlNo2"], df["day"], df["ist_dep"]))
    n_ev_before = len(ev_key(tk_out))
    dep_date = tk_out["ist_dep"].dt.normalize()
    canon = dep_date.groupby(tk_out["day"]).transform("min")
    k_days = (dep_date - canon).dt.days
    weekly = (k_days % 7 == 0)
    shift_d = np.where(weekly, k_days, 0)
    tk_out["ist_arr"] = tk_out["ist_arr"] - pd.to_timedelta(shift_d, unit="D")
    tk_out["ist_dep"] = tk_out["ist_dep"] - pd.to_timedelta(shift_d, unit="D")
    n_ev_after = len(ev_key(tk_out))
    log(f"\n[Hafta normalizasyonu] geri çekilen satır: {int((shift_d > 0).sum())} "
        f"(7'nin katı olmayan, dokunulmayan: {int((~weekly).sum())})")
```

```
    log(f"  Tekil (uçuş,gün,zaman) olay: {n_ev_before} -> {n_ev_after} "
        f"(birleşen hafta-kopyası: {n_ev_before - n_ev_after}; boşluk/J değişmedi)")
```

Veride ilginç bir tuzak var: **aynı operasyon günü** (örneğin "Gün 1") farklı pazarlarda farklı takvim haftalarına yazılmış olabilir. Bir satırda 4 Ağustos Pazartesi, başka satırda 11 Ağustos Pazartesi. İkisi de "gün 1"dir ama zaman damgaları bir hafta farklıdır.

Bu neden bir sorundur?

Model, karar birimlerini `(uçuş no, rol, saat)` anahtarıyla oluşturur. Aynı fiziksel uçuş iki farklı takvim tarihine yazılmışsa **iki ayrı karar olayı** doğar ve çözücü aynı uçuşa iki farklı yeni saat verebilir. Bu, uygulanamaz bir tarifedir.

Düzeltme (198–207) cerrahi bir müdahaledir:

1. Her satırın **kalkış tarihi** alınır (`ist_dep.dt.normalize()`).
2. Her operasyon günü için o günün **kanonik** (en erken) tarihi bulunur.
3. Fark tam olarak 7'nin katıysa satırın **iki** zaman damgası birlikte geri çekilir. **İkisi birlikte** olduğu için boşluk ve J hiç değişmez.
4. 7'nin katı değilse satıra **dokunulmaz** — bilmediğimiz bir düzen olabilir.

`ev_key` lambda'sı düzeltmeden önce ve sonra benzersiz olay sayısını ölçer; log "birleşen hafta-kopyası: N" satırıyla etkinin büyüklüğünü belgeler.

src/02_data_prep.py · satır 214–228 · birebir özdeş adayların tekilleştirilmesi

```
    # === aday tekilleştirme ================================================
    # Hafta-kopyaları birleşince AYNI fiziksel bağlantı (pazar, gün, uçuş çifti, saatler)
    # birebir özdeş iki satır olarak kalabiliyor. İki kopyayı iki ayrı aday saymak, aynı
    # ürüne çift aktarma ödülü yazdırır (hem baseline hem optimize tarafını şişirir).
    # Yalnız TÜM alanları özdeş satırlar atılır; saatleri gerçekten farklı olan (veri
    # anomalisi) satırlar korunur — model onları ayrı olaylar olarak ele alır.
    n0 = len(tk_out)
    tk_out = tk_out.drop_duplicates(
        ["market", "day", "FlNo1", "FlNo2", "ist_arr", "ist_dep"]).reset_index(drop=True)
    log(f"  Birebir özdeş aday satırı tekilleştirildi: {n0 - len(tk_out)} "
        f"({n0} -> {len(tk_out)} aday)")

    tk_out["gap_in_window"] = tk_out["gap_min"].between(CFG["L_min"], CFG["U_min"])
    log(f"\n[TK bağlantı] {len(tk_out)} aday | pencere içi: {int(tk_out['gap_in_window'].sum())}"
        f" ({100*tk_out['gap_in_window'].mean():.1f}%)")
```

Hafta kopyaları birleşince **tamamen özdeş iki satır** kalabilir. İkisini iki ayrı aday saymak, aynı ürüne çifte aktarma ödülü yazdırır — bu hem baseline'ı hem optimize sonucunu şişirir, yani sonuç *sahte* olur. `drop_duplicates` yalnız **tüm alanları** özdeş olanları atar; saatleri gerçekten farklı olan satırlar (veri anomalisi) korunur ve model onları ayrı olaylar olarak ele alır.

2.9 Bacak blok süreleri — A2 rotasyonunun yakıtı

src/02_data_prep.py · satır 230–248 · her (uçuş, gün, rol) için blok süresi

```
    # === TK bacak blok süreleri (A2 rotasyonu için; yalnız güncel şemada) =====
    # Brief §4.1: dönüş IST varışı, gidiş IST kalkışından en az (T_OB + τ + T_IB)
    # sonra olmalı. ElapsedTime'lı veri T_IB (FlNo1: o→IST) ve T_OB (FlNo2: IST→d)
    # süresini uçuş-örneği düzeyinde verir. Aynı (FlNo,gün) için birden çok değer
    # görülürse İHTİYATLI olan (en büyük blok) alınır — tur asla brief minimumunun
    # altına sıkıştırılamaz.
    blocks = None
    if has_elapsed:
        ib = (tk.groupby(["FlNo1", "Gün"], as_index=False)["E1_min"].max()
                .rename(columns={"FlNo1": "flno", "Gün": "day", "E1_min": "block_min"}))
        ib["role"] = "arr"
        obt = (tk.groupby(["FlNo2", "Gün"], as_index=False)["E2_min"].max()
                 .rename(columns={"FlNo2": "flno", "Gün": "day", "E2_min": "block_min"}))
        obt["role"] = "dep"
        n_multi = int((tk.groupby(["FlNo1", "Gün"])["E1_min"].nunique() > 1).sum()) \
                + int((tk.groupby(["FlNo2", "Gün"])["E2_min"].nunique() > 1).sum())
        blocks = pd.concat([ib, obt], ignore_index=True)[["flno", "day", "role", "block_min"]]
```

```
247         log(f"\n[Blok süreleri] TK bacak blokları (A2 için): {len(blocks)} (FlNo,gün,rol) kaydı"
248             f" | çok-değerli grup (max alındı): {n_multi}")
```

A2 kısıtı "dönüş varışı ≥ gidiş kalkışı + T_OB + τ + T_IB" der. T_OB ve T_IB'yi buradan üretiyoruz: `FlNo1`'in E1'i gelen bacağın (arr) bloğu, `FlNo2`'nin E2'si giden bacağın (dep) bloğudur.

> **Neden `.max()`?**
>
> Aynı `(uçuş no, gün)` için birden çok farklı blok süresi görülürse **en büyüğü** alınır. Böylece minimum tur süresi asla olduğundan kısa hesaplanmaz — yani tur, brief'in gerektirdiği minimumun altına **sıkıştırılamaz**. Kuşkulu durumda ihtiyatlı tarafta kalma tercihi.

2.10 Rakip itineraryleri ve N_od

`src/02_data_prep.py` · satır 250–268 · T_comp ve rakip sayısı

```
250     # === Rakip itineraryleri + (carrier bazlı) T_comp ========================
251     comp = conn[~conn["is_tk"]].copy()
252     comp_cols = ["o", "d", "O&D", "Gün", "Cr1", "Carrier Name", "J_min", "dewrap_k"]
253     if "ML2" in comp.columns:   # güncel veri: M=Mainline, L=Low-cost (bilgi amaçlı taşınır;
254         comp_cols.append("ML2")  # brief'in sıralama tanımı taşıyıcı tipi ayrımı yapmaz)
255         log(f"\n[ML2] Taşıyıcı tipi sütunu mevcut: {comp['ML2'].value_counts().to_dict()}"
256             f" (M=Mainline, L=Low-cost; ranking'e etkisiz, bilgi amaçlı taşındı)")
257     comp_it = comp[comp_cols].rename(
258         columns={"O&D": "market", "Gün": "day", "Cr1": "carrier"})
259     tcomp_carrier = (comp_it.groupby(["market", "day", "carrier"], as_index=False)["J_min"]
260                      .min().rename(columns={"J_min": "T_comp_min"}))
261     if CFG["n_od_def"] == "carrier":
262         n_od = tcomp_carrier.groupby(["market", "day"], as_index=False)["carrier"].nunique() \
263                             .rename(columns={"carrier": "N_od"})
264     else:
265         n_od = comp_it.groupby(["market", "day"], as_index=False).size().rename(columns={"size": "N_od"})
266     log(f"\n[Rakip] {len(comp_it)} itinerary | taşıyıcı-bazlı T_comp satırı: {len(tcomp_carrier)}")
267     log(f"  N_od ({CFG['n_od_def']}) aralığı: {int(n_od['N_od'].min())}–{int(n_od['N_od'].max())}"
268         f" | ranking lookup max N=40 -> kapsam {'TAM' if n_od['N_od'].max()<=40 else 'EKSİK'}")
```

İki türetme:

- **`tcomp_carrier`** — her (pazar, gün, taşıyıcı) için **en küçük J**. Yani "bu rakip bu pazarda en iyi ne yapabiliyor?" Modelin yeneceği eşik budur.
- **`n_od`** — pazardaki rakip sayısı. `n_od_def = "carrier"` olduğu için `nunique()` ile farklı **taşıyıcı** sayılır.

Satır 267–268 kapsamı doğrular: N_od aralığı ödül tablosunun desteklediği max N = 40'ı aşıyor mu? Log "kapsam TAM" veya "EKSİK" yazar — bu, 01'de sorulan açık sorunun kapanış kaydıdır.

`ML2` sütunu (Mainline / Low-cost) varsa bilgi amaçlı taşınır ama modele girmez; brief'in sıralama tanımı taşıyıcı tipi ayrımı yapmaz.

2.11 Pazar tablosu — ρ, kapsam ve dışlama

`src/02_data_prep.py` · satır 270–286 · markets.csv'nin oluşumu

```
270     # === Pazar tablosu: ρ (gelir ağırlığı) + N_od ============================
271     pax = pax.copy()
272     pax["market"] = pax["Orig Airport Code"].astype(str) + "-" + pax["Dest Airport Code"].astype(str)
273     pax = pax.drop_duplicates("market")
274     rho = pax.set_index("market")["OD importance factor"]
275
276     markets = pd.DataFrame({"market": sorted(conn["O&D"].unique())})
277     markets["o"] = markets["market"].str.split("-").str[0]
278     markets["d"] = markets["market"].str.split("-").str[1]
279     markets["rho"] = markets["market"].map(rho)
280     markets["has_tk"] = markets["market"].isin(set(tk_out["market"]))
281     markets["has_comp"] = markets["market"].isin(set(comp_it["market"]))
282     n_missing_rho = int(markets["rho"].isna().sum())
283     markets["excluded_no_rho"] = markets["rho"].isna() if CFG["missing_rho"] == "exclude" else False
284     log(f"\n[Pazar] {len(markets)} yönlü pazar | ρ eksik: {n_missing_rho}"
285         f" -> politika='{CFG['missing_rho']}' (dışlanan: {int(markets['excluded_no_rho'].sum())})")
286     log(f"  TK olan: {int(markets['has_tk'].sum())} | sadece-rakip: {int((~markets['has_tk']).sum())}")
```

Her yönlü pazar için tek satır üretilir ve dört bilgi eklenir:

Sütun	Anlamı	Nerede kullanılır?
`rho`	Gelir/önem ağırlığı	Amaç fonksiyonu çarpanı (03)
`has_tk`	Bu pazarda TK bağlantısı var mı?	Alt-örnek seçimi (03)
`has_comp`	Rakip var mı?	Sıralama teriminin tanımlı olup olmadığı
`excluded_no_rho`	ρ'su yok → amaç evreni dışı	03'te dışlanır, ama uçuşları F kapasitesinde sayılır

2.12 Rotasyon kapsamı

`src/02_data_prep.py` · satır 288–295 · partneri olmayan TK uçuşları

```
288     # === Rotasyon (Flight Pairs) — eksik partner = ayarlanamaz işareti =========
289     pair = pair.copy()
290     tk_flnos = set(tk["FlNo1"]).union(set(tk["FlNo2"]))
291     pair_flnos = set(pair["FlNo"])
292     no_partner = sorted(tk_flnos - pair_flnos)
293     rot = pair[pair["FlNo"].isin(tk_flnos)].copy()
294     log(f"\n[Rotasyon] TK uçuş no: {len(tk_flnos)} | Flight Pairs'te eşi olan: {len(tk_flnos & pair_flnos)}")
295     log(f"  Partnersiz TK uçuş no: {len(no_partner)} -> politika: 'ayarlanamaz/sabit' kabul (kısıt eklenmez)")
```

`Flight Pairs`'te karşılığı olmayan TK uçuş numaraları ayrı bir listeye çıkarılır. 03 bu listeyi `fixed_flnos` olarak okur ve o uçuşları **hiç kımıldatmaz** (`cands = [cur]`). Rotasyonunu bilmediğimiz uçağa dokunmama ilkesi.

2.13 `derived/` katmanının yazılması

`src/02_data_prep.py` · satır 297–317 · 9 CSV + dönüşüm logu

```
297     # === yaz (derived/) ========================================================
298     tk_out.to_csv(os.path.join(DERIVED, "tk_connections.csv"), index=False, encoding="utf-8-sig")
299     comp_it.to_csv(os.path.join(DERIVED, "competitor_itineraries.csv"), index=False, encoding="utf-8-sig")
300     tcomp_carrier.to_csv(os.path.join(DERIVED, "competitor_times_carrier.csv"), index=False, encoding="utf-8-
sig")
301     n_od.to_csv(os.path.join(DERIVED, "market_n_od.csv"), index=False, encoding="utf-8-sig")
302     markets.to_csv(os.path.join(DERIVED, "markets.csv"), index=False, encoding="utf-8-sig")
303     rank.to_csv(os.path.join(DERIVED, "ranking_lookup.csv"), index=False, encoding="utf-8-sig")
304     rot.to_csv(os.path.join(DERIVED, "flight_rotation.csv"), index=False, encoding="utf-8-sig")
305     if blocks is not None:
306         blocks.to_csv(os.path.join(DERIVED, "flight_block_times.csv"), index=False, encoding="utf-8-sig")
307     pd.DataFrame({"flno_no_rotation_partner": no_partner}).to_csv(
308         os.path.join(DERIVED, "tk_flights_without_partner.csv"), index=False, encoding="utf-8-sig")
309
310     with open(os.path.join(DERIVED, "transform_log.md"), "w", encoding="utf-8") as f:
311         f.write("\n".join(LOG) + "\n")
312
313     log("\n[OK] derived/ altına yazıldı:")
314     for fn in sorted(os.listdir(DERIVED)):
315         p = os.path.join(DERIVED, fn)
316         log(f"   {fn:38s} {os.path.getsize(p)//1024:>5d} KB")
317     log("\nKAYNAK DOSYALAR DEĞİŞMEDİ. Tüm dönüşümler bu logda.")
```

`encoding="utf-8-sig"` seçimi bilinçlidir: BOM'lu UTF-8, Excel'in Türkçe karakterleri doğru açmasını sağlar (jüri CSV'yi çift tıklayıp açtığında "BaÄŸlantÄ±" görmemeli). Son olarak `transform_log.md` yazılır ve log dosya listesini boyutlarıyla basar.

Dosya	İçerik	Kim kullanır?
`tk_connections.csv`	TK bağlantı adayları (pazar, gün, uçuşlar, saatler, J)	03, 05, 08
`competitor_itineraries.csv`	Rakiplerin tüm bağlantı seçenekleri	referans / rapor
`competitor_times_carrier.csv`	Taşıyıcı başına en iyi J (T_comp)	03, 05, 08
`market_n_od.csv`	(pazar, gün) → rakip taşıyıcı sayısı N	03, 05
`markets.csv`	Pazar listesi + ρ + kapsam bayrakları	03, 05
`ranking_lookup.csv`	Sıralama ödül tablosu (N, b, s, W)	03, 05, 08

Dosya	İçerik	Kim kullanır?
`flight_rotation.csv`	Uçak rotasyon eşleri	03, 05
`flight_block_times.csv`	(uçuş, gün, rol) → blok süresi	03, 05, 08
`tk_flights_without_partner.csv`	Rotasyon partneri olmayan uçuşlar	03 (sabit kabul)
`transform_log.md`	Yapılan tüm dönüşümlerin kaydı	jüri / izlenebilirlik

Kapanış satırı bir taahhüdü tekrar eder: **"KAYNAK DOSYALAR DEĞİŞMEDİ. Tüm dönüşümler bu logda."**

3 03_model.py — MIP Modeli (projenin çekirdeği)

Bu dosya ne yapar?

`derived/` katmanını okur, onu bir **Karışık Tamsayılı Programlama (MIP)** modeline çevirir, HiGHS çözücüsüyle çözer ve sonucu `output/` altına yazar. 1357 satırın tamamı tek bir soruya hizmet eder: *hangi uçuşun IST saati kaç dakika kaydırılırsa, tüm kısıtlar korunarak gelir-ağırlıklı rekabetçilik en yükseğe çıkar?*

	Değer
Girdi	`derived/` — 8 CSV
Çıktı	`output/optimized_schedule.csv`, `market_results.csv`, `selected_connections.csv`, `g_band_exemptions.csv`, `results_summary.md`
Çalıştırma	`python src/03_model.py byday` (tam veri, teslim modu)
Diğer modlar	`(argümansız)` küçük alt-örnek · `full` tek-MIP denemesi · `sweep` λ taraması
Bağımlılık	`pandas`, `numpy`, `pyomo`, `highspy`
Süre	byday ≈ 5–10 dk (gün başına ≤ 150 s)

Bu bölüm uzundur çünkü dosya uzundur. Kolay takip için sırayı şöyle kuruyoruz: önce **ayarlar**, sonra **veriyi modele hazırlama**, sonra **kısıt kısıt model kurma**, sonra **çözme**, en sonda **haftayı güne bölerek koşturma ve çıktı yazma**.

3.1 Docstring — modelin özeti ve alınan ölçekleme kararları

`src/03_model.py` · satır 1–20 · uygulanan kısıtların listesi

```
"""
TEKNOFEST Havayolu Optimizasyonu — MIP Modeli (3. adım) · Pyomo + HiGHS

Bu betik `docs/model_formulation.md` formülasyonunun TAM implementasyonudur.
Küçük alt-örnekte tüm kısıtlarla elle doğrulanır; tam veride (`full` profili) ölçeklenir.

Uygulanan kısıtlar (hepsi CFG ile açılır/kapanır):
  A1  Tekil zaman atama (10 dk ızgarada, pencere ±Wdev)              [docs §5-A]
  A2  Uçak rotasyonu — min tur = T_OB + τ + T_IB (brief-exact) (*)   [docs §5-A2]
  B1  Bağlantı uygunluk göstergesi  ℓ ≤ boşluk ≤ u  ⟺  y_π          [docs §5-B]
  B2  Uygun bağlantı zorunlu sunulur — VARSAYILAN KAPALI (*)         [docs §5-B2]
  C   Azalan-getiri / monoton slot ödülü (aktarma sayısı)            [docs §5-C]
  D   Rakip yenme (w, β) + sıralama ödülü (γ)                        [docs §5-D]
  E1  Yönsel SAYI dengesi                                            [docs §5-E1]
  E2  En iyi yolculuk süresi (Jmin) dengesi — tam-ölçekte KAPALI (*) [docs §5-E2]
  F   IST hub kova kapasitesi — brief'in gerçek 10/15 rezidüel cap'i  [docs §5-F]
  G   Tarife düzenliliği — SAAT-İÇİ referans-bant c_f (*)            [docs §5-G]
  P   Tarife-değişim cezası — λ·Σ|θ−cur| (tam doğrusal, ops.)        [docs §12]
  Amaç:  Σ ρ·(aktarma ödülü + sıralama ödülü) − λ·(toplam kaydırma)  [docs §4/§12]

```

İlk 20 satır, `docs/model_formulation.md` ile birebir eşleşen bir **kısıt envanteridir**. Yıldızlı (*) olanlar tam ölçekte özel muamele görür — nedenleri hemen altında yazılıdır:

`src/03_model.py` · satır 21–44 · ölçekleme kararları ve çalıştırma modları

```
Uygulama/ölçekleme kararları (*) — bkz. docs §10 (uygulama notları):
  (A2) Güncel veri bacak bloklarını (ElapsedTime1/2) verir → brief-exact T_OB+τ+T_IB kurulur;
       mevcut tur brief minimumunun altında görünen çiftler gece-aşan rotasyondur → mevcut tur
       çapası korunur (sep = min(exact, cur)); bloklar yoksa (eski şema) mevcut tur çapası.
  (B2) B2×E1 ÇATIŞMASI: B2 her uygun bağlantıyı zorlar; asimetrik pazarlarda E1 dengesiyle
       infeasible (bisection ile doğrulandı). Amaç zaten sunmayı ödüllendirir → B2 kapalı.
  (E2) Jmin min-seçimi büyük-M'i HiGHS'te incumbent buldurmuyor (tanı: çekirdek+E2 tek başına
       çözülemez). Küçük-ölçekte açık/doğrulanmış; `full` profilinde kapalı.
```

```
  (F)  Brief'in gerçek 10/15'i UYGULANIR: mevcut tarifedeki kova aşımları yalnız ayarlanabilir
       uçuşların kümelenmesinden; SABİT uçuşlar baz cap'i hiç aşmaz (kalkış ≤4, varış ≤5) → rezidüel
       cap ≥0, ayarlanabilirler pencerede yayılarak sığar. 7/7 gün feasible (warm-start gerekmez).
  (G)  Düzenlilik SAAT-İÇİ; grup-başına ortak bant c_f (yayılım ≤ Xdev); pencerede
       düzenlenemeyenler atlanır/raporlanır.

NOT (docs §9): sıralama ödülünde b_od, baseline 'son sırası' (s = N − yenilen, [1,N]) ile
BİREBİR aynı tabandan hesaplanır → mevcut tarife her pazarda lookup köşegenine (b==s = 'değişim
yok' nötr referansı, değeri 1/(N−1)) oturur (sıralama bir İYİLEŞTİRME ödülü). Kesin türetim
konvansiyonu organizatör teyidine tabi; model iç-tutarlı.

Çalıştırma:  python src/03_model.py            (küçük alt-örnek, tüm kısıtlar)
             python src/03_model.py full       (tam veri tek-MIP denemesi, E2 kapalı)
             python src/03_model.py byday [λ]  (güne-göre ayrıştırma + çıktı; ops. ceza λ)
             python src/03_model.py sweep      (λ ödünleşim taraması — Pareto tablo)
"""
```

Bu 24 satır bir yazılım dokümantasyonundan çok bir **mühendislik günlüğüdür**. Her karar, denendiği ve neden o şekilde bırakıldığıyla birlikte yazılmış:

Karar	Ne yapıldı?	Gerekçe
A2	Bloklar varsa brief-exact minimum tur; yoksa mevcut tur çapası	Gece-aşan turlarda mevcut desen korunur, gün-bağlaşımı yaratılmaz
B2	Varsayılan **kapalı**	B2 her uygun bağlantıyı zorlar; asimetrik pazarlarda E1 dengesiyle **infeasible** olur (bisection ile doğrulandı). Amaç zaten sunmayı ödüllendirir
E2	Tam ölçekte **kapalı**	Jmin min-seçimi büyük-M'i HiGHS'te incumbent buldurmuyor; küçük ölçekte açık ve doğrulanmış
F	Brief'in gerçek 10/15 kapasitesi **uygulanır**	Veri bulgusu: sabit uçuşlar baz kapasiteyi hiç aşmıyor → rezidüel kapasite ≥ 0
G	Saat-içi, grup başına ortak bant	Pencerede ortak bant kurulamayanlar atlanır ve raporlanır

Bu neden değerli?

Bir jüri "neden E2'yi kapattınız?" diye sorduğunda cevap kodun içinde, tanı yöntemiyle birlikte yazılıdır: "*çekirdek + E2 tek başına çözülemiyor; A2/G/F/E1 hepsi hızlı*". Bu, kararın keyfî değil **ölçülmüş** olduğunu gösterir.

3.2 Ayarlar: CFG ve PRESETS

`src/03_model.py` · satır 46–62 · import'lar ve konsol kodlaması

```
import os
import sys
import datetime as dt
import numpy as np
import pandas as pd
import pyomo.environ as pyo

# Windows konsolu (cp1254) π/Δ gibi karakterleri basamıyor -> UTF-8'e geç
try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

# ---------------------------------------------------------------------------- yollar
HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
DERIVED = os.path.join(ROOT, "derived")
```

`sys.stdout.reconfigure(encoding="utf-8")` küçük ama gerekli: Windows konsolu varsayılan olarak cp1254 kullanır ve π, Δ, ≤ gibi karakterlerde çöker. `try/except` içine alınmıştır çünkü bazı ortamlarda bu metot yoktur — kod o durumda da çalışmaya devam eder.

`src/03_model.py` · satır 65–102 · merkezî yapılandırma sözlüğü

```
CFG = {
    # --- alt-örnek seçimi ----------------------------------------------
```

```
    "days": [1],            # hangi operasyon günleri
    "markets": None,        # açık pazar listesi (None ise ρ'ya göre otomatik seçim)
    "top_k_markets": 6,     # ρ'ya göre en yüksek kaç (yön) pazar
    "require_competitors": True,  # yalnız ≥1 rakibi olan pazarları seç (sıralama terimini sınamak için)
    "prefer_improvable": True,    # baseline'da pencere-dışı (iyileştirilebilir) bağlantısı olan pazarları öne
al
    "add_reverse": True,    # seçilen pazarların ters yönünü de ekle (E1 için)
    # --- zaman ızgarası ve pencere ---------------------------------------
    "delta_min": 10,        # Δ kova genişliği (dk)
    "wdev_min": 60,         # ayarlanabilir uçuş için ± pencere (dk)  [docs D3]
    "L_min": 60, "U_min": 300,    # bağlantı boşluğu [ℓ,u] (dk)
    # --- amaç / kısıt anahtarları ----------------------------------------
    "ranking": True,        # sıralama ödülü terimi (geçici b_od kuralı, docs §9)
    "shift_penalty": 0.0,   # (P) tarife-değişim cezası λ: amaç-puanı / uçuş-dk (0 = kapalı)
    "alpha": 0.20,          # E1 yönsel dengesizlik eşiği
    "tau_min": 45,          # τ — minimum yer süresi (dk, brief §2.4; A2 rotasyonunda)
    "gamma_min": 30,        # Γ — E2 en iyi yolculuk süresi fark eşiği (dk)
    "xdev_min": 15,         # G — aynı uçuşun günler arası maks. IST sapması (dk, docs §5-G)
    "E1_balance": True,     # (E1) yönsel SAYI dengesi
    "E2_balance": True,     # (E2) en iyi yolculuk süresi dengesi (Jmin)
    # (B2) VARSAYILAN KAPALI: B2 her uygun bağlantıyı sunmaya zorlar; tam veride asimetrik
    # pazarlarda (bir yön çok, ters yön az bağlantı) E1 dengesiyle ÇATIŞIR → infeasible.
    # Amaç zaten sunmayı ödüllendirdiğinden B2 olmadan da model dengeyi sunum SEÇEREK
    # sağlar (doc §5-B2 notu: azaltılmış modelde B2 gevşetilebilir). Bisection ile doğrulandı.
    "B2_force_offer": False,
    "A2_rotation": True,    # (A2) uçak rotasyonu: min tur = T_OB+τ+T_IB (bloklar varsa; yoksa mevcut tur)
    "A2_overnight_guard": True,  # (A2×) gece-aşan turlara statik slack-bölüşümü sınırı (byday)
    "G_regularity": True,   # (G) aynı uçuş no'nun günler arası IST tutarlılığı
    "F_capacity": True,     # (F) IST hub kova kapasitesi
    "cap_dep": 10,          # kova başına maks. (ayarlanabilir+sabit) kalkış  [docs §2]
    "cap_arr": 15,          # kova başına maks. varış
    # --- çözücü ----------------------------------------------------------
    "solver": "appsi_highs",
    "time_limit": None,     # saniye (None = sınırsız); tam veride önerilir
    "mip_gap": None,        # bağıl MIP boşluğu (None = çözücü varsayılanı)
    "warmstart": False,     # appsi Highs MIP-start'ı kullanmıyor (yalnız LP bazı) — etkisiz
}
```

CFG dört gruba ayrılmıştır. En önemli anahtarlar:

Anahtar	Varsayılan	Ne yapar?
`delta_min`	10	Zaman ızgarasının adımı Δ. Saatler 10 dakikanın katı olur
`wdev_min`	60	Bir uçuş mevcut saatinden en fazla ±60 dk kaydırılabilir
`L_min` / `U_min`	60 / 300	Bağlantı boşluğu penceresi
`shift_penalty`	0.0	λ — tarife-değişim cezası (byday'de 0.3 verilir)
`alpha`	0.20	E1 yönsel dengesizlik toleransı
`tau_min`	45	τ — minimum yer süresi (A2)
`xdev_min`	15	G — günler arası maksimum saat sapması
`cap_dep` / `cap_arr`	10 / 15	F — kova başına kalkış/varış kapasitesi
`B2_force_offer`	False	B2 kapalı (E1 ile çatışıyor)
`E2_balance`	True	Küçük örnekte açık; `full` profilinde kapatılır
`solver`	`appsi_highs`	Pyomo'nun HiGHS arayüzü
`time_limit` / `mip_gap`	None / None	Profil bunları doldurur

src/03_model.py · satır 104–116 · "full" profili

```
# Hazır profiller: CFG.update(PRESETS["full"]) ile tüm veriye geç.
PRESETS = {
    "full": {                       # TÜM gün/pazar — ölçekleme koşusu
        "days": None,               # None = veride mevcut TÜM günler (hardcode etme; gizli veri uyumu)
        "markets": None, "top_k_markets": None,
        "require_competitors": False, "prefer_improvable": False,
        # E2 (Jmin zaman dengesi) tam-ölçekte KAPALI: min-seçimi büyük-M'i HiGHS'te
        # incumbent bulunamaz kılıyor (tanı: çekirdek+E2 tek başına çözülemiyor; A2/G/F/E1
        # hepsi hızlı). Küçük-ölçekte E2 açık ve doğrulanmış kalır. Bkz. docs §10.
        "E2_balance": False,
        "time_limit": 600, "mip_gap": 0.01,
    },
}
```

PRESETS, CFG'yi tek satırda topluca değiştirir: `CFG.update(PRESETS["full"])`. Dikkat edilecek nokta **`"days": None`**'dur:

> **Gün listesi neden hardcode edilmiyor?**
>
> Gizli test verisi 7 gün yerine 5 veya 10 gün içerebilir. `days=None` demek "veride ne varsa onu kullan" demektir; gerçek liste `all_operation_days()` ile `tk_connections.csv`'den okunur. Aynı ilke `markets`, `top_k_markets` için de geçerlidir — `None` = sınırlama yok.

3.3 Veriyi okumak

`src/03_model.py` · satır 120–125 · operasyon günlerini veriden türet

```python
def all_operation_days():
    """Veride mevcut TÜM operasyon günleri. Gün listesini HARDCODE etmemek için (gizli
    test verisi farklı sayıda/etikette güne sahip olabilir) tk_connections.csv'den okunur."""
    df = pd.read_csv(os.path.join(DERIVED, "tk_connections.csv"),
                     encoding="utf-8-sig", usecols=["day"])
    return sorted(int(d) for d in df["day"].dropna().unique())
```

`src/03_model.py` · satır 128–148 · load_data — sekiz CSV'yi tek çağrıda

```python
def load_data():
    rd = lambda f, **k: pd.read_csv(os.path.join(DERIVED, f), encoding="utf-8-sig", **k)
    markets = rd("markets.csv")
    nod = rd("market_n_od.csv")
    tk = rd("tk_connections.csv", parse_dates=["ist_arr", "ist_dep"])
    tcomp = rd("competitor_times_carrier.csv")
    rank = rd("ranking_lookup.csv")
    if rank.shape[1] != 4:                          # beklenen şema: N, ilk-rank, son-rank, ağırlık
        raise ValueError(f"ranking_lookup.csv 4 sütun bekler, {rank.shape[1]} bulundu "
                         f"({list(rank.columns)}) — sıralama ödülü tablosu şeması beklenenden farklı.")
    rank.columns = ["N", "b", "s", "W"]             # konumsal sabitleme (Rakip Sayısı/İlk/Son/Weight)
    nopart = rd("tk_flights_without_partner.csv")["flno_no_rotation_partner"].tolist()
    rot = rd("flight_rotation.csv")                 # FlNo, Orig, Dest, Pair
    # bacak blok süreleri (güncel şema; A2 için brief-exact minimum tur) — yoksa None
    blk_path = os.path.join(DERIVED, "flight_block_times.csv")
    blocks = None
    if os.path.exists(blk_path):
        bdf = pd.read_csv(blk_path, encoding="utf-8-sig")
        blocks = {(int(r.flno), int(r.day), str(r.role)): float(r.block_min)
                  for r in bdf.itertuples()}
    return markets, nod, tk, tcomp, rank, set(nopart), rot, blocks
```

Dikkat çekici üç ayrıntı:

- **135–138:** Sıralama tablosunun sütun adları farklı gelebilir diye **konumsal** sabitleme yapılır (`rank.columns = ["N","b","s","W"]`), ama önce sütun **sayısı** doğrulanır. Şema beklenenden farklıysa anlaşılır hata verir.
- **144–147:** `flight_block_times.csv` **isteğe bağlıdır**. Yoksa `blocks = None` döner ve A2 kısıtı ihtiyatlı moda (mevcut tur çapası) geçer. Eski şemayla uyumluluk böyle korunur.
- **139:** Rotasyon partneri olmayan uçuşlar `set` olarak döner — üyelik testi O(1) olsun diye.

`src/03_model.py` · satır 151–161 · rotation_pairs — hangi gidiş hangi dönüşle eşleşir?

```python
def rotation_pairs(rot):
    """Flight Pairs'ten (IST→o giden, o→IST gelen) uçak rotasyon eşleri.
    Aynı 'Pair' iki uçuş no'yu gruplar; Orig=IST olan giden (dep), Dest=IST olan gelen (arr)."""
    pairs = []
    for _, grp in rot.groupby("Pair"):
        outs = grp[grp["Orig"] == "IST"]["FlNo"].tolist()   # IST→o (kalkış)
        ins = grp[grp["Dest"] == "IST"]["FlNo"].tolist()    # o→IST (varış)
        for of in outs:
            for inf in ins:
                pairs.append((int(of), int(inf)))
    return pairs
```

`Flight Pairs` tablosunda aynı `Pair` etiketini taşıyan uçuşlar **aynı uçağı** paylaşır. Fonksiyon bunları `Orig == "IST"` (giden) ve `Dest == "IST"` (gelen) olarak ayırır ve tüm (giden, gelen) kombinasyonlarını üretir. Çıktı, A2 kısıtının kurulacağı çift listesidir.

`src/03_model.py` · satır 164–201 · select_markets — alt-örnek seçimi

```python
def select_markets(markets, nod, tk, days):
    """ρ'ya göre alt-örnek pazarları seç (has_tk, ρ tanımlı, seçilen günlerde bağlantısı olan)."""
    d = tk[tk["day"].isin(days)]
    mk_with_conn = set(d["market"])
    if CFG["markets"]:                      # açık liste verildiyse onu kullan
        chosen = [m for m in CFG["markets"] if m in mk_with_conn]
        if CFG["add_reverse"]:
            for m in list(chosen):
                o, dd = m.split("-")
                rev = f"{dd}-{o}"
                if rev in mk_with_conn and rev not in chosen:
                    chosen.append(rev)
        return chosen
    cand = markets[(markets["has_tk"]) & (~markets["excluded_no_rho"])
                   & (markets["rho"].notna()) & (markets["market"].isin(mk_with_conn))].copy()
    if CFG["require_competitors"]:
        nd = nod[nod["day"].isin(days)].groupby("market")["N_od"].max()
        has_comp = set(nd[nd >= 1].index)
        cand = cand[cand["market"].isin(has_comp)]
    if CFG["prefer_improvable"]:
        # baseline'da ≥1 bağlantısı pencere dışı (gap ∉ [L,U]) olan pazarları öne al
        gg = d.copy()
        gg["out"] = ~gg["gap_min"].between(CFG["L_min"], CFG["U_min"])
        improvable = set(gg.groupby("market")["out"].any().pipe(lambda s: s[s].index))
        cand["_imp"] = cand["market"].isin(improvable)
        cand = cand.sort_values(["_imp", "rho"], ascending=[False, False])
    else:
        cand = cand.sort_values("rho", ascending=False)
    chosen = list(cand["market"] if CFG["top_k_markets"] is None
                  else cand["market"].head(CFG["top_k_markets"]))
    if CFG["add_reverse"]:
        rho_map = markets.set_index("market")["rho"]
        for m in list(chosen):
            o, dd = m.split("-")
            rev = f"{dd}-{o}"
            if rev in mk_with_conn and rev not in chosen and not pd.isna(rho_map.get(rev, np.nan)):
                chosen.append(rev)
    return chosen
```

Bu fonksiyon esasen **geliştirme/hata ayıklama** içindir: tüm veriyle çalışmadan önce küçük ve anlamlı bir alt-örnekte tüm kısıtları elle doğrulamak için. Tam veri koşusunda (`full` profili) filtrelerin hepsi kapalıdır ve fonksiyon pratik olarak "hepsini seç" der. Yine de mantığı öğreticidir:

1. Aday havuzu: TK bağlantısı olan + ρ'su tanımlı + seçilen günlerde bağlantısı bulunan pazarlar (177–178).
2. `require_competitors` — rakibi olmayan pazarlarda sıralama terimi tanımsızdır; test için bunlar elenir (179–182).
3. `prefer_improvable` — baseline'da **pencere dışı** en az bir bağlantısı olan pazarları öne alır; yani "iyileştirilebilir" olanları (183–189).
4. `add_reverse` — seçilen her pazarın **ters yönü** de eklenir. E1 dengesi bir yön-çifti kısıtı olduğu için, tek yönü seçmek E1'i test edilemez kılar (194–200).

3.4 `Instance` ve `build_instance()` — veriden model nesnelerine

`src/03_model.py` · satır 205–207 · boş veri kabı sınıfı

```python
class Instance:
    """Modelin tükettiği önişlenmiş alt-örnek."""
    pass
```

Projedeki iki sınıftan biri. Gövdesi `pass`'tır çünkü görevi davranış değil, **taşımaktır**: `build_instance()` ona 12'den fazla alan atar ve `build_model()` bu tek nesneyi alır. Alternatif — 12 parametreli bir fonksiyon imzası — hem okunmaz hem hataya açık olurdu.

`src/03_model.py` · satır 210–231 · zaman referansı, mutlak dakika ve ızgaraya oturtma

```python
def build_instance():
    markets, nod, tk, tcomp, rank, fixed_flnos, rot, blocks = load_data()
    days = CFG["days"]
    chosen = select_markets(markets, nod, tk, days)

```

```
    rho_map = markets.set_index("market")["rho"].to_dict()
    nod_map = nod.set_index(["market", "day"])["N_od"].to_dict()
    delta = CFG["delta_min"]
    wd = CFG["wdev_min"]

    # zaman referansı: tüm zaman damgalarının en küçüğü (mutlak dakika için)
    sub = tk[tk["market"].isin(chosen) & tk["day"].isin(days)].copy()
    if sub.empty:                                    # ValueError → run_byday yakalayıp günü atlar
        raise ValueError("Seçilen alt-örnekte bağlantı yok — CFG'yi gevşetin.")
    t0 = min(sub["ist_arr"].min(), sub["ist_dep"].min()).normalize()

    def abs_min(ts):
        return (ts - t0).total_seconds() / 60.0

    def snap(m):
        return int(round(m / delta) * delta)

```

Üç kavram burada doğar ve dosyanın sonuna kadar yaşar:

Kavram	Kod	Anlamı
`t0`	satır 224	Tüm zaman damgalarının en küçüğünün gece yarısı. **Zaman ekseninin sıfır noktası.**
`abs_min(ts)`	226–227	Bir zaman damgasını `t0`'dan itibaren geçen **mutlak dakikaya** çevirir. Gün 3'ün 08:00'i ≈ 2×1440 + 480
`snap(m)`	229–230	Dakikayı en yakın 10'un katına yuvarlar. Izgaranın tanımı budur

> **Neden mutlak dakika?**
>
> Gerçek `datetime` nesneleriyle doğrusal kısıt yazılamaz. Her şeyi tek bir sayı eksenine (dakika) taşıyarak `θ_dep − θ_arr ≥ 60` gibi kısıtlar doğrudan yazılabilir hâle gelir. Sonuçta bu sayılar `t0 + Timedelta(minutes=...)` ile gerçek saate geri çevrilir.

`src/03_model.py` · satır 232–247 · olaylar (events) — modelin karar birimleri

```
    # --- olaylar (instances): her IST kalkış/varış olayı tek bir karar zamanı ----
    # anahtar = (flno, rol, snap(mevcut))  — aynı uçak/olay birden çok bağlantıda paylaşılır.
    events = {}        # key -> dict(id, flno, role, cur, day, adjustable, cands)

    def get_event(flno, role, cur_abs, day):
        cur = snap(cur_abs)
        key = (int(flno), role, cur)
        if key not in events:
            adj = int(flno) not in fixed_flnos          # rotasyon partneri yoksa SABIT (docs D6)
            if adj:
                cands = [cur + k * delta for k in range(-(wd // delta), wd // delta + 1)]
            else:
                cands = [cur]
            events[key] = dict(id=len(events), flno=int(flno), role=role,
                               cur=cur, day=int(day), adjustable=adj, cands=cands)
        return events[key]["id"]
```

Bu 16 satır modelin **atomudur**. Anlamak için üç soruyu ayırmak gerekir:

Olay nedir?

Bir **olay**, "şu uçuş numarasının, şu roldeki (varış/kalkış), şu andaki IST hareketi"dir. Anahtarı `(flno, role, snap(cur))` üçlüsüdür.

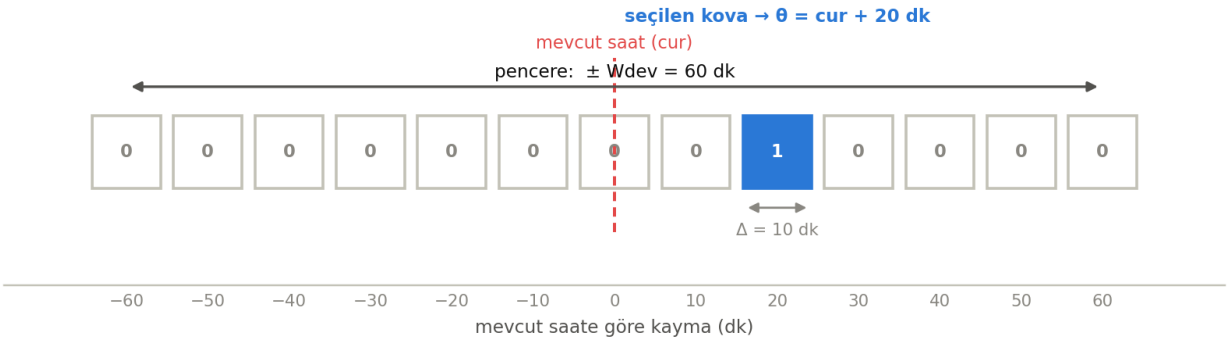
Neden bağlantı başına değil, olay başına karar veriyoruz?

Çünkü **bir uçuş birden fazla bağlantıda yer alır**. IST'e gelen bir uçuş, oradan kalkan 20 farklı uçuşla 20 bağlantı oluşturabilir. Eğer her bağlantı için ayrı saat kararı verilseydi, aynı uçağa 20 farklı varış saati atanabilirdi. Olay sözlüğü (`events`) bunu yapısal olarak imkânsız kılar: **aynı anahtar → aynı karar.** Bu, modelin doğruluğunu garanti eden en önemli tasarım kararıdır.

Aday saatler nasıl üretiliyor?

- Uçuşun rotasyon partneri yoksa (`flno in fixed_flnos`) **ayarlanamaz**: `cands = [cur]`, tek seçenek, hiç kımıldamaz.
- Ayarlanabilirse `cands` = mevcut saatten ±60 dk aralığında 10'ar dakikalık tüm değerler → **13 aday** (−60, −50, …, 0, …, +50, +60).

(A1) Her uçuş olayı için 13 aday kova — tam biri seçilir: Σ_v x[e,v] = 1



Şekil 3.1 — (A1) Bir olayın 13 aday kovası. Model tam birini seçer.

`src/03_model.py` · satır 249–259 · bağlantılar (π) listesi

```
249     # --- bağlantılar (π) ------------------------------------------------
250     conns = []          # her biri: dict(market, day, arr_eid, dep_eid, legsum, gap_cur)
251     for _, r in sub.iterrows():
252         arr_eid = get_event(r["FlNo1"], "arr", abs_min(r["ist_arr"]), r["day"])
253         dep_eid = get_event(r["FlNo2"], "dep", abs_min(r["ist_dep"]), r["day"])
254         conns.append(dict(market=r["market"], day=int(r["day"]),
255                           arr_eid=arr_eid, dep_eid=dep_eid,
256                           legsum=float(r["legsum_min"]),
257                           gap_cur=float(r["gap_min"])))
258
259     ev_by_id = {e["id"]: e for e in events.values()}
```

Her TK bağlantı satırı için varış ve kalkış olayları `get_event` ile alınır (varsa mevcut olan döner, yoksa yenisi yaratılır) ve bağlantı bu **iki olay kimliğiyle** tanımlanır. `legsum` sabittir; `gap_cur` mevcut tarifedeki boşluktur (baseline hesabı için).

`src/03_model.py` · satır 261–272 · extra_use — modellenmeyen uçuşların fiziksel yükü

```
261     # --- (F için) modellenmeyen uçuşların kova kullanımı ----------------
262     # Amaç evrenine girmeyen pazarların (ör. ρ tanımsız → dışlanan) uçuşları da IST
263     # kovalarını FİZİKSEL olarak doldurur. Karar değişkeni olmazlar ama F kapasitesinde
264     # sabit kullanım olarak sayılırlar (instance olayıyla çakışan uçuş zaten sayılıdır).
265     rest = tk[tk["day"].isin(days) & ~tk["market"].isin(set(chosen))]
266     rest_keys = set()
267     for r in rest.itertuples():
268         rest_keys.add((int(r.FlNo1), "arr", snap(abs_min(r.ist_arr))))
269         rest_keys.add((int(r.FlNo2), "dep", snap(abs_min(r.ist_dep))))
270     extra_use = {}
271     for (flno, role, cur) in rest_keys - set(events.keys()):
272         extra_use[(role, cur)] = extra_use.get((role, cur), 0) + 1
```

Bu blok neden var? ("amaç evreni eksikliği")

ρ'su olmayan pazarlar amaç fonksiyonundan **dışlanır** — ama o uçuşlar hâlâ havada! IST pistini ve kapılarını fiziksel olarak dolduruyorlar. Onları F kapasitesinde saymamak, gerçekte olmayan bir boşluk yaratıp modelin oraya uçuş yerleştirmesine izin verirdi. `extra_use`, bu uçuşları **karar değişkeni olmadan, sabit doluluk olarak** kova sayacına ekler.

`src/03_model.py` · satır 274–284 · full_role_time — A2 için aynı açığın kapatılması

```
274     # --- (A2 evren-eksiği) rotasyon ortaklarının TAM evrendeki fiziksel saatleri -----
275     # U4'ün F'de düzelttiği "amaç evreni eksikliği" A2 için de geçerlidir: bir rotasyon
276     # ortağının bağlantısı amaç evreninde (chosen) değilse (ör. ρ tanımsız pazar) o günkü
277     # olay HİÇ modellenmez → A2 kısıtı o gün için hiç kurulamaz (build_model bunu ANKOR
```

```
278     # olarak kullanır; bkz. docs §10-U9). full_role_time TÜM (chosen ∪ dışlanan) günler/
279     # pazarlar için gerçek snap'lenmiş saatleri tutar — extra_use'un A2 karşılığıdır.
280     full_all = tk[tk["day"].isin(days)]
281     full_role_time = {}
282     for r in full_all.itertuples():
283         full_role_time[(int(r.FlNo1), "arr", int(r.day))] = snap(abs_min(r.ist_arr))
284         full_role_time[(int(r.FlNo2), "dep", int(r.day))] = snap(abs_min(r.ist_dep))
```

`extra_use` F kısıtındaki açığı kapatır; `full_role_time` **aynı açığı A2 için** kapatır. Bir rotasyon ortağının bağlantısı amaç evreninde değilse o günkü olay hiç kurulmaz ve A2 kısıtı o gün için hiç oluşmazdı. Bu sözlük, tüm günler ve tüm pazarlar için **gerçek fiziksel saatleri** tutar ve A2 bloğu bunları *çapa* (anchor) olarak kullanır.

> **Bu düzeltmenin hikâyesi**
>
> Bu açığı **bağımsız doğrulayıcı (05) yakaladı**. Wdev=90 denenirken çözücü kaydırmayı artırınca gerçek bir A2 ihlali doğdu (uçuş 2200/2201, gün 2). 05'in notu `05_validate_solution.py` satır 52–60'ta hâlâ duruyor. Çift kayıt ilkesinin işe yaradığının kanıtı.

`src/03_model.py` · satır 286–296 · md — pazar-gün grupları

```
286     # --- pazar-gün grupları (md) ----------------------------------------
287     md = {}            # (market,day) -> dict(rho, N, comps{carrier:Tcomp}, conns[idx], b_od)
288     for pi, c in enumerate(conns):
289         key = (c["market"], c["day"])
290         if key not in md:
291             comps = tcomp[(tcomp["market"] == c["market"]) & (tcomp["day"] == c["day"])]
292             comp_map = dict(zip(comps["carrier"], comps["T_comp_min"]))
293             md[key] = dict(rho=float(rho_map.get(c["market"], 0.0)),
294                            N=int(nod_map.get(key, len(comp_map))),
295                            comps=comp_map, conns=[], b_od=None)
296         md[key]["conns"].append(pi)
```

`md` sözlüğü `(pazar, gün)` anahtarıyla şunları toplar: ρ ağırlığı, rakip sayısı N, rakip taşıyıcıların T_comp değerleri ve bu gruba ait bağlantıların indeksleri. Ödül hesabı **grup düzeyinde** yapılır — çünkü hem azalan getiri (C) hem sıralama (D) pazar bazlıdır.

`src/03_model.py` · satır 298–314 · b_od — başlangıç sırasının hesabı

```
298     # --- b_od (taban sıralama, docs §9): baseline 'son sıra' (s) ile BİREBİR tutarlı ----
299     # Sıralama ödülü bir İYİLEŞTİRME ödülüdür (brief §2.4): lookup köşegeni b==s = 'değişim yok'
300     # nötr referansı (değeri 1/(N-1)). Dolayısıyla mevcut tarife (baseline) HER pazarda buna oturmalı ⇒
301     # b_od, baseline'da s=clip(N-beaten,1,N) ile AYNI tabandan hesaplanır: pencere-içi
302     # ([L,U]) sunulan bağlantıların en iyi J'si üzerinden yenilen rakip sayısından türetilir.
303     # (Eski kural faster+1 idi; s=N-beaten konvansiyonuna göre 1 kayıyor ve baseline'ı
304     #  pazarların ~%29'unda köşegen-dışına itiyordu → mevcut tarifeye hak edilmemiş ödül.)
305     Lc, Uc = CFG["L_min"], CFG["U_min"]
306     for key, g in md.items():
307         if g["N"] >= 1 and g["conns"]:
308             offered0 = [pi for pi in g["conns"] if Lc <= conns[pi]["gap_cur"] <= Uc]
309             if offered0 and g["comps"]:
310                 j_best0 = min(conns[pi]["legsum"] + conns[pi]["gap_cur"] for pi in offered0)
311                 beaten0 = sum(1 for tc in g["comps"].values() if j_best0 <= tc)
312             else:
313                 beaten0 = 0
314             g["b_od"] = int(np.clip(g["N"] - beaten0, 1, g["N"]))
```

Sıralama ödülü bir **iyileştirme ödülüdür**: "b. sıradan s. sıraya çıktın, işte ödülün." Dolayısıyla b_od, **mevcut tarifenin sırası** olmalıdır. Kod bunu tam olarak s ile aynı formülden türetir:

1. Mevcut tarifede pencere içinde ([60,300]) olan bağlantılar bulunur (`offered0`).
2. Bunların **en iyi J**'si hesaplanır (`j_best0`).
3. Bu J'nin yendiği rakip sayısı sayılır (`beaten0`).
4. `b_od = clip(N − beaten0, 1, N)` — s formülüyle **birebir aynı taban**.

> **Neden bu kadar titiz davranılmış?**
>
> Ödül tablosunun **köşegeni** (b == s) "değişim yok" nötr referansıdır. Eğer b_od farklı bir kuralla hesaplansaydı, mevcut

tarife köşegenin dışına düşer ve **hiçbir şey yapmadan** sıralama ödülü kazanırdı — yani baseline şişer, iyileşme sahte görünürdü. Kodun yorumu eski kuralın (faster+1) pazarların ~%29'unda tam bu hatayı yaptığını yazıyor.

`src/03_model.py` · satır 316–331 · yardımcı indeksler ve Instance'ın doldurulması

```
316     # --- yardımcı indeksler (A2 rotasyon, G düzenlilik için) --------------
317     ev_by_flno_role = {}                # (flno, role) -> [eid]
318     for e in events.values():
319         ev_by_flno_role.setdefault((e["flno"], e["role"]), []).append(e["id"])
320
321     inst = Instance()
322     inst.events, inst.ev_by_id, inst.conns, inst.md = events, ev_by_id, conns, md
323     inst.extra_use = extra_use          # (rol, kova) -> modellenmeyen sabit uçuş sayısı (F)
324     inst.rank = rank
325     inst.L, inst.U = CFG["L_min"], CFG["U_min"]
326     inst.t0, inst.delta = t0, delta
327     inst.ev_by_flno_role = ev_by_flno_role
328     inst.rot_pairs = rotation_pairs(rot)
329     inst.blocks = blocks                # (flno, day, role) -> blok dk | None (eski şema)
330     inst.full_role_time = full_role_time  # (flno, role, day) -> snap dk (A2 evren-eksiği ankoru)
331     return inst
```

`ev_by_flno_role` — `(uçuş no, rol) → [olay kimlikleri]` haritası. A2 ve G kısıtları bu haritayı kullanarak "bu uçuşun tüm günlerdeki olayları" sorgusunu O(1) yapar. Son 10 satır `Instance` nesnesini doldurur ve döndürür.

3.5 Ödül ve zaman ifadeleri

`src/03_model.py` · satır 335–345 · sıralama ödül tablosu ve sorgusu

```
335 def rank_weight_table(rank_df):
336     return {(int(r.N), int(r.b), int(r.s)): float(r.W) for r in rank_df.itertuples()}
337
338
339 def rank_reward(rkW, N, b_od, beaten):
340     """Sıralama ödülü: yenilen sayısı 'beaten' iken W^r(N, b_od, s), s = clamp(N-beaten, 1, N).
341     docs §4/§9 (GEÇİCİ kural). N<1 veya b_od yoksa 0 (rakip yok → sıralama terimi tanımsız)."""
342     if N < 1 or b_od is None:
343         return 0.0
344     s = int(np.clip(N - beaten, 1, N))
345     return rkW.get((N, b_od, s), 0.0)
```

`rank_weight_table` lookup CSV'sini `(N, b, s) → W` sözlüğüne çevirir; `rank_reward` ise "N rakip varken b'den başlayıp, `beaten` rakip yenerek s'ye çıktım" sorgusunun ödülünü döndürür. `s = clip(N − beaten, 1, N)` — daha çok rakip yenmek s'yi küçültür, küçük s daha iyi sıralamadır.

`src/03_model.py` · satır 348–354 · theta_expr — bir olayın saatinin cebirsel ifadesi

```
348 def theta_expr(model, inst, eid):
349     """Olay eid'nin IST zamanı (dk): ayarlanabilirse Σ val·x, sabitse sabit."""
350     e = inst.ev_by_id[eid]
351     if e["adjustable"]:
352         return sum(v * model.x[eid, v] for v in e["cands"])
353     return float(e["cands"][0])
354
```

Modelin en çok çağrılan yardımcısı. Olay **ayarlanabilirse** θ = Σ (aday değeri × o adayın ikili değişkeni) — yani doğrusal bir ifade döner. **Sabitse** sadece bir sayı döner. Bu tek fonksiyon sayesinde kısıtlar "sabit mi değişken mi" ayrımı yapmadan yazılabilir:

Neden Σ v·x[e,v] işe yarıyor?

A1 kısıtı Σ_v x[e,v] = 1 olduğu için, ikili değişkenlerden **tam biri** 1'dir. O zaman Σ v·x[e,v] tam olarak seçilen adayın değerine eşittir. Sürekli bir saat değişkeni yerine bu "tek-sıcak" (one-hot) kodlama seçilmiştir — çünkü ileride **F kapasitesi** ve **P cezası** doğrudan bu değişkenler üzerinden, ek değişken ve big-M olmadan yazılabiliyor.

3.6 `build_model()` — değişkenler ve (A1)

`src/03_model.py` · satır 356–375 · x, y ve A1 kısıtı

```python
def build_model(inst):
    m = pyo.ConcreteModel()
    L, U = inst.L, inst.U

    # --- değişkenler: x[eid,val] (yalnız ayarlanabilir olaylar) -------------
    x_index = [(e["id"], v) for e in inst.ev_by_id.values() if e["adjustable"] for v in e["cands"]]
    m.x = pyo.Var(x_index, domain=pyo.Binary)

    # y_π : bağlantı sunuluyor (uygun) mu
    P = list(range(len(inst.conns)))
    m.y = pyo.Var(P, domain=pyo.Binary)
    if CFG["B2_force_offer"]:
        m.s = pyo.Var(P, domain=pyo.Binary)     # B2 ayrık "çok kısa / çok uzun" seçici

    m.cons = pyo.ConstraintList()

    # --- (A1) her ayarlanabilir olay tam bir kovaya atanır -----------------
    for e in inst.ev_by_id.values():
        if e["adjustable"]:
            m.cons.add(sum(m.x[e["id"], v] for v in e["cands"]) == 1)
```

Değişken	İndeks	Anlamı
`x[eid, v]`	ayarlanabilir olay × aday saat	Bu olay `v` saatine atandı mı? (0/1)
`y[pi]`	her bağlantı	Bu bağlantı **sunuluyor** (uygun) mu? (0/1)
`s[pi]`	her bağlantı (yalnız B2 açıksa)	Uygun değilse "çok kısa" mı "çok uzun" mu? (0/1)

Satır 372–375 = Kısıt A1: her ayarlanabilir olay için `Σ_v x[eid,v] == 1`. Türkçesi: "her uçuş hareketine tam bir saat ata." Üç satırlık bu kısıt, modelin tüm doğrusal yapısını mümkün kılar.

`m.cons = pyo.ConstraintList()` — Pyomo'da adlandırılmış kısıt blokları yerine tek bir listeye ekleme yapılır. Kısıtlar farklı mantıklardan geldiği için bu daha pratiktir; kaç kısıt eklendiği ayrı sayaçlarla (`m._n_a2`, `m._n_g`, `m._n_f`) izlenir.

3.7 (B1) Bağlantı uygunluğu ve big-M tekniği

`src/03_model.py` · satır 377–400 · gap ifadesi, sıkı big-M ve B1/B2

```python
    # --- gap_π ifadesi ve büyük-M aralıkları -------------------------------
    gap = {}
    for pi, c in enumerate(inst.conns):
        gap[pi] = theta_expr(m, inst, c["dep_eid"]) - theta_expr(m, inst, c["arr_eid"])
        ea, ed = inst.ev_by_id[c["arr_eid"]], inst.ev_by_id[c["dep_eid"]]
        gmin = min(ed["cands"]) - max(ea["cands"])       # ulaşılabilir en küçük boşluk
        gmax = max(ed["cands"]) - min(ea["cands"])       # ulaşılabilir en büyük boşluk
        c["gmin"], c["gmax"] = gmin, gmax

        # --- (B1) y=1 ⇒ ℓ ≤ gap ≤ u  (yön-yerel sıkı M) ------------------
        Mlo = max(0, L - gmin)
        Mhi = max(0, gmax - U)
        m.cons.add(gap[pi] >= L - Mlo * (1 - m.y[pi]))
        m.cons.add(gap[pi] <= U + Mhi * (1 - m.y[pi]))
        # uygun aralık hiç yakalanamıyorsa y=0 sabitlenir
        if gmax < L or gmin > U:
            m.cons.add(m.y[pi] == 0)
        # --- (B2) uygun ⇒ y=1  (zorunlu sunum; ayrık alt/üst seçimi s_π) ---
        elif CFG["B2_force_offer"]:
            d2 = inst.delta
            Mb = max(d2, gmax - (L - d2), (U + d2) - gmin)
            # y=0,s=0 ⇒ gap ≤ ℓ−Δ (çok kısa) ;  y=0,s=1 ⇒ gap ≥ u+Δ (çok uzun)
            m.cons.add(gap[pi] <= (L - d2) + Mb * (m.y[pi] + m.s[pi]))
            m.cons.add(gap[pi] >= (U + d2) - Mb * (m.y[pi] + (1 - m.s[pi])))
```

Bu blok modelin **mantıksal kalbidir**. Adım adım:

1) Boşluk ifadesi (380)

`gap[pi] = θ_kalkış − θ_varış`. İki `theta_expr` çağrısının farkı — doğrusal bir ifade.

2) Ulaşılabilir aralık (382–383)

Adaylar sınırlı olduğu için boşluk da sınırlıdır: en küçük boşluk = (en erken kalkış) − (en geç varış), en büyük boşluk = (en geç kalkış) − (en erken varış). Bu iki sayı `gmin` ve `gmax` olarak saklanır.

3) B1 kısıtı big-M ile (386–390)

İstenen mantık: **y_π = 1 ise 60 ≤ boşluk ≤ 300.** Bu bir "eğer-ise" ifadesidir ve doğrusal programlamada doğrudan yazılamaz. Big-M numarası şudur:

> **gap ≥ L − Mlo·(1 − y) nasıl çalışır?**
> **y = 1 ise:** sağ taraf L − 0 = 60 olur → kısıt `boşluk ≥ 60` olarak devreye girer.
> **y = 0 ise:** sağ taraf 60 − Mlo olur. Mlo = max(0, L − gmin) seçildiği için bu değer, boşluğun alabileceği **en küçük değerden bile küçüktür** → kısıt otomatik sağlanır, yani **etkisizleşir**. Üst sınır için de simetrik olarak Mhi = max(0, gmax − U) kullanılır.

M'in küçük seçilmesi kritiktir. Gereğinden büyük bir M, MIP'in *doğrusal gevşemesini* zayıflatır ve çözücü ağaçta kaybolur. Buradaki M'ler **yön-yerel** olarak, o bağlantının gerçekten ulaşabileceği aralıktan hesaplanır — mümkün olan en sıkı değerler.

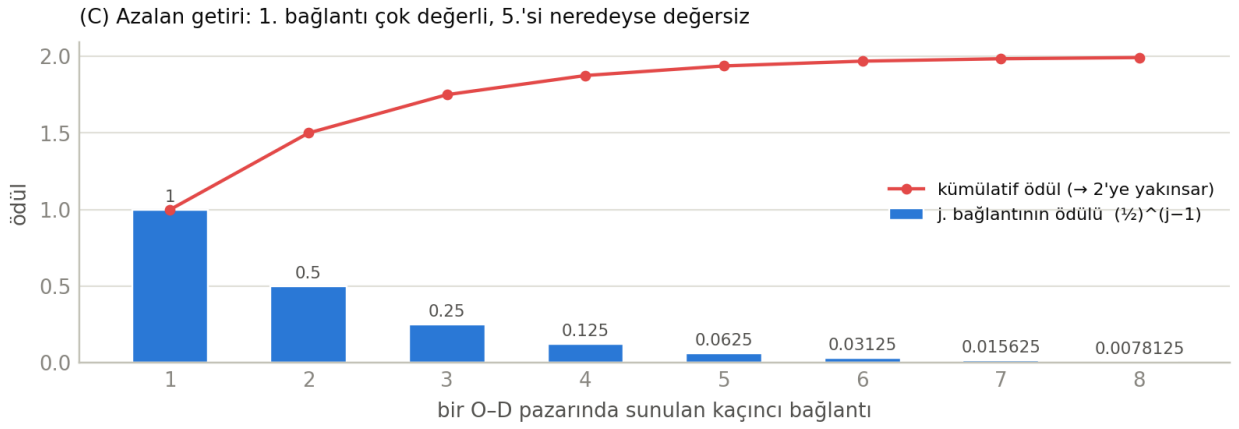
4) Erken sabitleme (391–393)

Eğer `gmax < L` veya `gmin > U` ise bu bağlantı **hiçbir kaydırma kombinasyonuyla** uygun olamaz → `y[pi] == 0` doğrudan sabitlenir. Çözücünün araması gereken alan küçülür.

5) B2 (kapalı)

394–400 arası B2'yi kurar: "uygunsa sunmak zorundasın". `s[pi]` ayrık seçicisi y=0 durumunda boşluğun ya ℓ−Δ'nın altında ya u+Δ'nın üstünde olmasını zorlar. Varsayılan olarak kapalıdır çünkü E1 dengesiyle çatışıp modeli infeasible yapar.

3.8 (C) Azalan getiri — neden 5. bağlantı değersizdir?



Şekil 3.2 — Ödül dizisi (½)^(j−1). Toplam ödül 2'ye yakınsar.

`src/03_model.py` · satır 402–418 · z değişkenleri ve monoton doldurma

```
402      # --- (C) azalan-getiri / monoton slotlar -------------------------------
```

```
403      # z[md_id, j] : md grubunda j. slot dolu mu
404      md_keys = list(inst.md.keys())
405      md_id = {k: i for i, k in enumerate(md_keys)}
406      z_index, Wc = [], {}
407      for k, g in inst.md.items():
408          M = len(g["conns"])
409          for j in range(1, M + 1):
410              z_index.append((md_id[k], j))
411              Wc[(md_id[k], j)] = 0.5 ** (j - 1)          # W^c_j = (1/2)^{j-1}
412      m.z = pyo.Var(z_index, domain=pyo.Binary)
413      for k, g in inst.md.items():
414          i = md_id[k]
415          M = len(g["conns"])
416          m.cons.add(sum(m.y[pi] for pi in g["conns"]) == sum(m.z[i, j] for j in range(1, M + 1)))
417          for j in range(1, M):
418              m.cons.add(m.z[i, j] >= m.z[i, j + 1])      # monoton doldurma
```

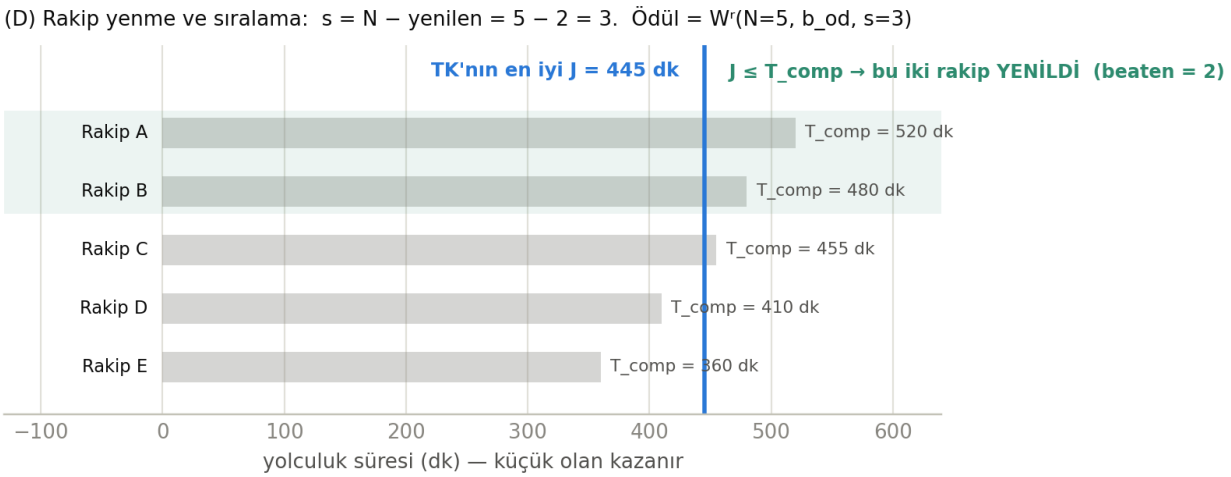
Gerçek hayatta bir pazarda **ilk** aktarma seçeneği çok değerlidir; ikincisi biraz daha az; beşincisi neredeyse hiç. Model bunu şöyle kurar:

1. Her pazar-gün grubu için `M` adet slot değişkeni `z[i,j]` tanımlanır (M = o gruptaki aday bağlantı sayısı).
2. **Bağlama kısıtı (416):** `Σ y[pi] == Σ z[i,j]` — kaç bağlantı sunuluyorsa o kadar slot dolu olmalı.
3. **Monotonluk (417–418):** `z[i,j] ≥ z[i,j+1]` — slotlar baştan doldurulmalı. Bu olmadan çözücü 1. slotu boş bırakıp 1 numaralı ödülü almadan 3. slotu doldurabilirdi.
4. Ödül ağırlıkları `Wc[(i,j)] = 0.5^(j-1)` olarak önceden hesaplanır (411).

Neden ayrı z değişkenleri? y'ye doğrudan azalan ağırlık verilemez mi?

Hayır — çünkü "kaçıncı bağlantı" olduğu, hangi bağlantıların seçildiğine bağlıdır ve önceden bilinemez. `z` dizisi "kaç tane sunuldu" sayısını **ödüle** çevirmenin doğrusal yoludur: n bağlantı sunulursa ilk n slot dolar ve toplam ödül tam olarak 1 + ½ + ¼ + … + (½)^(n−1) olur.

3.9 (D) Rakip yenme ve sıralama ödülü



Şekil 3.3 — Yenilen rakip sayısı, sıralama ödülünün girdisidir.

Adım 1 — w: "bu bağlantı bu rakibi yeniyor mu?"

`src/03_model.py` · satır 420–441 · koşullu w değişkenleri ve üç durum ayrımı

```
420      # --- (D) rakip yenme + sıralama ---------------------------------------
421      # w[(pi,carrier)] yalnız "koşullu" durumda yaratılır; "hep yener" → y, "asla" → yok.
422      w = {}                          # (pi, carrier) -> Var  (koşullu)
423      beat_eq_y = {}                  # (pi, carrier) -> True  (offered ⟹ beats; w := y)
424      m.w = pyo.Var(pyo.Any, domain=pyo.Binary, dense=False)   # seyrek binary havuzu
425      for k, g in inst.md.items():
426          for pi in g["conns"]:
427              c = inst.conns[pi]
```

```
            for carrier, Tc in g["comps"].items():
                G = Tc - c["legsum"]                    # izinli maks. boşluk (yenmek için)
                lo = max(L, c["gmin"])                  # sunulurken ulaşılabilir en küçük boşluk
                if G < lo:
                    continue                            # asla yenemez → w ≡ 0
                elif G >= U:
                    beat_eq_y[(pi, carrier)] = True     # sunulduğunda hep yener → w = y
                else:
                    wv = m.w[("w", pi, carrier)]        # koşullu
                    w[(pi, carrier)] = wv
                    Mw = max(1.0, c["gmax"] - G)
                    m.cons.add(wv <= m.y[pi])
                    m.cons.add(gap[pi] <= G + Mw * (1 - wv))
```

Yenmek demek J ≤ T_comp demektir. J = legsum + boşluk olduğundan bu, boşluk üzerine bir üst sınıra dönüşür: **boşluk ≤ G**, burada `G = T_comp − legsum`. Kod bu eşiği ulaşılabilir aralıkla karşılaştırıp **üç duruma** ayırır:

Durum	Koşul	Ne yapılır?	Kazanç
Asla yenemez	`G < lo`	Değişken **yaratılmaz** (`continue`)	Boş yere değişken üretilmez
Hep yener	`G ≥ U`	`beat_eq_y` işaretlenir; w yerine doğrudan `y` kullanılır	Bir değişken + iki kısıt tasarrufu
Koşullu	`lo ≤ G < U`	Gerçek `w` değişkeni + iki kısıt kurulur	Yalnız gerçekten gerekli olanlar

> **Bu ayrım neden bu kadar önemli?**
>
> Tam veride binlerce (bağlantı × rakip) çifti vardır. Hepsine değişken açmak modeli gereksiz yere şişirir. Bu üç yollu ayrım, model boyutunu **büyük ölçüde** küçültür — büyük MIP'lerde çözülebilirlik ile çözülemezlik arasındaki fark tam olarak bu tür ayıklamalardır. `m.w = pyo.Var(pyo.Any, dense=False)` ise **seyrek** bir değişken havuzudur: yalnız istenen indeksler yaratılır.

Adım 2 — β: "bu rakip yenildi mi?"

`src/03_model.py` · satır 442–457 · rakip düzeyinde toplama

```
    # β[(md_id,carrier)] : rakip yenildi mi  (yenen ≥1 sunulu bağlantı varsa)
    beta = {}
    for k, g in inst.md.items():
        i = md_id[k]
        for carrier in g["comps"]:
            terms = []
            for pi in g["conns"]:
                if (pi, carrier) in beat_eq_y:
                    terms.append(m.y[pi])
                elif (pi, carrier) in w:
                    terms.append(w[(pi, carrier)])
            if not terms:
                continue
            bv = m.w[("beta", i, carrier)]
            beta[(i, carrier)] = bv
            m.cons.add(bv <= sum(terms))                # β ≤ Σ w  (ödül β'yı yukarı iter)
```

Bir rakibi yenmek için **en az bir** sunulan bağlantının onu yenmesi yeterlidir. Kısıt `β ≤ Σ w` şeklindedir — yani hiçbir bağlantı yenmiyorsa β 0'a zorlanır. Ters yön (`β ≥ ...`) gerekmez çünkü **amaç fonksiyonu β'yı yukarı iter**: β büyüdükçe ödül artar, dolayısıyla çözücü onu mümkün olan en yükseğe çeker. Gereksiz kısıt yazmamak yine model boyutu kazancıdır.

Adım 3 — γ: yenilen sayısını ödüle çevirme

`src/03_model.py` · satır 459–475 · seçici değişkenler ve amaç terimleri

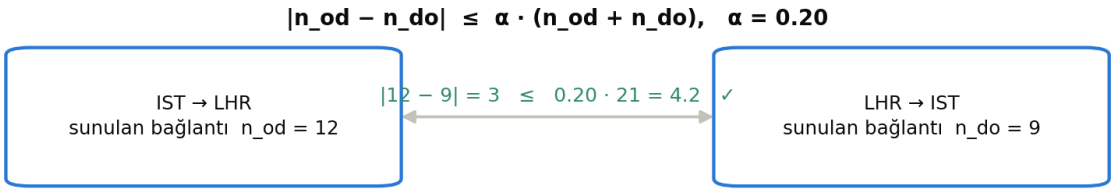
```
    # γ[(md_id,bcount)] : yenilen rakip sayısı seçici (Σγ=1, Σ b·γ = Σβ)
    # NOT: doküman §4'te Σγ=1 KOŞULSUZ — yenilebilir rakip olmasa da beaten=0 için
    # taban ödülü W^(N,b_od,N) kazanılır. Bu yüzden N≥1 olan her pazar için γ kurulur.
    obj_terms = []
    rkW = rank_weight_table(inst.rank)
```

```
464     if CFG["ranking"]:
465         for k, g in inst.md.items():
466             i = md_id[k]
467             N, b_od = g["N"], g["b_od"]
468             if N < 1 or b_od is None:
469                 continue
470             betas = [beta[(i, ca)] for ca in g["comps"] if (i, ca) in beta]
471             gam = {bn: m.w[("gam", i, bn)] for bn in range(0, N + 1)}
472             m.cons.add(sum(gam.values()) == 1)
473             m.cons.add(sum(bn * gam[bn] for bn in gam) == (sum(betas) if betas else 0))
474             for bn, gv in gam.items():
475                 obj_terms.append(g["rho"] * rank_reward(rkW, N, b_od, bn) * gv)
```

Ödül tablosu ayrık bir fonksiyondur: "2 rakip yendiysen şu kadar, 3 yendiysen bu kadar." Bunu doğrusal modele sokmanın standart yolu **seçici (indicator) değişken dizisidir**:

- `γ[i,bn]` — "bu pazarda tam olarak `bn` rakip yenildi" göstergesi.
- `Σ γ == 1` (472) — tam bir değer seçilmeli. **Koşulsuzdur**: hiç rakip yenilmese bile bn = 0 seçilir ve taban ödülü kazanılır.
- `Σ bn·γ[bn] == Σ β` (473) — seçilen sayı, gerçekten yenilen rakip sayısına eşit olmalı. Bu iki kısıt γ'yı β'ya kilitler.
- Amaç terimi (474–475): her bn için `ρ × Wʳ(N, b_od, bn) × γ[bn]`. γ'lardan biri 1 olduğu için toplam tam olarak doğru ödüle eşittir.

3.10 (E1) Yönsel denge



Neden? Tek yönü şişirip dönüşü ihmal eden bir tarife, kâğıt üstünde puan kazanır ama gerçekte satılabilir bir ürün değildir (gidiş-dönüş talep eder).

Şekil 3.4 — Gidiş ve dönüş yönleri arasında denge.

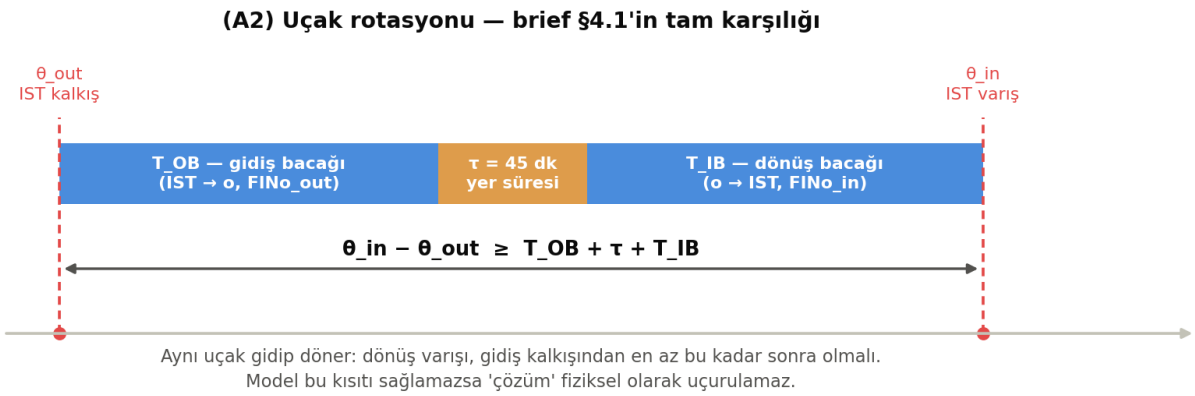
src/03_model.py · satır 477–490 · yön-çifti başına iki kısıt

```
477     # --- (E1) yönsel SAYI dengesi (opsiyonel) ------------------------------
478     if CFG["E1_balance"]:
479         a = CFG["alpha"]
480         seen = set()
481         for k, g in inst.md.items():
482             (mk, day) = k
483             o, dd = mk.split("-")
484             rk = (f"{dd}-{o}", day)
485             if rk in inst.md and (rk, k) not in seen:
486                 seen.add((k, rk))
487                 n_od = sum(m.y[pi] for pi in g["conns"])
488                 n_do = sum(m.y[pi] for pi in inst.md[rk]["conns"])
489                 m.cons.add(n_od - n_do <= a * (n_od + n_do))
490                 m.cons.add(n_do - n_od <= a * (n_od + n_do))
```

Her `(o-d, gün)` grubunun ters yönü `(d-o, gün)` aranır; ikisi de modeldeyse iki simetrik kısıt eklenir. `seen` kümesi aynı çiftin iki kez işlenmesini engeller.

Mutlak değer `|n_od − n_do| ≤ α(n_od + n_do)` iki eşitsizlikle yazılır — bu, doğrusal programlamada mutlak değerin standart açılımıdır ve **hiç ek değişken gerektirmez**.

3.11 (A2) Uçak rotasyonu — fiziksel gerçeklik kısıtı



Şekil 3.5 — Minimum tur süresi.

`src/03_model.py` · satır 492–523 · aynı-gün rotasyon kısıtları

```
492     # --- (A2) uçak rotasyonu: min tur = T_OB + τ + T_IB (brief §4.1) ---------
493     # Güncel veri bacak bloklarını verir → brief-exact minimum kurulur. Mevcut tur
494     # brief minimumunun ALTINDAYSA (gece-aşan rotasyonun aynı-gün izdüşümü; fiziksel
495     # tur gün h → h+1'e yayılır) mevcut tur çapası korunur: sep = min(exact, cur).
496     # Böylece (i) gerçek aynı-gün turlar asla brief minimumunun altına sıkışmaz,
497     # (ii) gece-aşan çiftlerde mevcut desen korunur (gün-bağlaşımı yaratılmaz;
498     # fiziksel h→h+1 kısıtı çözüm-sonrası ayrıca doğrulanır). Bloklar yoksa (eski
499     # şema) ihtiyatlı geri-dönüş: sep = mevcut turnaround.
500     m._n_a2 = 0
501     m._n_a2_exact = 0
502     if CFG["A2_rotation"]:
503         tau = CFG["tau_min"]
504         for out_f, in_f in inst.rot_pairs:
505             for oe in inst.ev_by_flno_role.get((out_f, "dep"), []):
506                 for ie in inst.ev_by_flno_role.get((in_f, "arr"), []):
507                     ed, ea = inst.ev_by_id[oe], inst.ev_by_id[ie]
508                     if ed["day"] != ea["day"]:
509                         continue                        # aynı operasyon günü rotasyonu
510                     turn = ea["cur"] - ed["cur"]        # mevcut turnaround (dk)
511                     if turn <= 0:
512                         continue
513                     sep = turn
514                     if inst.blocks is not None:
515                         t_ob = inst.blocks.get((out_f, ed["day"], "dep"))
516                         t_ib = inst.blocks.get((in_f, ea["day"], "arr"))
517                         if t_ob is not None and t_ib is not None:
518                             exact = t_ob + tau + t_ib
519                             if exact <= turn:           # gerçek aynı-gün tur
520                                 sep = exact
521                                 m._n_a2_exact += 1
522                     m.cons.add(theta_expr(m, inst, ie) - theta_expr(m, inst, oe) >= sep)
523                     m._n_a2 += 1
```

Her rotasyon çifti (giden uçuş, gelen uçuş) ve aynı operasyon günündeki her olay kombinasyonu için tek bir kısıt kurulur: **θ_gelen − θ_giden ≥ sep**. Asıl incelik `sep`'in nasıl seçildiğidir:

Durum	sep değeri	Gerekçe
Bloklar var, `exact ≤ turn`	`exact = T_OB + τ + T_IB`	Gerçek aynı-gün tur → brief'in tam minimumu uygulanır
Bloklar var, `exact > turn`	`turn` (mevcut tur)	Bu bir **gece-aşan** rotasyondur; fiziksel tur gün h→h+1'e yayılır. Mevcut desen korunur, yapay gün-bağlaşımı yaratılmaz
Bloklar yok (eski şema)	`turn`	İhtiyatlı geri-dönüş: en azından mevcut tur kadar boşluk korunur

sep = min(exact, cur) formülünün mantığı

İki riski aynı anda kapatır: (i) gerçek aynı-gün turlar **asla** brief minimumunun altına sıkıştırılamaz, (ii) gece-aşan çiftlerde model mevcut durumu bozmaz. Gece-aşan turların gerçek zaman eksenindeki doğruluğu ise çözüm **sonrasında** ayrıca

doğrulanır (`run_byday` içinde ve 05'te).

src/03_model.py · satır 525–566 · A2 × evren-ankoru — modellenmeyen tarafın sabitlenmesi

```
525         # --- (A2×evren) rotasyon ortağının amaç-evreni-dışı günleri (docs §10-U9) ---
526         # Üstteki döngü YALNIZ HER İKİ tarafın da modellendiği (gün,pazar-evreni-içi)
527         # günleri kapsar. Bir tarafın bağlantısı amaç evreninde değilse (ör. ρ tanımsız
528         # pazar → o günkü olay hiç kurulmaz, D5) o gün için kısıt HİÇ oluşmazdı — F'de
529         # (U4) düzeltilen "amaç evreni eksikliği" açığının A2 karşılığı (bağımsız
530         # doğrulayıcı 05'in yakaladığı gerçek ihlal). Bu blok, üstteki döngünün
531         # ATLADIĞI her (pair,gün) için modellenmeyen tarafı full_role_time'daki SABİT
532         # gerçek saatiyle ANKORLAYARAK kısıtı ekler (05'in sep=min(exact,turn) formülüyle
533         # birebir). Her iki taraf da modelsizse (fiziksel arşiv verisi, karar dışı)
534         # atlanır — o zaten sabit veridir, kısıt uygulanamaz/gereksizdir.
535         m._n_a2_anchor = 0
536         frt = getattr(inst, "full_role_time", None)
537         if frt:
538             for out_f, in_f in inst.rot_pairs:
539                 modeled_days = {inst.ev_by_id[oe]["day"]
540                                 for oe in inst.ev_by_flno_role.get((out_f, "dep"), [])
541                                 for ie in inst.ev_by_flno_role.get((in_f, "arr"), [])
542                                 if inst.ev_by_id[oe]["day"] == inst.ev_by_id[ie]["day"]}
543                 all_days = {d for (f, r, d) in frt if f in (out_f, in_f)}
544                 for d in all_days - modeled_days:
545                     dep_full = frt.get((out_f, "dep", d))
546                     arr_full = frt.get((in_f, "arr", d))
547                     if dep_full is None or arr_full is None:
548                         continue
549                     turn = arr_full - dep_full
550                     if turn <= 0:
551                         continue
552                     sep = turn
553                     t_ob = inst.blocks.get((out_f, d, "dep")) if inst.blocks else None
554                     t_ib = inst.blocks.get((in_f, d, "arr")) if inst.blocks else None
555                     if t_ob is not None and t_ib is not None:
556                         sep = min(t_ob + tau + t_ib, turn)
557                     oe = next((e for e in inst.ev_by_flno_role.get((out_f, "dep"), [])
558                                if inst.ev_by_id[e]["day"] == d), None)
559                     ie = next((e for e in inst.ev_by_flno_role.get((in_f, "arr"), [])
560                                if inst.ev_by_id[e]["day"] == d), None)
561                     if oe is None and ie is None:
562                         continue                        # her iki taraf da sabit veri → gereksiz
563                     dep_expr = theta_expr(m, inst, oe) if oe is not None else dep_full
564                     arr_expr = theta_expr(m, inst, ie) if ie is not None else arr_full
565                     m.cons.add(arr_expr - dep_expr >= sep)
566                     m._n_a2_anchor += 1
```

Üstteki döngü yalnız **her iki tarafın da modellendiği** günleri kapsar. Bir taraf amaç evreni dışındaysa (ρ'su olmayan pazar) o gün için kısıt hiç kurulmazdı. Bu blok o boşluğu kapatır: modellenmeyen taraf `full_role_time`'daki **gerçek sabit saatiyle** ifadeye konur (563–564) ve kısıt yine eklenir.

Her iki taraf da modelsizse atlanır — o zaten değiştiremeyeceğimiz arşiv verisidir, kısıt uygulamanın anlamı yoktur.

src/03_model.py · satır 568–588 · gece-aşan koruma sınırlarının modele bağlanması

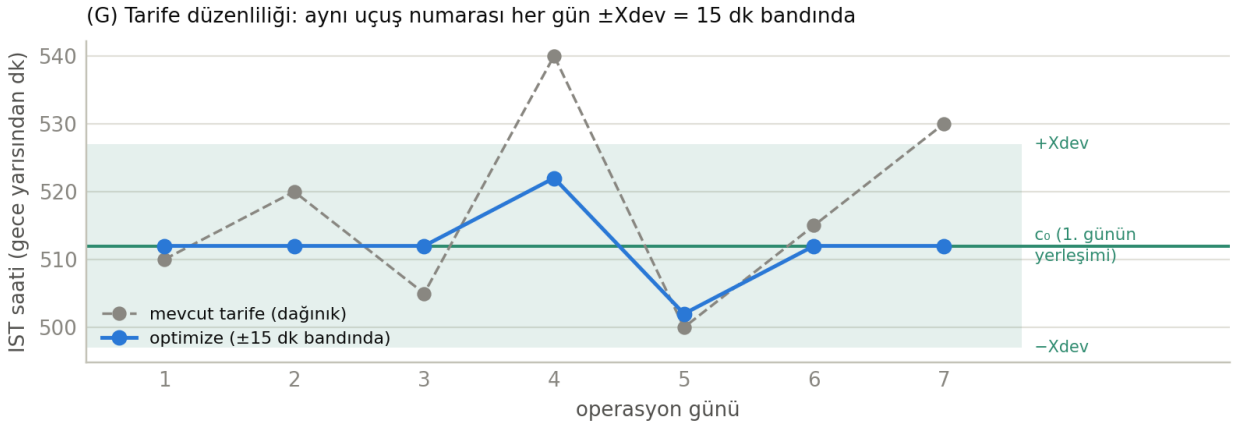
```
568     # --- (A2×) gece-aşan rotasyon koruması (byday, bkz. run_byday) ----------
569     # Fiziksel tur gün h→h+1'e yayılan çiftlerde gün-ayrıştırması turu bozmasın:
570     # mevcut tarifedeki pay (slack) statik ikiye bölünür —
571     #  rotcap : bugünün kalkışı en çok slack/2 GEÇ kayabilir (tavan),
572     #  rotlock: bugünün dönüş varışı en çok slack/2 ERKEN kayabilir (taban)
573     # → ϑ_in − ϑ_out ≥ T_OB+τ+T_IB yapısal garanti; gün-bağlaşımı doğmaz.
574     # Değerler GERÇEK zaman (datetime) verilir; bu örneğin eksenine çevrilir.
575     m._n_rot_chain = 0
576     for attr, role, sense in [("rotlock", "arr", "lo"), ("rotcap", "dep", "hi")]:
577         bounds = getattr(inst, attr, None) or {}
578         for (flno, day), val_dt in bounds.items():
579             val = (val_dt - inst.t0).total_seconds() / 60.0
580             for eid in inst.ev_by_flno_role.get((flno, role), []):
581                 e = inst.ev_by_id[eid]
582                 if e["day"] != day or not e["adjustable"]:
583                     continue
584                 if sense == "lo":
585                     m.cons.add(theta_expr(m, inst, eid) >= val)
586                 else:
587                     m.cons.add(theta_expr(m, inst, eid) <= val)
588                 m._n_rot_chain += 1
```

`byday` modunda gün h'nin kalkışı ile gün h+1'in dönüşü **farklı MIP'lerde** çözülür — yani model bu ikisini birbirine bağlayamaz. Çözüm: mevcut tarifedeki boşluk payı (slack) statik olarak ikiye bölünür ve iki tek-taraflı sınır konur:

- `rotcap` — bugünün kalkışı en fazla `slack/2` **geç** kayabilir (üst sınır).
- `rotlock` — bugünün dönüş varışı en fazla `slack/2` **erken** kayabilir (alt sınır).

İkisi birlikte `θ_in − θ_out ≥ T_OB + τ + T_IB` eşitsizliğini **yapısal olarak** garanti eder — iki gün birbirini hiç görmese bile ihlal matematiksel olarak imkânsızdır. Sınırlar `compute_overnight_guards()` içinde hesaplanır (bkz. 3.17).

3.12 (G) Tarife düzenliliği



Şekil 3.6 — Aynı uçuş numarası her gün aynı bandın içinde.

`src/03_model.py` · satır 590–627 · gün-içi grup bandı c_f

```
590     # --- (G) tarife düzenliliği: aynı uçuş no günler arası ≤ Xdev -----------
591     # Wdev⇔G gerilimi: pencere ±Wdev her olayı KENDİ mevcut saatine bağlar; iki günün
592     # mevcut saatleri 2·Wdev+Xdev'den fazla farklıysa ortak düzenli saat pencerede YOK.
593     # Bu yüzden G yalnız pencerenin izin verdiği çiftlere uygulanır; gerisi "pencere
594     # içinde düzenlenemez" olarak atlanır (belgeli — Wdev/Xdev büyütülerek genişletilebilir).
595     # NOT: ϑ MUTLAK dakika (t0'dan) → günler arası ~1440·gün fark içerir. Düzenlilik
596     # SAAT-İÇİ (time-of-day) tutarlılıktır; her olaydan kendi günü-gece-yarısı ofseti
597     # çıkarılır: ϑ' = ϑ − ⌊cur/1440⌋·1440.  "En fazla Xdev sapar" → grup-başına ortak
598     # bant c_f: tüm günlerin saat-içi zamanı [c_f, c_f+Xdev] içinde (yayılım ≤ Xdev).
599     # Pencerede ortak bant olamayan gruplar (saat-içi yayılım > 2·Wdev+Xdev) atlanır.
600     m._n_g = 0
601     m._n_g_skip = 0
602     if CFG["G_regularity"]:
603         Xd = CFG["xdev_min"]
604         reach = 2 * CFG["wdev_min"] + Xd
605         offset = lambda e: (inst.ev_by_id[e]["cur"] // 1440) * 1440
606         tod = lambda e: inst.ev_by_id[e]["cur"] - offset(e)        # mevcut saat-içi (dk)
607         reg_groups = []                                  # (gkey, [eids]) düzenlenebilir gruplar
608         for fr, eids in inst.ev_by_flno_role.items():
609             adj = [e for e in eids if inst.ev_by_id[e]["adjustable"]
610                    and len({inst.ev_by_id[x]["day"] for x in eids}) >= 1]
611             # günler arası (her gün bir olay; gün-içi anomali tekrarları dahil edilir)
612             days = {inst.ev_by_id[e]["day"] for e in adj}
613             if len(adj) < 2 or len(days) < 2:
614                 continue
615             tods = [tod(e) for e in adj]
616             if max(tods) - min(tods) > reach:
617                 m._n_g_skip += 1                         # pencerede ortak bant yok
618                 continue
619             reg_groups.append((fr, adj))
620         if reg_groups:
621             m.creg = pyo.Var(range(len(reg_groups)), domain=pyo.Reals)
622             for gi, (fr, adj) in enumerate(reg_groups):
623                 for e in adj:
624                     th = theta_expr(m, inst, e) - offset(e)
625                     m.cons.add(m.creg[gi] <= th)
626                     m.cons.add(th <= m.creg[gi] + Xd)
627                     m._n_g += 1
```

Yolcular "her gün 08:30 uçuşu" bekler. G kısıtı, aynı uçuş numarasının farklı günlerdeki saatlerinin birbirine yakın kalmasını sağlar. İki teknik ayrıntı kritiktir:

1) Saat-içi (time-of-day) dönüşümü

θ **mutlak** dakikadır, yani gün 3'ün 08:00'i ile gün 1'in 08:00'i arasında 2×1440 dakika fark vardır. Düzenlilik ise **saat** düzenliliğidir. Bu yüzden her olaydan kendi gününün gece yarısı ofseti çıkarılır: `θ' = θ − ⌊cur/1440⌋·1440` (605–606, 624).

2) Ortak bant değişkeni `c_f`

"En fazla Xdev sapar" ifadesi, grup başına tek bir sürekli değişken `creg[gi]` ile kurulur: tüm günlerin saat-içi zamanı `[c_f, c_f + Xdev]` aralığında olmalı (625–626). Bant nerede olacağını **model seçer**; yalnız genişliği sabittir.

> **Wdev ↔ G gerilimi ve "atlanan" gruplar**
>
> Her olay kendi mevcut saatinden ±60 dk uzaklaşabilir. İki günün mevcut saatleri arasındaki fark 2×60 + 15 = 135 dakikadan büyükse, pencerelerin kesişiminde ortak bir bant **yoktur**. Kod bu grupları sessizce zorlamaz; atlar ve `m._n_g_skip` sayacına ekler (616–618). Belgeli, savunulabilir bir sınırlama.

`src/03_model.py` · satır 628–647 · günler-arası bant kuplajı (byday için)

```
        # --- gün-arası G kuplajı (decomposition): önceki günlerden gelen bant ----
        # inst.gband[(flno,role)] = yerleşmiş saat-içi merkez. Bu günün aynı uçuşu o
        # merkezin ±Xdev'inde tutulur (pencere ulaşamıyorsa atlanır → kayıt edilir).
        gband = getattr(inst, "gband", None)
        if gband:
            for e in inst.ev_by_id.values():
                if not e["adjustable"]:
                    continue
                key = (e["flno"], e["role"])
                c0 = gband.get(key)
                if c0 is None:
                    continue
                etod = e["cur"] - (e["cur"] // 1440) * 1440
                if abs(etod - c0) > CFG["wdev_min"] + Xd:
                    m._n_g_skip += 1                    # bu günün penceresi banda ulaşamıyor
                    continue
                th = theta_expr(m, inst, e["id"]) - (e["cur"] // 1440) * 1440
                m.cons.add(th >= c0 - Xd)
                m.cons.add(th <= c0 + Xd)
                m._n_g += 1
```

`byday` modunda günler ayrı çözüldüğü için üstteki gün-içi bant yetmez. `inst.gband` sözlüğü **önceki günlerde yerleşmiş** merkezleri taşır; bu blok bugünün aynı uçuşunu o merkezin ±Xdev'inde tutar. Pencere merkeze yetişemiyorsa yine atlanır ve sayılır.

3.13 (E2) En iyi yolculuk süresi dengesi — açık ama tam ölçekte kapalı

`src/03_model.py` · satır 649–684 · Jmin min-seçimi ve yön karşılaştırması

```
    # --- (E2) en iyi yolculuk süresi (Jmin) dengesi ------------------------
    if CFG["E2_balance"]:
        ids = list(md_id.values())
        m.Jmin = pyo.Var(ids, domain=pyo.Reals)
        m.act = pyo.Var(ids, domain=pyo.Binary)
        m.u = pyo.Var(P, domain=pyo.Binary)            # Jmin'i veren bağlantı seçici
        for k, g in inst.md.items():
            i, cs = md_id[k], g["conns"]
            Mexpr = len(cs)
            n_md = sum(m.y[pi] for pi in cs)
            m.cons.add(m.act[i] <= n_md)
            m.cons.add(n_md <= Mexpr * m.act[i])
            Jlo = min(inst.conns[pi]["legsum"] + inst.conns[pi]["gmin"] for pi in cs)
            Jhi = max(inst.conns[pi]["legsum"] + inst.conns[pi]["gmax"] for pi in cs)
```

```
            g["_Jlo"], g["_Jhi"] = Jlo, Jhi
            MJ = max(1.0, Jhi - Jlo)
            for pi in cs:
                Jpi = inst.conns[pi]["legsum"] + gap[pi]
                m.cons.add(m.Jmin[i] <= Jpi + MJ * (1 - m.y[pi]))     # ≤ her sunulu J
                m.cons.add(m.Jmin[i] >= Jpi - MJ * (1 - m.u[pi]))     # ≥ seçilen J
                m.cons.add(m.u[pi] <= m.y[pi])
            m.cons.add(sum(m.u[pi] for pi in cs) == m.act[i])         # tam 1 seçili (aktifse)
        Gam = CFG["gamma_min"]
        seen = set()
        for k, g in inst.md.items():
            (mk, day) = k
            o, dd = mk.split("-")
            rk = (f"{dd}-{o}", day)
            if rk in inst.md and (rk, k) not in seen and (k, rk) not in seen:
                seen.add((k, rk))
                i, j = md_id[k], md_id[rk]
                gr = inst.md[rk]
                ME = max(1.0, max(g["_Jhi"], gr["_Jhi"]) - min(g["_Jlo"], gr["_Jlo"]))
                relax = ME * (2 - m.act[i] - m.act[j])
                m.cons.add(m.Jmin[i] - m.Jmin[j] <= Gam + relax)
                m.cons.add(m.Jmin[j] - m.Jmin[i] <= Gam + relax)
```

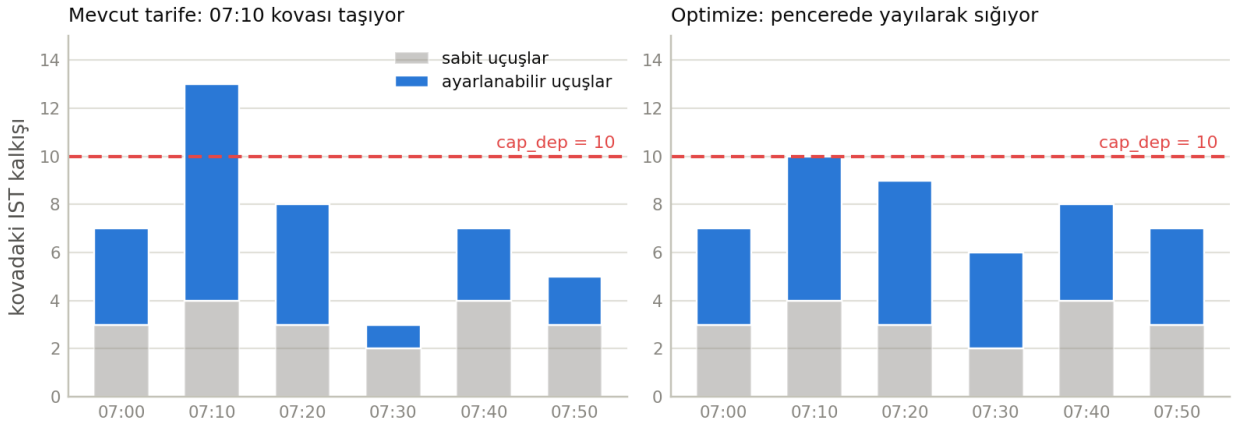
E2, gidiş ve dönüş yönlerinin **en iyi yolculuk sürelerinin** birbirinden 30 dakikadan fazla ayrılmamasını ister. Zor kısmı "minimum" işlemini doğrusallaştırmaktır:

1. `Jmin[i] ≤ J_π + MJ(1 − y_π)` — her sunulan bağlantıdan **küçük veya eşit** (bu tek başına Jmin'i −∞'a iter).
2. `Jmin[i] ≥ J_π − MJ(1 − u_π)` — **seçilen** bağlantıdan büyük veya eşit.
3. `Σ u_π == act[i]` — tam bir bağlantı "minimumu veren" olarak seçilmeli.
4. `act[i]` — bu pazarda hiç bağlantı sunulmuyorsa Jmin tanımsızdır; `act` bunu yakalar ve karşılaştırma kısıtı `relax` terimiyle etkisizleşir (682).

Neden tam ölçekte kapatıldı?

Bu yapı bağlantı başına **iki** big-M kısıtı ve bir ek ikili değişken (`u`) getirir. Tam veride HiGHS, doğrusal gevşemesi çok zayıf olan bu modelde makul sürede tek bir uygun çözüm (incumbent) bile bulamıyor. Tanı yöntemi kodda yazılı: çekirdek + yalnız E2 çözülemiyor, çekirdek + A2/G/F/E1 hepsi hızlı. Küçük alt-örnekte E2 açık ve doğrulanmış hâlde bırakılmıştır.

3.14 (F) IST hub kova kapasitesi



Şekil 3.7 — Kaydırma, taşan kovayı komşulara yayarak çözer.

`src/03_model.py` · satır 686–725 · kova başına kapasite kısıtları

```
    # --- (F) IST hub kova kapasitesi ------------------------------------
    # Her grid kovası t (mutlak dk; her t tek bir (gün, saat-içi)) için ayarlanabilir
    # atamalar + o kovadaki sabit uçuşlar ≤ brief baz kapasitesi (kalkış 10 / varış 15).
    # VERİ BULGUSU (doğrulandı): mevcut tarifedeki kova aşımları TAMAMEN ayarlanabilir
    # uçuşların mevcut saatlerinde kümelenmesinden gelir; SABİT (kımıldamayan) uçuşlar
    # hiçbir kovada baz cap'i aşmaz (kalkış max 4≤10, varış max 5≤15). Dolayısıyla baz
```

```
692     # 10/15 brief'e sadık biçimde UYGULANABİLİR: ayarlanabilir uçuşlar ±Wdev penceresinde
693     # yayılarak rezidüel kapasiteye sığar (gün başına ort. ~4-5/kova, ~120 kova). Mevcut
694     # tarife artık warm-start için uygun değil, ama appsi-Highs MIP-start kullanmadığından
695     # bu maliyetsiz; F gerçek bir kısıt olur (önceki max(baz,mevcut) F'i fiilen bağlamıyordu).
696     m._n_f = 0
697     m._n_f_overflow = 0                                  # kımıldatılamayanların tek başına aştığı kova
698     if CFG["F_capacity"]:
699         extra_use = getattr(inst, "extra_use", {})       # modellenmeyen uçuşların sabit kullanımı
700         for role, base in [("dep", CFG["cap_dep"]), ("arr", CFG["cap_arr"])]:
701             terms, cur_use = {}, {}
702             for e in inst.ev_by_id.values():
703                 if e["role"] != role:
704                     continue
705                 cur_use[e["cur"]] = cur_use.get(e["cur"], 0) + 1     # kova kümesi için
706                 if e["adjustable"]:
707                     for v in e["cands"]:
708                         terms.setdefault(v, []).append((e["id"], v))
709             fixed = {}                                   # kova -> sabit uçuş sayısı
710             for e in inst.ev_by_id.values():
711                 if e["role"] == role and not e["adjustable"]:
712                     fixed[e["cur"]] = fixed.get(e["cur"], 0) + 1
713             for (ro, v), n in extra_use.items():         # modellenmeyen pazarların uçuşları da
714                 if ro == role:                           # IST kovalarını fiziksel olarak doldurur
715                     fixed[v] = fixed.get(v, 0) + n
716             for v in set(terms) | set(cur_use) | set(fixed):
717                 tv = [m.x[eid, vv] for eid, vv in terms.get(v, [])]
718                 fc = fixed.get(v, 0)
719                 cap = base                               # brief §4.6: gerçek rezidüel cap (10/15)
720                 if fc > cap:                             # kımıldatılamayanlar tek başına aşıyor →
721                     m._n_f_overflow += 1                 # ayarlanabilir hiç giremez (veri-taşması raporlanır)
722                     cap = fc
723                 if len(tv) + fc > cap:                   # ancak bağlayabilirse ekle
724                     m.cons.add(sum(tv) + fc <= cap)
725                     m._n_f += 1
```

IST her 10 dakikalık dilimde sınırlı sayıda hareket kaldırabilir: en fazla 10 kalkış, 15 varış. Kod her rol için üç kaynağı toplar:

1. **Ayarlanabilir olaylar** — `terms[v]` sözlüğü, o kovaya *atanabilecek* tüm `x[eid,v]` değişkenlerini toplar (706–708).
2. **Sabit olaylar** — kımıldayamadıkları için mevcut kovalarına sabit sayı olarak eklenir (710–712).
3. **Modellenmeyen uçuşlar** — `extra_use`'dan gelen fiziksel doluluk (713–715).

Sonra her kova için `Σ x + sabit ≤ cap` kısıtı eklenir — ama **yalnız gerçekten bağlayıcı olabiliyorsa** (723). Toplam aday sayısı zaten kapasitenin altındaysa kısıt eklenmez; gereksiz kısıt = daha yavaş çözücü.

> **Kritik veri bulgusu (satır 689–695)**
>
> Mevcut tarifedeki kova aşımlarının **tamamı** ayarlanabilir uçuşların kümelenmesinden geliyor; sabit uçuşlar hiçbir kovada baz kapasiteyi aşmıyor (kalkışta en fazla 4 ≤ 10, varışta 5 ≤ 15). Bu bulgu sayesinde brief'in **gerçek** 10/15 değeri uygulanabildi. Aksi hâlde `max(baz, mevcut)` gibi gevşetilmiş bir kapasite kullanmak gerekirdi ki bu, F'yi fiilen etkisiz bir kısıta çevirirdi.

718–722 arası bir emniyet valfidir: eğer sabit uçuşlar tek başına kapasiteyi aşıyorsa (veri taşması), kapasite o değere yükseltilir ve `_n_f_overflow` sayacı durumu raporlar — model infeasible olmaz ama sorun gizlenmez de.

3.15 Amaç fonksiyonu ve (P) tarife-değişim cezası

`src/03_model.py` · satır 727–748 · amacın kurulması

```
727     # --- amaç: Σ ρ·(aktarma ödülü + sıralama ödülü) − λ·(toplam kaydırma) ----
728     for k, g in inst.md.items():
729         i = md_id[k]
730         M = len(g["conns"])
731         for j in range(1, M + 1):
732             obj_terms.append(g["rho"] * Wc[(i, j)] * m.z[i, j])
733     comp_expr = sum(obj_terms)                           # SAF rekabetçilik faydası (cezasız)
```

```
734     # (P) tarife-değişim cezası: A1 her olayda Σ_v x[eid,v]=1 olduğundan, kaydırmanın
735     # mutlak değeri |ϑ-cur| = Σ_v |v-cur|·x[eid,v] — TAM DOĞRUSAL (ek değişken / big-M YOK).
736     # λ = amaç-puanı / uçuş-dk. Yalnız ayarlanabilir olaylar cezalandırılır (sabitler kımıldamaz).
737     lam = CFG["shift_penalty"]
738     shift_terms = []
739     for e in inst.ev_by_id.values():
740         if e["adjustable"]:
741             for v in e["cands"]:
742                 dv = abs(v - e["cur"])
743                 if dv:
744                     shift_terms.append(dv * m.x[e["id"], v])
745     m._shift_expr = sum(shift_terms) if shift_terms else 0.0
746     m._comp_expr = comp_expr
747     m.OBJ = pyo.Objective(expr=(comp_expr - lam * m._shift_expr) if lam > 0 else comp_expr,
748                           sense=pyo.maximize)
```

Amaç iki parçadan oluşur:

> **Amaç = Σ ρ·(aktarma ödülü + sıralama ödülü) − λ·Σ\|θ − cur\|**
>
> **Birinci parça (saf rekabetçilik):** pazar ağırlığı ρ ile çarpılmış, azalan getirili aktarma ödülü (C) ve sıralama ödülü (D). Resmî amaç budur ve raporlanan tüm sonuçlar bu değerdir.
>
> **İkinci parça (ceza):** yapılan toplam kaydırma, λ katsayısıyla amaçtan düşülür.

Cezanın "tam doğrusal" oluşu

Mutlak değer normalde ek değişken ve big-M gerektirir. Burada gerekmiyor — çünkü A1 sayesinde her olayda tam bir `x[eid,v]` = 1'dir. O hâlde:

|θ − cur| = Σ_v |v − cur| · x[eid, v]

Katsayılar (`dv = abs(v - e["cur"])`) tamamen sabittir ve döngüde önceden hesaplanır (739–744). **Sıfır ek değişken, sıfır big-M.** Bu, one-hot kodlama tercihinin en zarif kazancıdır.

λ neden gerekli?

> **Cezasız modelin "dejenerasyon" sorunu**
>
> λ = 0 iken çözücü, amaç değerini **hiç değiştirmeyen** kaydırmaları da yapar — çünkü onları yapmamak için bir nedeni yoktur. Sonuç: uçuşların %90'ı boşuna kaydırılır. Havayolu için bu, sıfır kazanç karşılığı devasa bir operasyonel maliyettir. λ > 0 simetriyi kırar ve yalnız **gerçekten kazanç getiren** kaydırmalar hayatta kalır.

`src/03_model.py` · satır 749–757 · modelin yanına iliştirilen referanslar

```
749
750     m._md_id = md_id
751     m._gap = gap
752     m._w = w
753     m._beat_eq_y = beat_eq_y
754     m._beta = beta
755     return m
756
757
```

Fonksiyonun sonunda, model nesnesine alt çizgiyle başlayan beş referans iliştirilir: `_md_id` (pazar-gün → indeks), `_gap` (bağlantı → boşluk ifadesi), `_w` / `_beat_eq_y` / `_beta` (rakip yenme yapıları). Bunlar Pyomo bileşeni değildir, sadece **raporlama ve warm-start için gereken haritalardır** — çözümden sonra `report()`, `solution_total()` ve `set_warmstart()` bunlara ihtiyaç duyar. Alt çizgi öneki "bu bir model bileşeni değil, yardımcı veri" demektir.

`src/03_model.py` · satır 973–976 · λ = 0.3 seçiminin gerekçesi

```
973 # Varsayılan tarife-değişim cezası. λ-taraması (docs §12) λ∈[0.1,1.5] aralığında düz bir plato
974 # gösterdi (rekabetçilik ~+%4.2-4.3, kaydırma ~%22-26); λ=0.3 platonun knee'si (cezasız λ=0
975 # dejenerasyon uçurumunun hemen üstü: en yüksek kazanım + düşük kaydırma).
976 DEFAULT_LAMBDA = 0.3
```

λ değeri tahminle değil, **taramayla** seçilmiştir: λ ∈ [0.1, 1.5] aralığında rekabetçilik kazancı düz bir plato çizer (%4.2–4.3), kaydırma ise %22–26'da kalır. λ = 0.3 bu platonun **dizidir (knee)**: en yüksek kazanç, düşük kaydırma. Tarama `07_sensitivity.py` ile üretilir ve `output/sweep_results.csv`'de saklanır.

3.16 `set_warmstart()` — mevcut tarifeyi başlangıç çözümü yapmak

`src/03_model.py` · satır 758–809 · tüm ikili değişkenlere tutarlı değer yazma

```
758 def set_warmstart(inst, m):
759     """Mevcut tarifeyi (x=mevcut saat) TAM tutarlı uygun çözüm olarak değişkenlere yaz.
760     Büyük-M MIP'te HiGHS'in incumbent bulamaması sorununu çözer (baseline'dan iyileştirir)."""
761     md_id, L, U = m._md_id, inst.L, inst.U
762     cur = {e["id"]: e["cur"] for e in inst.ev_by_id.values()}
763     sv = lambda var, val: var.set_value(val, skip_validation=True)
764     for e in inst.ev_by_id.values():                  # x = mevcut kova
765         if e["adjustable"]:
766             for v in e["cands"]:
767                 sv(m.x[e["id"], v], 1 if v == e["cur"] else 0)
768     yv, Jc = {}, {}
769     for pi, c in enumerate(inst.conns):               # y, J (mevcut snapped boşluk)
770         g = cur[c["dep_eid"]] - cur[c["arr_eid"]]
771         yv[pi] = 1 if (L <= g <= U) else 0
772         Jc[pi] = c["legsum"] + g
773         sv(m.y[pi], yv[pi])
774     for k, g in inst.md.items():                      # z (monoton doldurma)
775         i, M = md_id[k], len(g["conns"])
776         n = sum(yv[pi] for pi in g["conns"])
777         for j in range(1, M + 1):
778             sv(m.z[i, j], 1 if j <= n else 0)
779     for k, g in inst.md.items():                      # w, θ, γ
780         i = md_id[k]
781         beaten = {}
782         for ca, Tc in g["comps"].items():
783             bt = 0
784             for pi in g["conns"]:
785                 if yv[pi] != 1:
786                     continue
787                 if (pi, ca) in m._beat_eq_y:
788                     bt = 1
789                 elif (pi, ca) in m._w:
790                     beats = (cur[inst.conns[pi]["dep_eid"]] - cur[inst.conns[pi]["arr_eid"]]) \
791                         <= (Tc - inst.conns[pi]["legsum"])
792                     sv(m._w[(pi, ca)], 1 if beats else 0)
793                     bt = bt or int(beats)
794             if (i, ca) in m._beta:
795                 sv(m._beta[(i, ca)], bt)
796             beaten[ca] = bt
797         if CFG["ranking"] and g["N"] >= 1 and g["b_od"] is not None:
798             nb = sum(beaten.values())
799             for bn in range(0, g["N"] + 1):
800                 sv(m.w[("gam", i, bn)], 1 if bn == nb else 0)
801     if CFG["E2_balance"]:                             # act, u, Jmin
802         for k, g in inst.md.items():
803             i, cs = md_id[k], g["conns"]
804             offered = [pi for pi in cs if yv[pi] == 1]
805             sv(m.act[i], 1 if offered else 0)
806             amin = min(offered, key=lambda pi: Jc[pi]) if offered else None
807             for pi in cs:
808                 sv(m.u[pi], 1 if pi == amin else 0)
809             sv(m.Jmin[i], min((Jc[pi] for pi in offered), default=g["_Jlo"]))
```

Fonksiyon mevcut tarifeyi alır ve modelin **her** ikili değişkenine tutarlı bir değer yazar. Sıra önemlidir çünkü her adım bir öncekine dayanır:

1. `x` — her olay mevcut kovasında (764–767).
2. `y` — boşluk pencerede mi? (768–773)
3. `z` — n bağlantı sunuluyorsa ilk n slot dolu (774–778).
4. `w`, β — her rakip için mevcut saatlerle yenilip yenilmediği hesaplanır (779–796).
5. γ — yenilen sayısına karşılık gelen tek gösterge 1 (797–800).
6. `act`, `u`, `Jmin` — E2 açıksa min-seçici değişkenler (801–809).

`skip_validation=True` kullanılır çünkü ara adımlarda değerler geçici olarak tutarsız görünebilir; Pyomo'nun doğrulaması burada gereksiz yavaşlık yaratır.

> **03'te bu fonksiyon fiilen kullanılmıyor — ama 08'de hayat kurtarıyor**
>
> CFG'de `warmstart: False`'tur; yorumda "appsi Highs MIP-start'ı kullanmıyor (yalnız LP bazı)" yazar. Ancak `08_fullweek_polish.py` bu fikri genelleştirilmiş hâliyle (`set_start_values`) kullanır ve orada gerçekten HiGHS `setSolution` MIP-start'ına dönüşür. Yani bu fonksiyon, 08'in prototipidir.

3.17 Baseline, rapor ve çözücü

`src/03_model.py` · satır 813–828 · current_baseline — dürüst karşılaştırma tabanı

```
813 def current_baseline(inst):
814     """Mevcut tarifenin (optimizasyon yok) amaç değeri — dürüst karşılaştırma için."""
815     rkW = rank_weight_table(inst.rank)
816     total = 0.0
817     for k, g in inst.md.items():
818         offered = [pi for pi in g["conns"]
819                    if inst.L <= inst.conns[pi]["gap_cur"] <= inst.U]
820         n = len(offered)
821         conn_rew = sum(0.5 ** (j - 1) for j in range(1, n + 1))
822         beaten = 0
823         if CFG["ranking"] and offered and g["comps"]:
824             j_best = min(inst.conns[pi]["legsum"] + inst.conns[pi]["gap_cur"] for pi in offered)
825             beaten = sum(1 for tc in g["comps"].values() if j_best <= tc)
826         rank_rew = rank_reward(rkW, g["N"], g["b_od"], beaten) if CFG["ranking"] else 0.0
827         total += g["rho"] * (conn_rew + rank_rew)
828     return total
```

İyileşme iddiası ancak sağlam bir tabana göre anlamlıdır. Bu fonksiyon **hiçbir optimizasyon yapmadan**, mevcut tarifenin amaç değerini birebir aynı formüllerle hesaplar: pencere içindeki bağlantılar sayılır, azalan getiri ödülü toplanır, en iyi J'nin yendiği rakipler sayılır, sıralama ödülü lookup'tan alınır.

> **Neden ayrı bir fonksiyon?**
>
> Baseline'ı "modelden" almak (örneğin x'leri mevcut saate sabitleyip çözmek) cazip görünür ama tehlikelidir: modeldeki bir hata baseline'a da bulaşır ve iyileşme oranı yanlış görünmez. Bağımsız hesap, hatanın **fark olarak** ortaya çıkmasını sağlar. 05 doğrulayıcısı da baseline'ı bir kez daha, tamamen sıfırdan hesaplar.

`src/03_model.py` · satır 831–874 · report — pazar/gün kırılımı

```
831 def report(inst, m, detail=12):
832     """Pazar-gün kırılımı. Çok pazar varsa yalnız en yüksek katkılı `detail` tanesi
833     ayrıntılı yazılır; toplam her zaman tüm pazarlar üzerinden hesaplanır."""
834     rkW = rank_weight_table(inst.rank)
835     md_id = m._md_id
836     rows = []                                          # (val, k, g, n, beaten, gaps)
837     total = 0.0
838     for k, g in inst.md.items():
839         i = md_id[k]
840         offered = [pi for pi in g["conns"] if pyo.value(m.y[pi]) > 0.5]
841         n = len(offered)
842         conn_rew = sum(0.5 ** (j - 1) for j in range(1, n + 1))
843         gaps = [(pi, pyo.value(m._gap[pi]),
844                  inst.conns[pi]["legsum"] + pyo.value(m._gap[pi])) for pi in offered]
845         beaten = sum(1 for ca in g["comps"]
846                      if (i, ca) in m._beta and pyo.value(m._beta[(i, ca)]) > 0.5)
847         rank_rew = rank_reward(rkW, g["N"], g["b_od"], beaten) if CFG["ranking"] else 0.0
848         val = g["rho"] * (conn_rew + rank_rew)
849         total += val
850         rows.append((val, k, g, n, beaten, conn_rew, rank_rew, gaps))
851
852     rows.sort(key=lambda r: -r[0])
853     print("\n" + "=" * 78)
854     print(f"SONUÇ — pazar/gün kırılımı (en yüksek katkılı {min(detail, len(rows))}/{len(rows)})")
855     print("=" * 78)
856     for val, k, g, n, beaten, conn_rew, rank_rew, gaps in rows[:detail]:
857         print(f"\n{k[0]} (gün {k[1]})  ρ={g['rho']:.0f}  N_rakip={g['N']}  b_od={g['b_od']}")
```

```
        print(f"   sunulan bağlantı: {n}/{len(g['conns'])}  | yenilen rakip: {beaten}/{g['N']}")
        for pi, gv, j in gaps[:6]:
            c = inst.conns[pi]
            print(f"     π#{pi}: boşluk {c['gap_cur']:.0f}→{gv:.0f} dk  J={j:.0f} dk"
                  f"  (legsum={c['legsum']:.0f})")
        if len(gaps) > 6:
            print(f"     … (+{len(gaps) - 6} bağlantı daha)")
        print(f"   ödül: aktarma={conn_rew:.3f}  sıralama={rank_rew:+.3f}"
              f"  → ρ·toplam={val:.1f}")
    sh = shift_value(m)
    print("\n" + "-" * 78)
    print(f"SAF rekabetçilik (raporlanan)     : {total:.2f}")
    print(f"Toplam kaydırma                   : {sh:.0f} uçuş-dk  (λ={CFG['shift_penalty']})")
    print(f"Pyomo amaç (rekabet − λ·kaydırma) : {pyo.value(m.OBJ):.2f}")
    print(f"Mevcut tarife (baseline)          : {current_baseline(inst):.2f}")
    print("-" * 78)
    return total
```

En yüksek katkılı pazarları ayrıntılı, gerisini toplu yazar. Alt kısımdaki dört satır projenin **dürüstlük paneli**dir: saf rekabetçilik, toplam kaydırma, cezalı Pyomo amacı ve baseline yan yana basılır — hiçbiri diğerinin arkasına saklanmaz.

src/03_model.py · satır 878–915 · main — tek örneklik koşu ve tanı çıktısı

```
def main():
    if len(sys.argv) > 1 and sys.argv[1] in PRESETS:
        CFG.update(PRESETS[sys.argv[1]])
        if CFG["days"] is None:                          # gün listesini veriden türet (hardcode YOK)
            CFG["days"] = all_operation_days()
        print(f">> profil: {sys.argv[1]}  (gün {CFG['days']}, "
              f"süre sınırı {CFG['time_limit']}s, gap {CFG['mip_gap']})")
    print(">> örnek kuruluyor …")
    inst = build_instance()
    n_adj = sum(1 for e in inst.ev_by_id.values() if e["adjustable"])
    print(f"   pazar-gün: {len(inst.md)} | bağlantı(π): {len(inst.conns)} | "
          f"olay: {len(inst.events)} (ayarlanabilir {n_adj}) | "
          f"pencere ±{CFG['wdev_min']}dk / Δ{CFG['delta_min']}dk")

    print(">> model kuruluyor …")
    m = build_model(inst)
    on = [n for n, f in [("A1", True), ("A2", CFG["A2_rotation"]), ("B1", True),
                         ("B2", CFG["B2_force_offer"]), ("C", True), ("D", CFG["ranking"]),
                         ("E1", CFG["E1_balance"]), ("E2", CFG["E2_balance"]),
                         ("F", CFG["F_capacity"]), ("G", CFG["G_regularity"])] if f]
    nbin = sum(1 for v in m.component_data_objects(pyo.Var) if v.is_binary())
    print(f"   aktif kısıtlar: {'+'.join(on)}  | A2 bağ:{m._n_a2} (brief-exact {m._n_a2_exact}, "
          f"evren-ankoru {getattr(m, '_n_a2_anchor', 0)}) "
          f"G bağ:{m._n_g}(atlanan {m._n_g_skip}) F bağ:{m._n_f} | ikili değişken: {nbin}")

    if CFG["warmstart"]:
        set_warmstart(inst, m)
        print(f"   warm start: mevcut tarife (obj={pyo.value(m.OBJ):.1f})")

    print(f">> çözülüyor ({CFG['solver']}) …")
    res = solve_model(m)
    print(f"   durum: {res['tc']}" + (f"  | MIP boşluğu: {res['gap']:.3%}"
          if res.get("gap") is not None else "") + f"  | süre: {res['t']:.1f}s")
    if not res["ok"]:
        print("   [!] Uygun çözüm yüklenemedi (infeasible / zaman aşımı).")
        return
    report(inst, m)

```

Argümansız (küçük alt-örnek) ve `full` modlarının akışı: profil uygula → örnek kur → model kur → çöz → raporla. Konsola basılan iki tanı satırı geliştirme sırasında çok iş görür:

- **888–890:** kaç pazar-gün, kaç bağlantı, kaç olay (kaçı ayarlanabilir), pencere ve ızgara ayarları.
- **899–901:** hangi kısıtlar **aktif**, A2 kaç bağ kurdu (kaçı brief-exact, kaçı evren-ankoru), G kaç bağ kurdu ve kaç grup atlandı, F kaç bağ kurdu, toplam kaç ikili değişken var. Bir kısıt beklenmedik şekilde 0 bağ kuruyorsa burada hemen görülür.

src/03_model.py · satır 917–948 · solve_model — çözücü ayarları ve dürüst durum raporu

```
def solve_model(m):
    """appsi Highs ile çöz; zaman/gap sınırları + dürüst durum raporu."""
    import time
    from pyomo.contrib.appsi.solvers import Highs
    opt = Highs()
```

```
    opts = {"threads": 1, "random_seed": 0}    # determinizm (makineler arası tekrar-üretim)
    opts.update(CFG.get("highs_extra_options") or {})      # ops. ek HiGHS ayarları (08 kullanır)
    opt.highs_options = opts
    if CFG["time_limit"] is not None:
        opt.config.time_limit = CFG["time_limit"]
    if CFG["mip_gap"] is not None:
        opt.config.mip_gap = CFG["mip_gap"]
    opt.config.load_solution = False
    if CFG["warmstart"] and opt.warm_start_capable():
        opt.config.warmstart = True
    t0 = time.time()
    res = opt.solve(m)
    dt_ = time.time() - t0
    tc = res.termination_condition
    from pyomo.contrib.appsi.base import TerminationCondition as TC
    ok = tc in (TC.optimal, TC.maxTimeLimit, TC.maxIterations) and \
        res.best_feasible_objective is not None
    gap = None
    try:
        bf, bp = res.best_feasible_objective, res.best_objective_bound
        if bf not in (None, 0) and bp is not None:
            gap = abs(bp - bf) / max(1e-9, abs(bf))
    except Exception:
        pass
    if ok:
        res.solution_loader.load_vars()
    return {"ok": ok, "tc": tc, "gap": gap, "t": dt_}
```

- `threads: 1`, `random_seed: 0` — **determinizm**. Aynı girdi farklı makinede aynı sonucu verir; yarışma teslimi için bu şarttır.
- `load_solution = False` + elle `load_vars()` — çözüm ancak durum uygunsa yüklenir; zaman aşımında çöp değerlerle çalışılmaz.
- `ok` bayrağı (937–938): kabul edilen durumlar optimal / zaman-limiti / iterasyon-limiti **ve** ortada gerçek bir uygun çözüm olması.
- `gap` hesabı (940–945): |sınır − en iyi| / |en iyi| — çözümün optimale ne kadar yakın olduğunun kanıtı.

src/03_model.py · satır 951–970 · shift_value ve solution_total

```
def shift_value(m):
    """Çözümdeki toplam tarife kaydırması (uçuş-dk). Ceza kapalıyken (ifade float 0)."""
    e = getattr(m, "_shift_expr", 0.0)
    return 0.0 if isinstance(e, (int, float)) else float(pyo.value(e))


def solution_total(inst, m):
    """Çözümün ρ-ağırlıklı SAF rekabetçilik toplamı (cezasız, raporlamadan)."""
    rkW = rank_weight_table(inst.rank)
    md_id = m._md_id
    total = 0.0
    for k, g in inst.md.items():
        i = md_id[k]
        n = sum(1 for pi in g["conns"] if pyo.value(m.y[pi]) > 0.5)
        conn = sum(0.5 ** (j - 1) for j in range(1, n + 1))
        beaten = sum(1 for ca in g["comps"]
                     if (i, ca) in m._beta and pyo.value(m._beta[(i, ca)]) > 0.5)
        rr = rank_reward(rkW, g["N"], g["b_od"], beaten) if CFG["ranking"] else 0.0
        total += g["rho"] * (conn + rr)
    return total
```

İki küçük ama sık kullanılan yardımcı: toplam kaydırmayı (uçuş-dakika) ve **cezasız** saf rekabetçilik toplamını döndürürler. Raporlanan tüm "iyileşme" rakamları `solution_total` üzerindendir — yani λ cezası sonucu **süzer** ama **ölçmez**.

3.18 `compute_overnight_guards()` — gece-aşan turların statik koruması

src/03_model.py · satır 979–1024 · slack'in ikiye bölünmesi

```
def compute_overnight_guards(days):
    """(A2×) gece-aşan rotasyon koruması: STATİK slack-bölüşümü sınırları.
    Mevcut tarifede fiziksel turu gün h→h+1'e yayılan (aynı-gün dönüşü olmayan,
    gün h+1 dönüşü brief minimumunu T_OB+τ+T_IB SAĞLAYAN) çiftlerde mevcut pay:
      slack = (dönüş_varış − kalkış) − minimum_tur ≥ 0.
    Pay ikiye bölünür: kalkış en çok slack/2 GEÇ, dönüş en çok slack/2 ERKEN
    kayabilir → θ_in − θ_out ≥ minimum tur YAPISAL olarak garanti (ihlal imkânsız),
```

```
    gün-bağlaşımı ve G-bandı çatışması hiç doğmaz (sınırlar statik/deterministik).
    Mevcutta bile sağlanmayan çiftler eşleşme-yorumu artefaktıdır (o gün tek uçak
    dönmüyor) → sınır konmaz (çözüm-sonrası raporda 'taban' olarak görünür).
    Döner: (rotcap_by_day, rotfloor_by_day) — gün -> {(flno,gün): tavan/taban (dt)}."""
    _, _, tk_g, _, _, fixed_g, rot_g, blocks_g = load_data()
    rp_g = rotation_pairs(rot_g)
    cur_arr_g = {(int(r.FlNo1), int(r.day)): r.ist_arr for r in tk_g.itertuples()}
    cur_dep_g = {(int(r.FlNo2), int(r.day)): r.ist_dep for r in tk_g.itertuples()}
    tau_g = CFG["tau_min"]
    rotcap_by_day, rotfloor_by_day = {}, {}              # (flno,gün) -> tavan/taban (dt)
    if blocks_g and CFG["A2_overnight_guard"]:
        for of, inf in rp_g:
            for d in days:
                dep_c = cur_dep_g.get((of, d))
                if dep_c is None:
                    continue
                a_same = cur_arr_g.get((inf, d))
                if a_same is not None and a_same > dep_c:
                    continue                             # aynı-gün dönüş → model-içi A2 kapsar
                a_next = cur_arr_g.get((inf, d + 1))
                t_ob = blocks_g.get((of, d, "dep"))
                t_ib = blocks_g.get((inf, d + 1, "arr"))
                if a_next is None or a_next <= dep_c or t_ob is None or t_ib is None:
                    continue
                exact = t_ob + tau_g + t_ib
                slack = (a_next - dep_c).total_seconds() / 60.0 - exact
                if slack < 0:
                    continue                             # mevcut bile sağlamıyor → artefakt
                if slack >= 2 * CFG["wdev_min"]:
                    continue                             # pencere ihlale yetişemez → sınır gereksiz
                cap_dt = dep_c + pd.Timedelta(minutes=slack / 2)     # kalkış geç-kayma tavanı
                flr_dt = a_next - pd.Timedelta(minutes=slack / 2)    # dönüş erken-kayma tabanı
                caps = rotcap_by_day.setdefault(d, {})
                kc = (of, d)
                caps[kc] = min(caps.get(kc, cap_dt), cap_dt)
                floors = rotfloor_by_day.setdefault(d + 1, {})
                kf = (inf, d + 1)
                floors[kf] = max(floors.get(kf, flr_dt), flr_dt)
    return rotcap_by_day, rotfloor_by_day
```

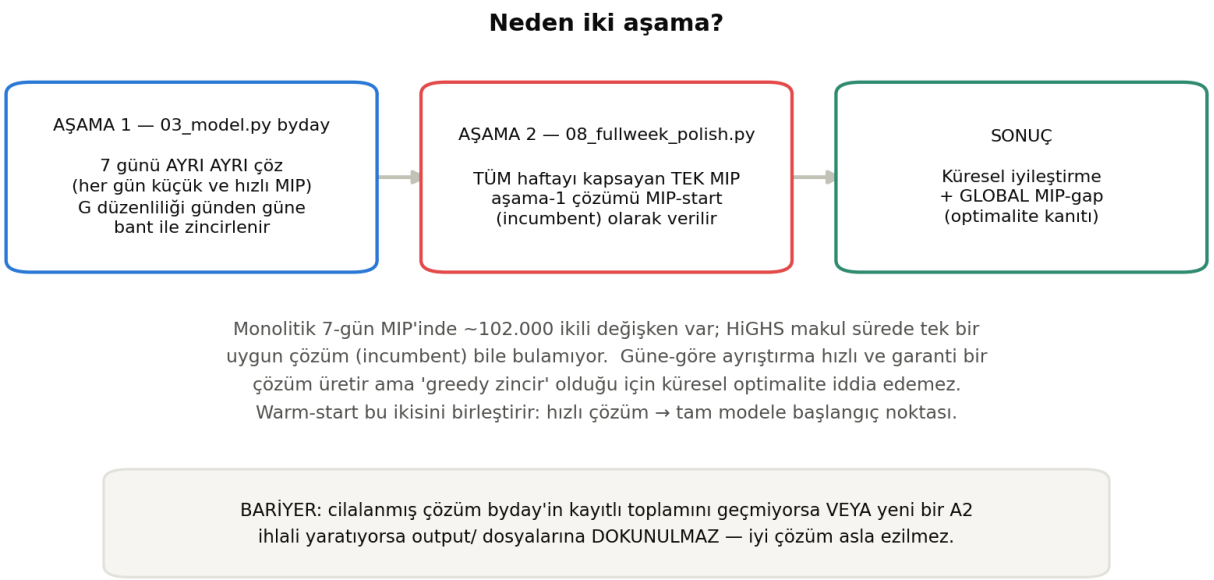
byday ayrıştırmasının tek yapısal zayıflığı, bir günün kalkışı ile ertesi günün dönüşünü aynı modelde görememesidir. Bu fonksiyon o boşluğu **çözüm öncesinde, deterministik olarak** kapatır:

1. Her rotasyon çifti ve gün için mevcut kalkış saati bulunur.
2. Aynı gün dönüş varsa atlanır — model-içi A2 zaten kapsıyor (1002–1004).
3. Ertesi gün dönüş varsa mevcut pay hesaplanır: `slack = (dönüş − kalkış) − (T_OB + τ + T_IB)`.
4. `slack < 0` ise bu bir **eşleşme artefaktıdır** (o gün o çift tek uçakla dönmüyor) → sınır konmaz (1012–1013).
5. `slack ≥ 2·Wdev` ise pencere ihlale zaten yetişemez → sınır gereksiz (1014–1015).
6. Aksi hâlde: kalkışa `+slack/2` tavan, dönüşe `−slack/2` taban konur.

Neden "statik"?

Sınırlar çözümden **bağımsız** hesaplanır ve sabittir. Böylece iki gün birbirini hiç görmese bile ihlal matematiksel olarak imkânsız hâle gelir; gün-bağlaşımı doğmaz, G bandıyla çatışma yaratmaz, çözüm tekrarlanabilir kalır.

3.19 `run_byday()` — haftayı güne bölerek çözmek



Şekil 3.8 — İki aşamalı çözüm mimarisi.

src/03_model.py · satır 1027–1054 · hazırlık: profil, günler, sayaçlar

```
1027 def run_byday(lam=0.0, gap=0.02, time_limit=150, write=True, verbose=True):
1028     """TÜM veriyi GÜNE GÖRE ayrıştırarak çöz (çekirdek). Günleri bağlayan tek kısıt G;
1029     her gün ayrı (küçük, hızlı) çözülür, G önceki günlerden gelen saat-içi bantla
1030     zincirlenir. Monolitik 7-gün MIP (102k ikili) HiGHS'te incumbent bulamadığından gerekli.
1031     lam>0 ise amaca λ·(toplam kaydırma) cezası girer; istatistik sözlüğü döner."""
1032     CFG.update(PRESETS["full"])
1033     CFG["shift_penalty"] = lam
1034     CFG["time_limit"], CFG["mip_gap"] = time_limit, gap
1035     if CFG["days"] is None:                                  # gün listesini veriden türet (hardcode YOK)
1036         CFG["days"] = all_operation_days()
1037     days = CFG["days"]
1038     if verbose:
1039         print(f">> GÜNE-GÖRE ayrıştırma  (λ={lam} obj/uçuş-dk, gap≤{gap:.2%}, "
1040               f"gün-başına ≤{time_limit}s, günler {days})")
1041     band = {}                                               # (flno,role) -> yerleşmiş saat-içi
1042     g_opt = g_base = 0.0
1043     g_shift = 0
1044     ok_days = relaxed_days = 0
1045     n_band_dropped = 0                                      # hedefli G-gevşetmede bırakılan bant
1046     band_exempt_rows = []                                   # (gün, uçuş, rol): bandı bırakılan olaylar
1047     sched_rows, day_stats = [], []                          # çıktı tarifesi + özet
1048     market_rows, conn_rows = [], []                         # O–D/gün + seçilen-bağlantı (teslim #3)
1049     final_dt = {}                                           # (flno,gün,rol) -> optimize GERÇEK zaman
1050     cur_dt = {}                                             # (flno,gün,rol) -> MEVCUT gerçek zaman
1051     blocks_ref = rot_pairs_ref = None                       # A2 gece-aşan doğrulaması için
1052
1053     # (A2×) gece-aşan rotasyon koruması — statik sınırlar (bkz. compute_overnight_guards)
1054     rotcap_by_day, rotfloor_by_day = compute_overnight_guards(days)
```

Fonksiyonun docstring'i sorunu net koyar: "*Monolitik 7-gün MIP (102k ikili) HiGHS'te incumbent bulamadığından gerekli.*" Yani bu bir tercih değil, **zorunluluktur**. Günleri bağlayan tek kısıt G'dir ve o da bantla zincirlenir.

src/03_model.py · satır 1056–1088 · gün döngüsü ve iki aşamalı G gevşetmesi

```
1056     for d in days:
1057         CFG["days"] = [d]
1058         try:
1059             inst = build_instance()
1060             inst.gband = band
1061             inst.rotcap = rotcap_by_day.get(d, {})
1062             inst.rotlock = rotfloor_by_day.get(d, {})
1063             m = build_model(inst)
1064             res = solve_model(m)
1065             relaxed = False
1066             if not res["ok"]:                               # bant kuplajı aşırı-kısıtladı →
1067                 # 1. aşama (HEDEFLİ): yalnız rotasyon-korumalı uçuşların bandını bırak
```

```
                guarded = {f for (f, _d) in list(inst.rotcap) + list(inst.rotlock)}
                if guarded and inst.gband:
                    dropped = [k for k in band if k[0] in guarded]
                    inst.gband = {k: v for k, v in band.items() if k[0] not in guarded}
                    m = build_model(inst)
                    res = solve_model(m)
                    n_band_dropped += len(dropped)
                    if res["ok"]:                          # muafiyet fiilen kullanıldı → belgele
                        for (fl, ro) in dropped:           # (rotasyon güvenliği > G bandı önceliği)
                            band_exempt_rows.append(dict(
                                gun=d, ucus_no=fl,
                                rol=("kalkış" if ro == "dep" else "varış")))
                if not res["ok"]:                          # 2. aşama: o gün G'yi tam gevşet
                    inst.gband = {}
                    m = build_model(inst)
                    res = solve_model(m)
                    relaxed = True
            except Exception as ex:                        # gizli veride tek gün hatası TÜM koşuyu çökertmesin
                if verbose:
                    print(f"   gün {d}: ATLANDI (hata: {type(ex).__name__}: {ex})")
                continue
```

Her gün için örnek kurulur, önceki günlerden gelen bant (`inst.gband`) ve rotasyon sınırları eklenir, model çözülür. Çözülemezse **kademeli** bir gevşetme uygulanır:

Aşama	Ne yapılır?	Mantık
1 — Hedefli	Yalnız **rotasyon korumalı** uçuşların G bandı bırakılır	Rotasyon güvenliği (fiziksel) > tarife düzenliliği (konfor). Bırakılan her bağ `band_exempt_rows`'a yazılır
2 — Tam	O gün için G tamamen gevşetilir	Son çare; `relaxed` bayrağı raporda "G-gevşetildi" olarak görünür

1085–1088: Tüm blok `try/except` içindedir. Gizli veride tek bir günün hatası tüm koşuyu çökertmemeli; o gün atlanır, sebebi yazılır, diğerleri devam eder.

`src/03_model.py` · satır 1089–1134 · çözümün işlenmesi ve tarife satırlarının üretilmesi

```
        if not res["ok"]:
            if verbose:
                print(f"   gün {d}: ÇÖZÜLEMEDİ ({res['tc']}, {res['t']:.0f}s)")
            continue
        relaxed_days += int(relaxed)
        opt = solution_total(inst, m)
        base = current_baseline(inst)
        g_opt += opt
        g_base += base
        ok_days += 1
        # tüm olayların optimize IST zamanı (sabitler yerinde durur)
        new_time = {e["id"]: (round(sum(v * pyo.value(m.x[e["id"], v]) for v in e["cands"]))
                              if e["adjustable"] else e["cur"]) for e in inst.ev_by_id.values()}
        for e in inst.ev_by_id.values():                   # gece-aşan A2 doğrulaması için gerçek zaman
            final_dt[(e["flno"], d, e["role"])] = inst.t0 + pd.Timedelta(minutes=new_time[e["id"]])
            cur_dt[(e["flno"], d, e["role"])] = inst.t0 + pd.Timedelta(minutes=e["cur"])
        if blocks_ref is None:
            blocks_ref, rot_pairs_ref = inst.blocks, inst.rot_pairs
        n_moved = 0
        day_shift = 0
        for e in inst.ev_by_id.values():                   # bantları kaydet + tarife satırı
            if not e["adjustable"]:
                continue
            new_abs = new_time[e["id"]]
            band.setdefault((e["flno"], e["role"]), new_abs - (e["cur"] // 1440) * 1440)
            delta = new_abs - e["cur"]
            n_moved += int(delta != 0)
            day_shift += abs(delta)
            sched_rows.append(dict(
                gun=d, ucus_no=e["flno"],
                rol=("kalkış" if e["role"] == "dep" else "varış"),
                eski_IST=hhmm(e["cur"]), yeni_IST=hhmm(new_abs),
                degisim_dk=delta, G_gevsetildi=int(relaxed)))
        g_shift += day_shift
        if write:                                          # teslim #3: O–D/gün + seçilen bağlantı
            collect_market_outputs(inst, m, d, new_time, hhmm, market_rows, conn_rows)
        gp = f"{res['gap']:.2%}" if res.get("gap") is not None else "?"
        tag = " [G-gevşetildi]" if relaxed else ""
        day_stats.append(dict(gun=d, pazar_gun=len(inst.md), baglanti=len(inst.conns),
```

```
                                OPT=round(opt), baseline=round(base), delta=round(opt - base),
                                gap=gp, ucus_kaydirilan=n_moved, kaydirma_dk=day_shift,
                                G_gevsetildi=int(relaxed)))
        if verbose:
            print(f"   gün {d}: md={len(inst.md):3d} bağ={len(inst.conns):4d} | "
                  f"OPT={opt:>9.0f} baz={base:>9.0f} Δ={opt - base:>+8.0f} | gap={gp:>6} "
                  f"kaydırılan={n_moved} ({day_shift:,}dk) ({res['t']:.0f}s){tag}")
```

- **1100–1101:** Her olayın optimize saati `Σ v·x[e,v]` toplamından okunur ve yuvarlanır. Sabit olaylar mevcut saatlerinde kalır.
- **1113:** `band.setdefault(...)` — bir uçuşun bandı **ilk yerleştiği günde** belirlenir ve sonraki günler ona uyar. `setdefault` bunu tek satırda yapar.
- **1117–1121:** Tarife satırı: gün, uçuş no, rol, eski saat, yeni saat, değişim, G gevşetildi mi. `output/optimized_schedule.csv`'nin ham hâli.
- **1124:** Pazar/bağlantı çıktıları `collect_market_outputs` ile biriktirilir.

src/03_model.py · satır 1135–1174 · A2 fiziksel rotasyon doğrulaması (gece-aşan dahil)

```
    dpct = 100 * (g_opt - g_base) / g_base if g_base else 0
    moved = sum(1 for r in sched_rows if r["degisim_dk"] != 0)
    nfl = len(sched_rows)
    # --- A2 fiziksel rotasyon doğrulaması (gece-aşan turlar dahil) ----------
    # Model-içi A2 aynı-gün kurulur (byday gün-bağlaşımı yaratamaz); burada her IST
    # kalkışının MEVCUT tarifedeki dönüş örneği (aynı-gün veya ertesi gün — uçağın
    # gerçek eşleşmesi) sabitlenir ve optimize saatlerde brief minimumuna
    # (T_OB+τ+T_IB) uyduğu GERÇEK zaman ekseninde doğrulanır. Mevcut tarifenin
    # kendisinin sağlamadığı eşleşmeler artefakttır (o gün o çift tek uçakla
    # dönülmüyordur) → 'taban' olarak ayrıca raporlanır, ihlal sayılmaz.
    a2x_chk = a2x_viol = a2x_base = 0
    if blocks_ref is not None and rot_pairs_ref:
        tau = CFG["tau_min"]
        for out_f, in_f in rot_pairs_ref:
            for d in days:
                t_ob = blocks_ref.get((out_f, d, "dep"))
                dep_c = cur_dt.get((out_f, d, "dep"))
                dep_o = final_dt.get((out_f, d, "dep"))
                if dep_c is None or dep_o is None or t_ob is None:
                    continue
                # dönüş örneğini MEVCUT tarife belirler (ilk cur-varış > cur-kalkış)
                pick = None
                for d2 in (d, d + 1):
                    a_c = cur_dt.get((in_f, d2, "arr"))
                    tb = blocks_ref.get((in_f, d2, "arr"))
                    if a_c is not None and tb is not None and a_c > dep_c:
                        pick = (d2, a_c, tb)
                        break
                if pick is None:
                    continue
                d2, a_c, tb = pick
                a_o = final_dt.get((in_f, d2, "arr"))
                if a_o is None:
                    continue
                a2x_chk += 1
                need = pd.Timedelta(minutes=t_ob + tau + tb)
                if a_c < dep_c + need:                  # mevcut bile sağlamıyor → artefakt
                    a2x_base += 1
                elif a_o < dep_o + need:                # optimizasyonun YENİ yarattığı ihlal
                    a2x_viol += 1
```

Model-içi A2 aynı-gün kurulur; bu blok çözümü **gerçek zaman ekseninde**, gece-aşan turlar dahil kontrol eder. Ayrım son derece dürüsttür:

Sonuç	Koşul	Yorum
artefakt	Mevcut tarife de sağlamıyor (`a_c < dep_c + need`)	O gün o çift tek uçakla dönülmüyor — eşleşme yorumunun sınırı. İhlal sayılmaz ama **sayılır ve raporlanır**
YENİ ihlal	Mevcut sağlıyor ama optimize bozuyor	Gerçek hata. Bu sayı 0 olmalıdır

src/03_model.py · satır 1175–1199 · istatistikler ve özet çıktısı

```
    stats = dict(lam=lam, opt=g_opt, base=g_base, dpct=dpct, shift=g_shift,
                 moved=moved, nflights=nfl, movedpct=100 * moved / max(1, nfl),
```

```
                ok_days=ok_days, n_days=len(days), relaxed_days=relaxed_days,
                a2x_chk=a2x_chk, a2x_viol=a2x_viol, a2x_base=a2x_base,
                band_exempt=len(band_exempt_rows))
    if verbose:
        print("=" * 70)
        print(f"TAM VERİ ({ok_days}/{len(days)} gün, G-gevşetilen {relaxed_days}) — "
              f"GÜNE-GÖRE AYRIŞTIRMA  (λ={lam})")
        print(f"  OPTIMIZE toplam : {g_opt:>12,.0f}")
        print(f"  BASELINE toplam : {g_base:>12,.0f}")
        print(f"  İYİLEŞME (Δ)    : {g_opt - g_base:>+12,.0f}   ({dpct:+.2f}%)")
        print(f"  Tarife kaydırma : {moved:>5}/{nfl} uçuş hareketi (%{stats['movedpct']:.0f}), "
              f"toplam {g_shift:,} uçuş-dk")
        if a2x_chk:
            print(f"  A2 rotasyon doğrulaması (gece-aşan dahil, gerçek zaman ekseni): "
                  f"{a2x_chk} tur kontrol, YENİ ihlal: {a2x_viol} "
                  f"(mevcut tarifede de aynı yorumda görünen: {a2x_base} — eşleşme artefaktı)")
        if band_exempt_rows:
            print(f"  G hedefli bant-muafiyeti (rotasyon güvenliği önceliği): "
                  f"{len(band_exempt_rows)} (gün,uçuş,rol) — output/g_band_exemptions.csv")
        print("=" * 70)
    if write:
        write_outputs(sched_rows, day_stats, stats, market_rows, conn_rows, band_exempt_rows)
    return stats
```

Özet paneli tek ekranda: kaç gün çözüldü, kaçında G gevşetildi, optimize toplam, baseline, iyileşme (mutlak ve yüzde), kaç uçuş kaydırıldı, toplam kaç uçuş-dakika, A2 doğrulama sonucu, bant muafiyeti sayısı.

3.20 Çıktıların üretilmesi

src/03_model.py · satır 1202–1232 · collect_market_outputs — teslim #3'ün iki dosyası

```
def collect_market_outputs(inst, m, day, new_time, hhmm, market_rows, conn_rows):
    """O–D/gün özeti (yenilen rakip, sıralama, ödüller) ve seçilen her bağlantı satırını
    biriktirir (teslim #3: 'seçilen bağlantılar' + 'O–D bazında yenilen rakipler')."""
    rkW = rank_weight_table(inst.rank)
    md_id = m._md_id
    for k, g in inst.md.items():
        i = md_id[k]
        offered = [pi for pi in g["conns"] if pyo.value(m.y[pi]) > 0.5]
        n = len(offered)
        conn_rew = sum(0.5 ** (j - 1) for j in range(1, n + 1))
        beaten_cas = [ca for ca in g["comps"]
                      if (i, ca) in m._beta and pyo.value(m._beta[(i, ca)]) > 0.5]
        beaten = len(beaten_cas)
        rank_rew = rank_reward(rkW, g["N"], g["b_od"], beaten) if CFG["ranking"] else 0.0
        market_rows.append(dict(
            gun=day, pazar=k[0], rho=round(g["rho"]), N_rakip=g["N"],
            sunulan_baglanti=n, aday_baglanti=len(g["conns"]),
            yenilen_rakip=beaten, baslangic_sira_bod=g["b_od"],
            son_sira=(g["N"] - beaten) if g["N"] >= 1 else None,
            aktarma_odul=round(conn_rew, 4), siralama_odul=round(rank_rew, 4),
            rho_katki=round(g["rho"] * (conn_rew + rank_rew), 2)))
        for pi in offered:
            c = inst.conns[pi]
            ea, ed = inst.ev_by_id[c["arr_eid"]], inst.ev_by_id[c["dep_eid"]]
            gv = pyo.value(m._gap[pi])
            J = c["legsum"] + gv
            beats = [ca for ca, Tc in g["comps"].items() if J <= Tc]
            conn_rows.append(dict(
                gun=day, pazar=k[0], gelen_ucus=ea["flno"], giden_ucus=ed["flno"],
                IST_varis=hhmm(new_time[c["arr_eid"]]), IST_kalkis=hhmm(new_time[c["dep_eid"]]),
                bosluk_dk=round(gv), J_dk=round(J), yendigi_rakip=len(beats)))
```

Her pazar-gün için bir özet satırı (`market_rows`) ve sunulan her bağlantı için bir ayrıntı satırı (`conn_rows`) üretir. Bu iki tablo, brief'in "seçilen bağlantılar" ve "O–D bazında yenilen rakipler" taleplerinin doğrudan karşılığıdır.

src/03_model.py · satır 1235–1240 · main_byday — teslim modu

```
def main_byday(lam=None):
    """Varsayılan (DEFAULT_LAMBDA) cezayla tam veri koşusu + çıktı yazımı.
    gap=0.1%: kalibre edildi — 0.5% raporlanan (saf) amaçta ~0.5 puan bırakıyor;
    0.01% ise cezalı amacı sıkarken saf rekabetçiliği DÜŞÜRÜyor (ceza-ödün etkisi)."""
    lam = DEFAULT_LAMBDA if lam is None else lam
    run_byday(lam=lam, gap=(0.001 if lam > 0 else 0.02), time_limit=150, write=True)
```

`gap = 0.001` (%0.1) seçimi kalibre edilmiştir: %0.5 raporlanan amaçta ~0.5 puan bırakıyor, %0.01 ise cezalı amacı sıkarken **saf rekabetçiliği düşürüyor** — çünkü çözücü cezayı azaltmak için ödülden ödün veriyor. %0.1 dengeli noktadır.

src/03_model.py · satır 1243–1267 · main_sweep — λ Pareto taraması

```
def main_sweep(lams=None, gap=0.001, time_limit=150):
    """λ-TARAMASI: rekabetçilik-kazanımı ↔ tarife-değişimi ödünleşim (Pareto) tablosu.
    Cezasız (λ=0) referansa göre korunan kazanım yüzdesini ve %kaydırılan'ı verir."""
    lams = lams if lams is not None else [0.0, 0.1, 0.3, 0.7, 1.5]
    print(f">> λ-TARAMASI  (gap≤{gap:.2%}, ≤{time_limit}s/gün)  — ödünleşim: "
          f"rekabetçilik vs tarife-değişimi\n")
    rows = []
    for lam in lams:
        s = run_byday(lam=lam, gap=gap, time_limit=time_limit, write=False, verbose=False)
        rows.append(s)
        print(f"   λ={lam:<5}: Δ={s['opt'] - s['base']:>+9,.0f} ({s['dpct']:+.2f}%)  | "
              f"kaydırılan %{s['movedpct']:>3.0f} ({s['moved']}/{s['nflights']})  | "
              f"{s['shift']:>8,} uçuş-dk  [G-gevş {s['relaxed_days']}]")
    base_gain = (rows[0]["opt"] - rows[0]["base"]) if rows else 0.0
    print("\n" + "=" * 72)
    print(f"{'λ':>6} | {'Δ rekabet':>12} | {'korunan':>8} | {'kaydırılan':>11} | {'uçuş-dk':>10}")
    print("-" * 72)
    for s in rows:
        gain = s["opt"] - s["base"]
        keep = 100 * gain / base_gain if base_gain else 0.0
        print(f"{s['lam']:>6} | {gain:>+12,.0f} | {keep:>6.0f}%  | {s['movedpct']:>9.0f}%  | "
              f"{s['shift']:>10,}")
    print("=" * 72)
    print("Not: 'korunan' = λ=0 kazanımının yüzdesi. Knee = yüksek korunan + düşük kaydırma.")
    return rows
```

Beş λ değeri için `write=False` ile koşar (teslim çıktılarına dokunmaz) ve "korunan kazanım yüzdesi" ile "kaydırılan uçuş yüzdesini" karşılaştıran bir tablo basar. Bu tablo, λ seçiminin gerekçesidir.

src/03_model.py · satır 1270–1347 · write_outputs — dört CSV ve sonuç özeti

```
def write_outputs(sched_rows, day_stats, stats, market_rows=None, conn_rows=None,
                  band_exempt_rows=None):
    """Teslim #3 çıktıları: tarife (optimized_schedule.csv), O-D/gün sonuçları
    (market_results.csv), seçilen bağlantılar (selected_connections.csv) + özet (.md)."""
    out = os.path.join(ROOT, "output")
    os.makedirs(out, exist_ok=True)
    sched = pd.DataFrame(sched_rows).sort_values(["gun", "ucus_no", "rol"])
    sched.to_csv(os.path.join(out, "optimized_schedule.csv"), index=False, encoding="utf-8-sig")
    if market_rows:                                   # O-D/gün: yenilen rakip + ödüller
        pd.DataFrame(market_rows).sort_values(["gun", "rho_katki"], ascending=[True, False]) \
          .to_csv(os.path.join(out, "market_results.csv"), index=False, encoding="utf-8-sig")
    if conn_rows:                                     # seçilen (sunulan) her bağlantı
        pd.DataFrame(conn_rows).sort_values(["gun", "pazar"]) \
          .to_csv(os.path.join(out, "selected_connections.csv"), index=False, encoding="utf-8-sig")
    # G hedefli bant-muafiyetleri (şeffaflık: doğrulayıcı bu listeyi okur; boşsa da yazılır)
    pd.DataFrame(band_exempt_rows or [], columns=["gun", "ucus_no", "rol"]) \
      .to_csv(os.path.join(out, "g_band_exemptions.csv"), index=False, encoding="utf-8-sig")
    g_opt, g_base, dpct = stats["opt"], stats["base"], stats["dpct"]
    moved, nfl, lam = stats["moved"], stats["nflights"], stats["lam"]
    avg_shift = stats["shift"] / max(1, moved)
    pen_line = (f"- **Tarife-değişim cezası (λ):** {lam} amaç-puanı / uçuş-dk "
                f"({'açık' if lam > 0 else 'KAPALI — referans'})")
    L = ["# THY Tarife Optimizasyonu — Sonuç Özeti", "",
         "**Takım: Büke** · TEKNOFEST · Yapay Zeka Destekli Havayolu Optimizasyonu", "",
         f"*Üretim: {dt.datetime.now():%Y-%m-%d %H:%M} · `python src/03_model.py byday`*", "",
         "## Tam veri sonucu (güne göre ayrıştırma)", "",
         f"- **Çözülen gün:** {stats['ok_days']}/{stats['n_days']}  "
         f"(G-gevşetilen {stats['relaxed_days']})",
         pen_line,
         f"- **Ayarlanabilir uçuş hareketi:** {nfl} (kaydırılan {moved}, "
         f"%{stats['movedpct']:.0f}; toplam {stats['shift']:,} uçuş-dk, ort. {avg_shift:.0f} dk)",
         f"- **OPTIMIZE amaç (saf rekabetçilik):** {g_opt:,.0f}",
         f"- **BASELINE (mevcut tarife):** {g_base:,.0f}",
         f"- **İYİLEŞME:** {g_opt-g_base:+,.0f}  (**{dpct:+.2f}%** gelir-ağırlıklı rekabetçilik faydası)",
         "", "## Gün kırılımı", "",
         "| Gün | Pazar-gün | Aday bağlantı | OPT | Baseline | Δ | gap | Kaydırılan | Kaydırma-dk | G-gevş. |",
         "|----|----|----|----|----|----|----|----|----|----|"]
    for s in day_stats:
        L.append(f"| {s['gun']} | {s['pazar_gun']} | {s['baglanti']} | {s['OPT']:,} | "
                 f"{s['baseline']:,} | {s['delta']:+,} | {s['gap']} | {s['ucus_kaydirilan']} | "
                 f"{s.get('kaydirma_dk', 0):,} | {'evet' if s['G_gevsetildi'] else '—'} |")
    L += ["", "## Notlar (bkz. `docs/model_formulation.md` §10–§12)", "",
```

```
            f"- **Tarife-değişim cezası (P, docs §12):** amaca −λ·Σ|θ−cur| eklenir (tam doğrusal: "
            f"A1 sayesinde Σ_v|v−cur|·x). λ={lam}. Amaç değişimi cezalandırmadığında çözücü "
            "uçuşları keyfi kaydırıyordu (bedava/dejenere kaydırma); λ bunları geri çeker.",
            "- 'OPTIMIZE amaç' ve 'İYİLEŞME' SAF rekabetçiliktir (cezasız ölçülür); λ yalnızca "
            "hangi kaydırmaların yapılacağını süzer.",
            "- Kısıtlar: A1,A2,B1,C,D,E1,F,G gün-içinde tam uygulanır; G günler-arası saat-içi",
            "  bantla zincirlenir (çatışan günde gevşetilir)."
            + (f" Rotasyon-güvenliği önceliğiyle bandı bırakılan (gün,uçuş,rol): "
               f"{stats.get('band_exempt', 0)} — `output/g_band_exemptions.csv`."
               if stats.get("band_exempt") else ""),
            "- A2 rotasyonu brief-exact minimum turla (T_OB+τ+T_IB, bacak blokları veriden) kurulur;"
            " gece-aşan turlar statik slack-bölüşümüyle korunur"
            + (f" — doğrulama: {stats.get('a2x_chk', 0)} tur, YENİ ihlal {stats.get('a2x_viol', 0)}"
               f" (mevcut tarifede de görünen {stats.get('a2x_base', 0)} eşleşme-artefaktı hariç)."
               if stats.get("a2x_chk") else "."),
            "- E2 (Jmin zaman dengesi) tam-ölçekte kapalı (büyük-M min-seçimi HiGHS'i kilitliyor);",
            "  küçük örnekte açık ve doğrulanmış.",
            "- F kova kapasitesi brief'in gerçek 10/15'ini uygular (sabit uçuşlar baz cap'i hiç",
            "  aşmadığından rezidüel cap ≥0; ayarlanabilirler pencerede yayılarak sığar).",
            "  Modellenmeyen (ρ'suz → amaç dışı) pazarların uçuşları da kovalarda SABİT doluluk",
            "  olarak sayılır (fiziksel kapasite tüm TK uçuşlarıyla korunur).",
            "- `eski_IST`/`yeni_IST` 10 dk ızgaraya yuvarlanmış saat-içi değerlerdir.",
            "", "## Çıktı dosyaları (teslim #3)", "",
            "- `optimized_schedule.csv` — ayarlanan IST saatleri (uçuş × eski→yeni).",
            "- `market_results.csv` — O–D/gün: sunulan bağlantı, **yenilen rakip**, sıralama, ρ-katkı.",
            "- `selected_connections.csv` — seçilen her bağlantı (gelen/giden uçuş, boşluk, J, yendiği rakip).",
            "- Toplam amaç değeri (saf rekabetçilik): "
            f"**{g_opt:,.0f}** (baseline {g_base:,.0f}, Δ {g_opt-g_base:+,.0f}).]
    with open(os.path.join(out, "results_summary.md"), "w", encoding="utf-8") as f:
        f.write("\n".join(L) + "\n")
    extra = ""
    if market_rows:
        extra = (f", output/market_results.csv ({len(market_rows)} O–D/gün), "
                 f"output/selected_connections.csv ({len(conn_rows)} bağlantı)")
    print(f">> çıktı yazıldı: output/optimized_schedule.csv ({len(sched)} satır){extra}, "
          f"output/results_summary.md")
```

Dört CSV yazılır ve `results_summary.md` markdown olarak üretilir. Dikkat çeken iki ayrıntı:

- **1285–1286:** `g_band_exemptions.csv` **boş olsa bile** yazılır. Doğrulayıcı bu dosyayı okur; yokluğu ile boşluğu arasındaki farkı belirsiz bırakmamak için.
- **1311–1339:** Özet dosyasının "Notlar" bölümü, sonucun hangi varsayımlarla üretildiğini okuyucuya doğrudan söyler: E2 kapalı, F gerçek 10/15, G zincirleme, A2 gece-aşan koruma, λ'nın rolü. **Sonuçla birlikte sınırlarını da yayınlamak**, bu projenin tutarlı bir tavrıdır.

src/03_model.py · satır 1350–1357 · komut satırı arayüzü

```
if __name__ == "__main__":
    arg = sys.argv[1] if len(sys.argv) > 1 else ""
    if arg == "byday":                                  # tam veri + çıktı; ops. λ: byday 0.3
        main_byday(float(sys.argv[2]) if len(sys.argv) > 2 else None)
    elif arg == "sweep":                                # λ ödünleşim taraması (Pareto tablo)
        main_sweep()
    else:
        main()
```

Üç mod: `byday` (isteğe bağlı λ ile), `sweep`, ve argümansız çağrıda küçük alt-örnek (`main()`). `full` argümanı `main()` içinde PRESETS ile yakalanır.

4 08_fullweek_polish.py — Tam-Hafta Warm-Start Cilalaması

Bu dosya ne yapar?

03'ün güne-göre ürettiği çözümü, **haftanın tamamını kapsayan tek MIP'e** başlangıç noktası (MIP-start) olarak verir ve HiGHS'in bu noktadan küresel olarak iyileştirmesini sağlar. Kazancı iki katlıdır: (i) günler-arası koordinasyon kazanılır, (ii) **global MIP-gap** raporlanabilir — yani greedy gün zincirinin veremediği *optimalite kanıtı* elde edilir.

	Değer
Girdi	`derived/` + `output/optimized_schedule.csv` (aşama-1 çözümü)
Çıktı	`output/` (iyileşme varsa yeniden yazılır) + `reports/fullweek_polish.md`
Çalıştırma	`python src/08_fullweek_polish.py [süre_s] [λ] [Wdev_dk]` — `run_all.py` bunu 1800 s ile çağırır
Bağımlılık	`03_model.py` (dinamik import), `pyomo`, `pandas`
Süre	verilen süre sınırı kadar (varsayılan 600 s, teslim koşusunda 1800 s)

4.1 Neden böyle bir adım gerekiyor?

`src/08_fullweek_polish.py` · satır 1–21 · yöntemin gerekçesi ve beş adımı

```
"""
TEKNOFEST Havayolu Optimizasyonu — Tam-Hafta Warm-Start Cilalaması (8. adım)

AMAÇ: Güne-göre ayrıştırma (03 `byday`) greedy zincirdir — G düzenliliği üzerinden
günler-arası koordinasyon kaybı yaşar ve global optimalite iddiası veremez. Bu betik,
byday çözümünü HAFTANIN TAMAMINI kapsayan tek MIP'e **MIP-start (incumbent)** olarak
verir ve HiGHS branch-and-bound'unun bu noktadan küresel olarak İYİLEŞTİRMESİNİ sağlar.

Yöntem (warm-start matheuristiği; literatürde "fix-and-optimize/warm-start" ailesi):
  1. Aşama-1 çözümü `output/optimized_schedule.csv`'den okunur (yoksa byday koşulur).
  2. Tam-hafta modeli kurulur: 03'ün TÜM kısıtları (A1, A2 + gece-aşan koruma, B1, C,
     D, E1, F) değişmeden; G düzenliliği ise bağımsız doğrulayıcıyla (05) BİREBİR aynı
     "ilk-gün çapalı ±Xdev" biçiminde kurulur (aşağıya bkz.).
  3. Byday çözümü tüm ikili değişkenlere yazılır; Pyomo 6.10 appsi-HiGHS arayüzü
     `config.warmstart=True` ile bunu `Highs.setSolution` MIP-start'ına çevirir.
  4. Süre sınırı içinde HiGHS incumbent'ı iyileştirir; ayrıca üst sınırdan (bound)
     **global MIP-gap** raporlanır — greedy zincirin veremediği optimalite kanıtı.
  5. SAF rekabetçilik iyileşirse `output/` dosyaları aynı formatta yeniden yazılır
     (05 doğrulayıcısı değişmeden geçer); her durumda `reports/fullweek_polish.md`
     yazılır.

```

Sorun şudur: `byday`, günleri **sırayla** çözer ve her günün kararı bir sonrakini kısıtlar (G bandı). Bu bir **greedy zincirdir** — 3. günde alınan bir karar 6. gün için daha iyi bir seçeneği kapatmış olabilir ve bunu geri alma imkânı yoktur. Ayrıca greedy bir yöntem "bulduğum çözüm optimumdan en fazla %X uzak" diyemez.

Warm-start matheuristiği nedir?

Literatürde "fix-and-optimize / warm-start" ailesine giren bir yöntemdir: hızlı bir sezgiselle **uygun** bir çözüm bulunur, sonra bu çözüm tam modelin branch-and-bound arayışına *incumbent* (eldeki en iyi) olarak verilir. Çözücü artık sıfırdan çözüm aramaz — elindeki iyi noktayı **iyileştirmeye** çalışır ve aynı anda üst sınırı (bound) daraltarak gap üretir.

`src/08_fullweek_polish.py` · satır 22–42 · G düzenliliğinin özel kurulumu ve Wdev deneyi

```
G DÜZENLİLİĞİ (neden "ilk-gün çapalı"): Doğrulayıcı (05) G'yi şöyle denetler: her
```

```
(uçuş,rol) grubunda İLK günün optimize yerleşimi bant merkezidir (c0); sonraki günler
|yerleşim − c0| ≤ Xdev sağlamalı, `g_band_exemptions.csv`'deki belgeli muafiyetler ve
penceresi banda yetişmeyen günler atlanır. byday'in zincirleme bandı da tam budur. Bu
betik aynı kısıtı DEĞİŞKEN çapayla kurar: θ'_gün − θ'_ilkgün ∈ [−Xdev, +Xdev] (çapa da
kayabilir → günler-arası eşgüdüm serbestliği). Warm-start noktasının sağlamadığı veya
penceresi yetişmeyen az sayıda bağ kısıtlanmaz ve doğrulayıcı formatında BELGELİ
MUAFİYET olarak `g_band_exemptions.csv`'ye eklenir (byday muafiyetleri korunur).

KURALLARA ETKİ YOKTUR: resmi amaç fonksiyonu ve tüm kısıt grupları birebir aynıdır;
yalnız çözüm YÖNTEMİ güçlenir. Çözüm yine 05 bağımsız doğrulayıcısından geçirilir.

Çalıştırma:  python src/08_fullweek_polish.py [süre_s] [λ] [Wdev_dk]
             (varsayılan: 600 s, λ = 03'teki DEFAULT_LAMBDA = 0.3, Wdev = 60)

Wdev_dk > 60 verilirse pencere genişletme DENEYİ yapılır: aşama-1 çözümü (±60)
geniş pencerede de fizibıldir → warm-start geçerli kalır; fizibıl bölge büyüdüğü
için gerçek optimum ancak yükselir (brief pencereye sınır koymaz, docs D3).
DİKKAT: geniş-Wdev çözümü BENİMSENİRSE 05 doğrulayıcısının WDEV sabiti ve docs
D3 senkron güncellenmelidir (aksi hâlde A1 pencere kontrolü FAIL verir).
"""
```

Kritik bir uyum ayrıntısı: **G kısıtı bu betikte farklı kurulur.** Nedeni, bağımsız doğrulayıcının (05) G'yi "ilk günün yerleşimi çapadır, sonraki günler ±Xdev" biçiminde denetlemesidir. 08 aynı kısıtı **birebir aynı biçimde** kurar — ama çapayı da değişken bırakarak (çapa da kayabilir) günler-arası eşgüdüme serbestlik tanır.

Wdev genişletme deneyi ve neden iptal edildi

Docstring, `Wdev_dk > 60` verilirse pencere genişletme denemesi yapılabileceğini söyler. Wdev = 90 denendiğinde sonuç %5.34'e çıktı — **ama 05 doğrulayıcısı gerçek bir A2 ihlali yakaladı**. Sonuç iptal edildi ve Wdev = 60'a dönüldü. Daha yüksek bir rakam uğruna doğrulanmamış bir çözüm teslim edilmedi; bu, projenin en net karakter göstergelerinden biridir.

4.2 03'ü modül olarak yüklemek

`src/08_fullweek_polish.py` · satır 44–69 · dinamik import ve kısayollar

```
import os
import sys
import datetime as dt
import importlib.util

import pandas as pd
import pyomo.environ as pyo

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
OUTPUT = os.path.join(ROOT, "output")
REPORTS = os.path.join(ROOT, "reports")

spec = importlib.util.spec_from_file_location("model03", os.path.join(HERE, "03_model.py"))
M3 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(M3)
CFG = M3.CFG

HHMM = lambda t: f"{(t % 1440) // 60:02d}:{(t % 1440) % 60:02d}"
ROL = {"dep": "kalkış", "arr": "varış"}
TOD = lambda v: v - (v // 1440) * 1440
```

`03_model.py` normal `import` ile yüklenemez — Python modül adları rakamla başlayamaz. Çözüm `importlib.util` ile **dosya yolundan** yüklemektir (62–64). Bundan sonra `M3.build_instance()`, `M3.CFG`, `M3.solve_model()` gibi her şey erişilebilir.

Bu neden değerli?

Amaç fonksiyonu, kısıtlar ve çözücü ayarları **kopyalanmaz, paylaşılır**. Yani 08, 03'ün modelini kelimesi kelimesine aynı kurar; yalnızca *çözüm yöntemi* değişir. "Kurallara dokunulmadı, yalnız yöntem güçlendi" iddiası böyle savunulabilir hâle

gelir.

Üç kısayol lambda'sı: `HHMM` (dakika → "08:30"), `ROL` ("dep"→"kalkış"), `TOD` (mutlak dakika → saat-içi dakika).

4.3 `load_phase1()` — aşama-1 çözümünü okumak

`src/08_fullweek_polish.py` · satır 72–101 · kaydırmalar, sunum listesi ve muafiyetler

```python
# ------------------------------------------------------------- aşama-1 çözümü
def load_phase1(lam):
    """byday çözümünü (uçuş-olay kaydırmaları + SUNULAN bağlantı listesi) ve G
    muafiyetlerini oku. Tarife dosyası yoksa byday'i çalıştırır.

    Sunum listesi kritiktir: amaç sunmayı ödüllendirse de byday, E1 yönsel dengesi
    için uygun bağlantıların bir kısmını BİLİNÇLİ sunmaz (docs §10-U5, ~265 aday).
    Warm-start'ta y'yi 'uygunluk'tan türetmek E1'i ihlal eden (reddedilecek) bir
    başlangıç üretir — y byday'in GERÇEK sunumundan alınmalıdır."""
    sched_p = os.path.join(OUTPUT, "optimized_schedule.csv")
    if not os.path.exists(sched_p):
        print(">> output/optimized_schedule.csv yok — aşama-1 (byday) koşuluyor …")
        M3.run_byday(lam=lam, gap=0.001, time_limit=150, write=True)
    df = pd.read_csv(sched_p, encoding="utf-8-sig", dtype={"eski_IST": str})
    shift = {(int(r.gun), int(r.ucus_no), r.rol, str(r.eski_IST)): int(r.degisim_dk)
             for r in df.itertuples()}
    sel = pd.read_csv(os.path.join(OUTPUT, "selected_connections.csv"),
                      encoding="utf-8-sig", dtype={"IST_varis": str, "IST_kalkis": str})
    offered = {(int(r.gun), r.pazar, int(r.gelen_ucus), int(r.giden_ucus),
                str(r.IST_varis), str(r.IST_kalkis)) for r in sel.itertuples()}
    # byday'in KAYITLI saf rekabetçilik toplamı — çıktı-yeniden-yazma bariyeri bunu
    # kullanır (warm nokta eşleşme kayıplarıyla düşebilir; kıyas gerçek toplamla yapılmalı)
    mres = pd.read_csv(os.path.join(OUTPUT, "market_results.csv"), encoding="utf-8-sig")
    byday_total = float(mres["rho_katki"].sum())
    exempt = set()
    exq = os.path.join(OUTPUT, "g_band_exemptions.csv")
    if os.path.exists(exq):
        for r in pd.read_csv(exq, encoding="utf-8-sig").itertuples():
            exempt.add((int(r.gun), int(r.ucus_no), r.rol))
    return shift, offered, exempt, byday_total
```

Dört şey okunur:

Okunan	Kaynak	Ne için?
`shift`	`optimized_schedule.csv`	(gün, uçuş, rol, eski saat) → değişim dakikası
`offered`	`selected_connections.csv`	byday'in gerçekten **sunduğu** bağlantı kümesi
`byday_total`	`market_results.csv`	byday'in kayıtlı saf rekabetçilik toplamı — yeniden yazma bariyerinin eşiği
`exempt`	`g_band_exemptions.csv`	byday'in belgeli G muafiyetleri

`offered` neden uygunluktan türetilemez? (docstring 77–80)

"Boşluk 60–300 arasındaysa sunuluyor demektir" varsayımı **yanlıştır**. byday, E1 yönsel dengesini sağlamak için uygun bağlantıların bir kısmını **bilinçli olarak sunmaz** (~265 aday). Warm-start'ta y'yi uygunluktan türetmek, E1'i ihlal eden bir başlangıç noktası üretir — HiGHS bunu reddeder ve warm-start hiç işe yaramaz. Bu yüzden y, byday'in gerçek sunum listesinden alınır.

Tarife dosyası yoksa fonksiyon `M3.run_byday(...)` çağırıp aşama-1'i kendisi koşturur — betik tek başına da çalışabilir.

4.4 `pin_merged_events()` — gün-görünümü kopyalarının sabitlenmesi

`src/08_fullweek_polish.py` · satır 104–142 · birleşen olayların byday saatine kilitlenmesi

```python
def pin_merged_events(inst, days, shift):
```

```
105      """Aynı fiziksel zaman damgası birden çok operasyon gününde kayıtlı olan olaylar
106      (gün-görünümü kopyaları) tam-hafta ekseninde TEK karara birleşir; byday ve bağımsız
107      doğrulayıcı (05) ise bunları gün-görünümü başına AYRI değerlendirir. Tutarlılık için:
108        • birleşik AYARLANABİLİR olay byday'in verdiği saate SABİTLENİR (cands=[t*]) —
109          warm nokta birebir byday olur ve çözücü olayı oynatamayacağından öbür
110          gün-görünümünün (modelde kurulmayan) A2/F bağları da riske girmez;
111        • olayın TÜM gün-görünümleri kaydedilir (inst.view_days) — tarife çıktısına her
112          görünüm için satır yazılır (doğrulayıcı iki günü de tutarlı görür; byday da
113          kopyalar için ayrı satır yazıyordu).
114      Bu veride 1 böyle olay vardır (uçuş 1304 varışı). Sabit birleşikler zaten durur."""
115      _, _, tk, _, _, _, _, _ = M3.load_data()
116      tk = tk[tk["day"].isin(days)]
117      delta = CFG["delta_min"]
118      absmin = lambda ts: (ts - inst.t0).total_seconds() / 60.0
119      sp = lambda mv: int(round(mv / delta) * delta)
120      evdays = {}
121      for r in tk.itertuples():
122          evdays.setdefault((int(r.FlNo1), "arr", sp(absmin(r.ist_arr))), set()).add(int(r.day))
123          evdays.setdefault((int(r.FlNo2), "dep", sp(absmin(r.ist_dep))), set()).add(int(r.day))
124      pinned = 0
125      view_days = {}                                        # eid -> [görünüm günleri]
126      for key, e in inst.events.items():
127          vds = evdays.get(key, {e["day"]})
128          if len(vds) <= 1:
129              continue
130          view_days[e["id"]] = sorted(vds)
131          if not e["adjustable"]:
132              continue
133          # byday kaydırması: önce olayın kayıtlı günü, yoksa diğer görünüm günleri
134          dv = None
135          for d in [e["day"]] + [d for d in sorted(vds) if d != e["day"]]:
136              dv = shift.get((d, e["flno"], ROL[e["role"]], HHMM(e["cur"])))
137              if dv is not None:
138                  break
139          e["cands"] = [e["cur"] + (dv or 0)]
140          pinned += 1
141      inst.view_days = view_days
142      return pinned
```

İnce ama gerçek bir tutarsızlık kaynağı: aynı fiziksel zaman damgası birden çok operasyon gününde kayıtlı olabilir. Tam-hafta ekseninde bu **tek bir olaya** birleşir; ama byday ve 05 doğrulayıcısı onu gün-görünümü başına **ayrı** değerlendirir. Bu uyumsuzluk iki şekilde çözülür:

1. Birleşik **ayarlanabilir** olay, byday'in verdiği saate **sabitlenir** (`cands = [t*]`, satır 139). Çözücü onu oynatamaz → öteki gün-görünümünün (modelde kurulmayan) A2/F bağları riske girmez.
2. Olayın **tüm gün görünümleri** `inst.view_days`'e kaydedilir (141). Tarife çıktısına her görünüm için ayrı satır yazılır → doğrulayıcı iki günü de tutarlı görür.

Yorum bu veride tam **1** böyle olay olduğunu söylüyor (uçuş 1304 varışı). Tek bir satır için yazılmış 39 satırlık kod — ama o tek satır doğrulamayı FAIL'e düşürebilirdi.

4.5 `warm_times()` — başlangıç noktasının zamanları

`src/08_fullweek_polish.py` · satır 145–162 · her olayın aşama-1 saati

```
145  def warm_times(inst, shift):
146      """Her olayın aşama-1 (byday) zamanı: t*[eid] (tam eksende dk). Ayarlanabilir olay
147      tarife dosyasında bulunamazsa sayılır (byday çıktısı bayat → koşu durdurulur)."""
148      t, missing = {}, []
149      for e in inst.ev_by_id.values():
150          if not e["adjustable"]:
151              t[e["id"]] = e["cur"]
152              continue
153          dv = shift.get((e["day"], e["flno"], ROL[e["role"]], HHMM(e["cur"])))
154          if dv is None:
155              missing.append((e["day"], e["flno"], e["role"]))
156              t[e["id"]] = e["cur"]
157          else:
158              t[e["id"]] = e["cur"] + dv
159              if t[e["id"]] not in set(e["cands"]):
160                  raise ValueError(f"byday kaydırması pencere dışı: olay {e} Δ={dv} "
161                                   "(Wdev/Δ ayarları 03 ile uyumsuz?)")
162      return t, missing
```

Her olay için `t*[eid]` hesaplanır: sabitler mevcut saatlerinde, ayarlanabilirler `cur + değişim`. İki emniyet kontrolü var:

- Bir ayarlanabilir olay tarife dosyasında **bulunamazsa** `missing` listesine eklenir. `main()` bunu görürse "byday çıktısı bayat" diyip çıkar — eski bir çözümle yeni bir model karıştırılmaz.
- Kaydırma **pencere dışındaysa** `ValueError` fırlatılır. Bu, 03 ile 08 arasında Wdev/Δ uyumsuzluğunun sessizce geçmesini engeller.

4.6 `add_star_G()` — doğrulayıcı-birebir G kısıtı

`src/08_fullweek_polish.py` · satır 165–206 · ilk-gün çapalı ±Xdev bandı

```
165 # ------------------------------------------------------------- G (ilk-gün çapalı)
166 def add_star_G(inst, m, tstar, byday_exempt):
167     """G düzenliliğini doğrulayıcı-birebir kur: her (uçuş,rol) grubunda ilk günün
168     yerleşimi (değişken) çapadır; sonraki günler θ'_g − θ'_çapa ∈ [−Xdev, +Xdev].
169     Kısıtlanmayan her (gün,uçuş,rol) belgeli muafiyet olarak döndürülür — böylece
170     doğrulayıcıda hiçbir bağ 'bant dışı' kalamaz (kısıtlı → geçer; kısıtsız → muaf)."""
171     Xd = CFG["xdev_min"]
172     W = CFG["wdev_min"]
173     n_c, new_ex = 0, []
174     for (flno, role), eids in sorted(inst.ev_by_flno_role.items()):
175         adj = sorted((inst.ev_by_id[e] for e in eids if inst.ev_by_id[e]["adjustable"]),
176                      key=lambda e: (e["day"], e["cur"]))
177         if len({e["day"] for e in adj}) < 2:
178             continue
179         day_ct = {}
180         for e in adj:
181             day_ct[e["day"]] = day_ct.get(e["day"], 0) + 1
182         anchor = adj[0]
183         off_a = (anchor["cur"] // 1440) * 1440
184         ws_a = TOD(tstar[anchor["id"]])
185         th_a = None                                      # tembel: gerekirse kur
186         for e in adj[1:]:
187             key3 = (e["day"], flno, ROL[role])
188             if key3 in byday_exempt:
189                 continue                                 # zaten belgeli muafiyet
190             # kısıt ancak şu üç koşulla kurulur (aksi → belgeli muafiyet):
191             #  (i) çapa/olay günü tekil (3 anomali-olay grubu determinizm için muaf),
192             #  (ii) pencereler ortak banda yetişebilir (|Δcur| ≤ 2·Wdev+Xdev),
193             #  (iii) warm-start noktası kısıtı sağlar (byday bant-dışı bıraktıysa muaf).
194             ok = (day_ct[e["day"]] == 1 and day_ct[anchor["day"]] == 1
195                   and abs(TOD(e["cur"]) - TOD(anchor["cur"])) <= 2 * W + Xd
196                   and abs(TOD(tstar[e["id"]]) - ws_a) <= Xd)
197             if not ok:
198                 new_ex.append(dict(gun=e["day"], ucus_no=flno, rol=ROL[role]))
199                 continue
200             if th_a is None:
201                 th_a = M3.theta_expr(m, inst, anchor["id"]) - off_a
202             th_e = M3.theta_expr(m, inst, e["id"]) - (e["cur"] // 1440) * 1440
203             m.cons.add(th_e - th_a <= Xd)
204             m.cons.add(th_a - th_e <= Xd)
205             n_c += 1
206     return n_c, new_ex
```

Her `(uçuş, rol)` grubunda **ilk günün olayı çapadır** ve sonraki her gün için iki kısıt yazılır: `θ'_gün − θ'_çapa ∈ [−Xdev, +Xdev]`. Çapanın kendisi de bir değişken olduğu için tüm grup birlikte kayabilir — byday'in sabit bandına göre **daha fazla serbestlik**.

Kısıt yalnız üç koşul birden sağlanırsa kurulur (194–196):

1. Çapa ve olay günü **tekil** olmalı (anomali olay grupları determinizm için muaf tutulur).
2. Pencereler ortak banda yetişebilmeli: `|Δcur| ≤ 2·Wdev + Xdev`.
3. **Warm-start noktası kısıtı sağlamalı**: byday zaten bandın dışında bıraktıysa, kısıt eklemek başlangıç noktasını geçersiz kılardı.

> **"Kısıtlı → geçer, kısıtsız → muaf" tasarımı**
>
> Kurulmayan her bağ, doğrulayıcı formatında **belgeli muafiyet** olarak `new_ex` listesine yazılır ve

`g_band_exemptions.csv`'ye eklenir. Böylece 05'te hiçbir bağ "bant dışı" olarak FAIL veremez: ya kısıtlanmıştır (geçer) ya da muaf olarak ilan edilmiştir. Sessiz ihlal imkânsız hâle getirilmiştir.

4.7 `set_start_values()` — MIP-start'ın yazılması

src/08_fullweek_polish.py · satır 209–235 · x ve y değerleri

```
209 # ------------------------------------------------------------- warm-start yazımı
210 def set_start_values(inst, m, t, offered):
211     """Aşama-1 çözümünü modelin TÜM ikili değişkenlerine tutarlı biçimde yaz
212     (03.set_warmstart'ın t[eid] zamanlarıyla genellenmiş hali; E2/B2 tam-ölçekte
213     kapalı olduğundan o bloklar yoktur). appsi `config.warmstart=True` bu değerleri
214     HiGHS `setSolution` MIP-start'ına çevirir.
215
216     y, UYGUNLUKTAN DEĞİL byday'in gerçek sunum listesinden (`offered`) alınır:
217     byday E1 dengesi için bazı uygun bağlantıları bilinçli sunmaz; hepsini y=1
218     yapmak E1'i ihlal eden (HiGHS'in reddedeceği) bir başlangıç üretir."""
219     md_id, L, U = m._md_id, inst.L, inst.U
220     sv = lambda var, val: var.set_value(val, skip_validation=True)
221     for e in inst.ev_by_id.values():                    # x = byday kovası
222         if e["adjustable"]:
223             for v in e["cands"]:
224                 sv(m.x[e["id"], v], 1 if v == t[e["id"]] else 0)
225     yv = {}
226     n_drop = 0
227     for pi, c in enumerate(inst.conns):                 # y = byday'in sunduğu ∩ hâlâ uygun
228         g = t[c["dep_eid"]] - t[c["arr_eid"]]
229         ea, ed = inst.ev_by_id[c["arr_eid"]], inst.ev_by_id[c["dep_eid"]]
230         off = (c["day"], c["market"], ea["flno"], ed["flno"],
231                HHMM(t[c["arr_eid"]]), HHMM(t[c["dep_eid"]])) in offered
232         if off and not (L <= g <= U):
233             off = False                                 # (dondurulan olay) artık uygun değil
234             n_drop += 1
235         yv[pi] = 1 if off else 0
```

`03.set_warmstart`'ın genelleştirilmiş hâli. `x` byday kovasına, `y` ise byday'in **gerçek sunum listesine** göre atanır. Ek bir kontrol var (232–234): `pin_merged_events` ile dondurulmuş bir olay yüzünden bir bağlantı artık pencerede değilse, o bağlantı sunumdan düşürülür ve sayılır.

src/08_fullweek_polish.py · satır 236–264 · E1 onarımı

```
236     # E1 onarımı: dondurma kaynaklı düşürmeler bir yönün sayısını azaltıp warm
237     # noktada E1'i bozabilir; sağlanana dek büyük yönden sunum geri çekilir
238     # (deterministik: en yüksek indeksli). byday noktası E1'i sağladığından bu
239     # onarım yalnız düşürme yaşanan yön-çiftlerinde tetiklenebilir.
240     n_rep = 0
241     if CFG["E1_balance"]:
242         a = CFG["alpha"]
243         seen = set()
244         for k in inst.md:
245             (mk, day) = k
246             o, dd = mk.split("-")
247             rk = (f"{dd}-{o}", day)
248             if rk not in inst.md or (rk, k) in seen:
249                 continue
250             seen.add((k, rk))
251             c1, c2 = inst.md[k]["conns"], inst.md[rk]["conns"]
252             n1 = sum(yv[p] for p in c1)
253             n2 = sum(yv[p] for p in c2)
254             while abs(n1 - n2) > a * (n1 + n2) + 1e-9:
255                 side = c1 if n1 > n2 else c2
256                 yv[max(p for p in side if yv[p] == 1)] = 0
257                 n_rep += 1
258                 n1 = sum(yv[p] for p in c1)
259                 n2 = sum(yv[p] for p in c2)
260     for pi in range(len(inst.conns)):
261         sv(m.y[pi], yv[pi])
262     if n_drop or n_rep:
263         print(f"   [not] warm-start: dondurma nedeniyle düşürülen bağlantı {n_drop}, "
264               f"E1 onarımıyla geri çekilen {n_rep}")
```

Düşürmeler bir yönün bağlantı sayısını azaltıp warm noktada E1 dengesini bozabilir. Bu döngü, denge sağlanana kadar **fazla olan yönden** sunum geri çeker. Seçim deterministiktir (en yüksek indeksli) — aynı girdi her makinede aynı sonucu verir.

Neden bu kadar uğraşılıyor?

HiGHS, MIP-start olarak verilen noktayı **uygunluk açısından denetler**. Tek bir kısıt ihlali varsa noktayı tümüyle reddeder ve çözücü sıfırdan başlar — yani 08'in tüm varlık sebebi ortadan kalkar. Bu yüzden başlangıç noktasının **kusursuz uygun** olması gerekir.

`src/08_fullweek_polish.py` · satır 265–291 · z, w, β, γ değerlerinin tutarlı yazılması

```
    for k, gmd in inst.md.items():                          # z (monoton doldurma)
        i, Mn = md_id[k], len(gmd["conns"])
        n = sum(yv[pi] for pi in gmd["conns"])
        for j in range(1, Mn + 1):
            sv(m.z[i, j], 1 if j <= n else 0)
    for k, gmd in inst.md.items():                          # w, β, γ
        i = md_id[k]
        beaten = {}
        for ca, Tc in gmd["comps"].items():
            bt = 0
            for pi in gmd["conns"]:
                if yv[pi] != 1:
                    continue
                if (pi, ca) in m._beat_eq_y:
                    bt = 1
                elif (pi, ca) in m._w:
                    beats = (t[inst.conns[pi]["dep_eid"]] - t[inst.conns[pi]["arr_eid"]]) \
                        <= (Tc - inst.conns[pi]["legsum"])
                    sv(m._w[(pi, ca)], 1 if beats else 0)
                    bt = bt or int(beats)
            if (i, ca) in m._beta:
                sv(m._beta[(i, ca)], bt)
            beaten[ca] = bt
        if CFG["ranking"] and gmd["N"] >= 1 and gmd["b_od"] is not None:
            nb = sum(beaten.values())
            for bn in range(0, gmd["N"] + 1):
                sv(m.w[("gam", i, bn)], 1 if bn == nb else 0)
```

Kalan tüm ikili değişkenler y'den türetilir: `z` monoton doldurma, `w`/β rakip yenme göstergeleri, `γ` yenilen sayısı seçicisi. Sıra ve tutarlılık, 03'teki `set_warmstart` ile birebir aynıdır (E2/B2 blokları yoktur çünkü tam ölçekte kapalıdırlar).

4.8 Raporlama yardımcıları

`src/08_fullweek_polish.py` · satır 294–319 · totals_by_day — gün kırılımı

```
# ------------------------------------------------------------- raporlama yardımcıları
def totals_by_day(inst, m):
    """Gün kırılımı: optimize (çözümden) ve baseline (mevcut tarifeden) SAF rekabetçilik."""
    rkW = M3.rank_weight_table(inst.rank)
    md_id = m._md_id
    opt_d, base_d, nmd_d, nconn_d = {}, {}, {}, {}
    for k, g in inst.md.items():
        d, i = k[1], md_id[k]
        nmd_d[d] = nmd_d.get(d, 0) + 1
        nconn_d[d] = nconn_d.get(d, 0) + len(g["conns"])
        n = sum(1 for pi in g["conns"] if pyo.value(m.y[pi]) > 0.5)
        conn = sum(0.5 ** (j - 1) for j in range(1, n + 1))
        beaten = sum(1 for ca in g["comps"]
                     if (i, ca) in m._beta and pyo.value(m._beta[(i, ca)]) > 0.5)
        rr = M3.rank_reward(rkW, g["N"], g["b_od"], beaten) if CFG["ranking"] else 0.0
        opt_d[d] = opt_d.get(d, 0.0) + g["rho"] * (conn + rr)
        offered0 = [pi for pi in g["conns"]
                    if inst.L <= inst.conns[pi]["gap_cur"] <= inst.U]
        c0 = sum(0.5 ** (j - 1) for j in range(1, len(offered0) + 1))
        b0 = 0
        if CFG["ranking"] and offered0 and g["comps"]:
            jb = min(inst.conns[pi]["legsum"] + inst.conns[pi]["gap_cur"] for pi in offered0)
            b0 = sum(1 for tc in g["comps"].values() if jb <= tc)
        rr0 = M3.rank_reward(rkW, g["N"], g["b_od"], b0) if CFG["ranking"] else 0.0
        base_d[d] = base_d.get(d, 0.0) + g["rho"] * (c0 + rr0)
    return opt_d, base_d, nmd_d, nconn_d
```

Tam-hafta çözümünde "gün" artık bir çözüm birimi değil, sadece bir **etikettir** (`k[1]`). Bu fonksiyon hem optimize hem baseline değerini gün gün toplar, böylece `results_summary.md`'nin gün tablosu byday ile aynı formatta kalır.

src/08_fullweek_polish.py · satır 322–353 · collect_market_outputs_week

```python
def collect_market_outputs_week(inst, m, new_time):
    """03.collect_market_outputs'un çok-günlü örnek karşılığı (gun = pazar-gün anahtarından)."""
    rkW = M3.rank_weight_table(inst.rank)
    md_id = m._md_id
    market_rows, conn_rows = [], []
    for k, g in inst.md.items():
        day, i = k[1], md_id[k]
        offered = [pi for pi in g["conns"] if pyo.value(m.y[pi]) > 0.5]
        n = len(offered)
        conn_rew = sum(0.5 ** (j - 1) for j in range(1, n + 1))
        beaten_cas = [ca for ca in g["comps"]
                      if (i, ca) in m._beta and pyo.value(m._beta[(i, ca)]) > 0.5]
        beaten = len(beaten_cas)
        rank_rew = M3.rank_reward(rkW, g["N"], g["b_od"], beaten) if CFG["ranking"] else 0.0
        market_rows.append(dict(
            gun=day, pazar=k[0], rho=round(g["rho"]), N_rakip=g["N"],
            sunulan_baglanti=n, aday_baglanti=len(g["conns"]),
            yenilen_rakip=beaten, baslangic_sira_bod=g["b_od"],
            son_sira=(g["N"] - beaten) if g["N"] >= 1 else None,
            aktarma_odul=round(conn_rew, 4), siralama_odul=round(rank_rew, 4),
            rho_katki=round(g["rho"] * (conn_rew + rank_rew), 2)))
        for pi in offered:
            c = inst.conns[pi]
            ea, ed = inst.ev_by_id[c["arr_eid"]], inst.ev_by_id[c["dep_eid"]]
            gv = pyo.value(m._gap[pi])
            J = c["legsum"] + gv
            beats = [ca for ca, Tc in g["comps"].items() if J <= Tc]
            conn_rows.append(dict(
                gun=day, pazar=k[0], gelen_ucus=ea["flno"], giden_ucus=ed["flno"],
                IST_varis=HHMM(new_time[c["arr_eid"]]), IST_kalkis=HHMM(new_time[c["dep_eid"]]),
                bosluk_dk=round(gv), J_dk=round(J), yendigi_rakip=len(beats)))
    return market_rows, conn_rows
```

`03.collect_market_outputs`'un çok-günlü karşılığı. Tek fark: gün bilgisi parametreden değil, pazar-gün anahtarından (`k[1]`) alınır. Üretilen satır formatı **birebir aynıdır** — bu yüzden 05 doğrulayıcısı hangi aşamanın ürettiğini bilmeden çalışabilir.

src/08_fullweek_polish.py · satır 356–394 · a2x_check — gece-aşan rotasyon doğrulaması

```python
def a2x_check(inst, new_time, days):
    """A2 fiziksel rotasyon doğrulaması (gece-aşan dahil, gerçek zaman ekseni) —
    03.run_byday'deki kontrolün tam-hafta çözümü üzerindeki karşılığı."""
    final_dt, cur_dt = {}, {}
    for e in inst.ev_by_id.values():
        final_dt[(e["flno"], e["day"], e["role"])] = \
            inst.t0 + pd.Timedelta(minutes=new_time[e["id"]])
        cur_dt[(e["flno"], e["day"], e["role"])] = inst.t0 + pd.Timedelta(minutes=e["cur"])
    a2x_chk = a2x_viol = a2x_base = 0
    if inst.blocks is None or not inst.rot_pairs:
        return a2x_chk, a2x_viol, a2x_base
    tau = CFG["tau_min"]
    for out_f, in_f in inst.rot_pairs:
        for d in days:
            t_ob = inst.blocks.get((out_f, d, "dep"))
            dep_c = cur_dt.get((out_f, d, "dep"))
            dep_o = final_dt.get((out_f, d, "dep"))
            if dep_c is None or dep_o is None or t_ob is None:
                continue
            pick = None
            for d2 in (d, d + 1):
                a_c = cur_dt.get((in_f, d2, "arr"))
                tb = inst.blocks.get((in_f, d2, "arr"))
                if a_c is not None and tb is not None and a_c > dep_c:
                    pick = (d2, a_c, tb)
                    break
            if pick is None:
                continue
            d2, a_c, tb = pick
            a_o = final_dt.get((in_f, d2, "arr"))
            if a_o is None:
                continue
            a2x_chk += 1
            need = pd.Timedelta(minutes=t_ob + tau + tb)
```

```
            if a_c < dep_c + need:
                a2x_base += 1                              # mevcutta da yok → artefakt
            elif a_o < dep_o + need:
                a2x_viol += 1                              # YENİ ihlal (olmamalı)
    return a2x_chk, a2x_viol, a2x_base
```

`run_byday` içindeki A2 kontrolünün tam-hafta çözümü üzerindeki karşılığı. Aynı artefakt/YENİ-ihlal ayrımını yapar ve üç sayaç döndürür.

4.9 `main()` — akışın tamamı

src/08_fullweek_polish.py · satır 398–418 · sıralama tuzağı ve CFG kurulumu

```
def main(time_limit=600, lam=None, wdev=None):
    lam = M3.DEFAULT_LAMBDA if lam is None else lam
    # ÖNCE aşama-1: load_phase1 gerekirse run_byday çalıştırır ve run_byday CFG'yi
    # (days=[son gün], time_limit, mip_gap) mutasyona uğratır — CFG kurulumunu
    # bu yüzden aşama-1'den SONRA yapıyoruz.
    shift, offered, byday_exempt, byday_total = load_phase1(lam)
    CFG.update(M3.PRESETS["full"])
    CFG["days"] = M3.all_operation_days()
    CFG["shift_penalty"] = lam
    if wdev is not None:
        CFG["wdev_min"] = int(wdev)    # pencere genişletme deneyi (docstring'e bkz.)
    CFG["time_limit"], CFG["mip_gap"] = time_limit, 0.001
    CFG["G_regularity"] = False        # G'yi bu betik doğrulayıcı-birebir (ilk-gün çapalı) kurar
    CFG["warmstart"] = True            # appsi → HiGHS setSolution MIP-start
    # incumbent çevresi sezgisellere (RINS vb.) daha çok bütçe: warm-start'lı büyük
    # MIP'te iyileştirmeler çoğunlukla sezgisellerden gelir (varsayılan 0.05)
    CFG["highs_extra_options"] = {"mip_heuristic_effort": 0.3}
    days = CFG["days"]

    print(f">> TAM-HAFTA CİLALAMA  (λ={lam}, Wdev=±{CFG['wdev_min']}dk, "
          f"gap≤{CFG['mip_gap']:.2%}, ≤{time_limit}s, günler {days})")
```

400–402'deki yorum bir hata ayıklama hikâyesidir

`load_phase1` gerektiğinde `run_byday` çağırır — ve `run_byday`, CFG'yi **mutasyona uğratır** (`days` son güne düşer, `time_limit` ve `mip_gap` değişir). Bu yüzden CFG kurulumu aşama-1'den **sonra** yapılmalıdır. Sırayı ters çevirmek, sessizce yalnız son günü çözen bir betik üretirdi.

CFG'de üç kritik ayar:

- `G_regularity = False` — G'yi bu betik kendi (doğrulayıcı-birebir) biçiminde kuracak.
- `warmstart = True` — appsi katmanı bunu HiGHS `setSolution` MIP-start'ına çevirir.
- `highs_extra_options = {"mip_heuristic_effort": 0.3}` — varsayılan 0.05'ten yükseltilir. Warm-start'lı büyük MIP'te iyileştirmeler çoğunlukla incumbent çevresinde çalışan sezgisellerden (RINS vb.) gelir; onlara daha çok bütçe verilir.

src/08_fullweek_polish.py · satır 420–435 · örnek kurulumu ve bayatlık kontrolü

```
    print(">> tam-hafta örneği kuruluyor …")
    inst = M3.build_instance()
    caps, floors = M3.compute_overnight_guards(days)     # (A2×) gece-aşan koruma (statik)
    inst.rotcap = {k: v for d in caps for k, v in caps[d].items()}
    inst.rotlock = {k: v for d in floors for k, v in floors[d].items()}
    n_pin = pin_merged_events(inst, days, shift)         # gün-görünümü kopyası → byday'e sabitle
    n_adj = sum(1 for e in inst.ev_by_id.values() if e["adjustable"])
    print(f"   pazar-gün: {len(inst.md)} | bağlantı(π): {len(inst.conns)} | "
          f"olay: {len(inst.events)} (ayarlanabilir {n_adj}, "
          f"gün-görünümü kopyası byday'e sabitlenen {n_pin})")

    tstar, missing = warm_times(inst, shift)
    if missing:
        print(f"   [!] {len(missing)} ayarlanabilir olay tarife dosyasında yok — "
              "byday çıktısı bayat görünüyor; önce `python src/03_model.py byday` koşun.")
        return 1
```

Tam-hafta örneği kurulur, gece-aşan koruma sınırları **tüm günler için** düzleştirilerek eklenir (423–424), gün-görünümü kopyaları sabitlenir. Ardından `warm_times` çağrılır; eksik olay varsa betik **1 ile çıkar** ve kullanıcıya byday'i yeniden koşmasını söyler.

`src/08_fullweek_polish.py` · satır 437–455 · model, G kısıtları ve başlangıç noktasının ölçülmesi

```
437     print(">> model kuruluyor …")
438     m = M3.build_model(inst)
439     n_star, new_ex = add_star_G(inst, m, tstar, byday_exempt)
440     all_exempt_rows = ([dict(gun=g, ucus_no=f, rol=r) for (g, f, r) in sorted(byday_exempt)]
441                        + new_ex)
442     nbin = sum(1 for v in m.component_data_objects(pyo.Var) if v.is_binary())
443     print(f"   ikili değişken: {nbin} | A2 bağ: {m._n_a2} "
444           f"(evren-ankoru {getattr(m, '_n_a2_anchor', 0)}) | F bağ: {m._n_f} | "
445           f"G ilk-gün-çapalı bağ: {n_star} (byday muafiyeti {len(byday_exempt)}, "
446           f"yeni muafiyet {len(new_ex)})")
447
448     set_start_values(inst, m, tstar, offered)
449     warm_pen = pyo.value(m.OBJ)
450     warm_pure = M3.solution_total(inst, m)
451     base = M3.current_baseline(inst)
452     print(f"   MIP-start: SAF={warm_pure:,.1f}  cezalı={warm_pen:,.1f}  "
453           f"(baseline {base:,.1f}, Δ={warm_pure - base:+,.1f})")
454     print(f"   byday kayıtlı toplam: {byday_total:,.1f}  "
455           f"(warm − byday = {warm_pure - byday_total:+,.1f} — 0'a yakın olmalı)")
```

Konsola basılan iki satır tanı açısından çok değerlidir:

- `MIP-start: SAF=… cezalı=… (baseline …, Δ=…)` — başlangıç noktasının kalitesi.
- `warm − byday = …` — **0'a yakın olmalıdır**. Değilse, warm noktanın kurulumunda bir eşleşme kaybı var demektir ve bu hemen görülür.

`src/08_fullweek_polish.py` · satır 457–474 · çözüm ve iyileşme ölçümü

```
457     print(f">> çözülüyor (appsi_highs + MIP-start, ≤{time_limit}s) …")
458     res = M3.solve_model(m)
459     print(f"   durum: {res['tc']}" + (f"  | global MIP-gap (cezalı amaç): {res['gap']:.3%}"
460           if res.get("gap") is not None else "") + f"  | süre: {res['t']:.1f}s")
461     if not res["ok"]:
462         # byday çıktısı geçerli kalır → uçtan-uca koşuyu (run_all) durdurma, uyar ve geç
463         print("   [!] Çözüm yüklenemedi — MIP-start reddedilmiş olabilir; byday çıktısı korunuyor.")
464         return 0
465
466     pol_pen = pyo.value(m.OBJ)
467     pol_pure = M3.solution_total(inst, m)
468     sh = M3.shift_value(m)
469     print(f"   CİLALANMIŞ: SAF={pol_pure:,.1f}  cezalı={pol_pen:,.1f}  "
470           f"kaydırma={sh:,.0f} uçuş-dk")
471     d_pure = pol_pure - warm_pure
472     print(f"   İYİLEŞME (aşama-1'e göre, SAF): {d_pure:+,.1f}  "
473           f"(baseline'a göre {pol_pure - base:+,.1f} = {100*(pol_pure-base)/base:+.2f}%)")
474
```

461–464: Çözüm yüklenemezse betik **0 ile çıkar**, 1 ile değil. Neden? Çünkü byday çıktısı hâlâ geçerlidir — cilalama isteğe bağlı bir iyileştirmedir ve başarısızlığı uçtan uca koşuyu durdurmamalıdır.

`src/08_fullweek_polish.py` · satır 475–484 · yeniden yazma bariyeri ve A2 kontrolü

```
475     # BARİYER: yeniden yazma kararı warm noktaya göre DEĞİL, byday'in kayıtlı gerçek
476     # toplamına göre verilir (warm nokta eşleşme kayıplarıyla düşük olabilir; kötü
477     # bir çözümle iyi byday çıktısını ezmek yasak).
478     improved = pol_pure > byday_total + 0.5
479     new_time = {e["id"]: (int(round(sum(v * pyo.value(m.x[e["id"], v]) for v in e["cands"])))
480                           if e["adjustable"] else e["cur"]) for e in inst.ev_by_id.values()}
481     a2x_chk, a2x_viol, a2x_base = a2x_check(inst, new_time, days)
482     print(f"   A2 gece-aşan doğrulama: {a2x_chk} tur, YENİ ihlal: {a2x_viol} "
483           f"(artefakt taban: {a2x_base})")
484
```

İki koşullu bariyer

`output/` dosyaları **yalnız** şu iki koşul birlikte sağlanırsa yeniden yazılır:

(1) `pol_pure > byday_total + 0.5` — cilalanmış çözüm byday'in **kayıtlı gerçek toplamını** anlamlı biçimde geçmeli. Karşılaştırma warm noktaya göre değil byday'in kaydına göre yapılır; warm nokta eşleşme kayıplarıyla düşük olabilir ve kötü bir çözümle iyi bir çıktıyı ezmek yasaktır.

(2) `a2x_viol == 0` — hiçbir yeni fiziksel rotasyon ihlali doğmamalı.

`src/08_fullweek_polish.py` · satır 485–522 · çıktıların yeniden yazılması

```
    if improved and a2x_viol == 0:
        print(">> iyileşme var — output/ dosyaları tam-hafta çözümüyle yeniden yazılıyor …")
        sched_rows = []
        moved = tot_shift = 0
        view_days = getattr(inst, "view_days", {})
        for e in sorted(inst.ev_by_id.values(), key=lambda e: (e["day"], e["flno"], e["role"])):
            if not e["adjustable"]:
                continue
            delta = new_time[e["id"]] - e["cur"]
            # birleşik olayların HER gün-görünümü için satır yaz (doğrulayıcı her iki
            # günü de aynı kaydırmayla görür; byday kopya başına satır yazıyordu)
            for vd in view_days.get(e["id"], [e["day"]]):
                moved += int(delta != 0)
                tot_shift += abs(delta)
                sched_rows.append(dict(gun=vd, ucus_no=e["flno"], rol=ROL[e["role"]],
                                       eski_IST=HHMM(e["cur"]), yeni_IST=HHMM(new_time[e["id"]]),
                                       degisim_dk=delta, G_gevsetildi=0))
        market_rows, conn_rows = collect_market_outputs_week(inst, m, new_time)
        opt_d, base_d, nmd_d, nconn_d = totals_by_day(inst, m)
        shift_d, moved_d = {}, {}
        for r in sched_rows:
            shift_d[r["gun"]] = shift_d.get(r["gun"], 0) + abs(r["degisim_dk"])
            moved_d[r["gun"]] = moved_d.get(r["gun"], 0) + int(r["degisim_dk"] != 0)
        gp = f"{res['gap']:.2%}" if res.get("gap") is not None else "?"
        day_stats = [dict(gun=d, pazar_gun=nmd_d.get(d, 0), baglanti=nconn_d.get(d, 0),
                          OPT=round(opt_d.get(d, 0)), baseline=round(base_d.get(d, 0)),
                          delta=round(opt_d.get(d, 0) - base_d.get(d, 0)), gap=gp,
                          ucus_kaydirilan=moved_d.get(d, 0), kaydirma_dk=shift_d.get(d, 0),
                          G_gevsetildi=0) for d in days]
        stats = dict(lam=lam, opt=pol_pure, base=base,
                     dpct=100 * (pol_pure - base) / base if base else 0.0,
                     shift=tot_shift, moved=moved, nflights=len(sched_rows),
                     movedpct=100 * moved / max(1, len(sched_rows)),
                     ok_days=len(days), n_days=len(days), relaxed_days=0,
                     a2x_chk=a2x_chk, a2x_viol=a2x_viol, a2x_base=a2x_base,
                     band_exempt=len(all_exempt_rows))
        M3.write_outputs(sched_rows, day_stats, stats, market_rows, conn_rows,
                         all_exempt_rows)
```

İyileşme varsa tüm çıktılar aynı formatta yeniden üretilir. **496:** birleşik olayların her gün görünümü için ayrı satır yazılır — doğrulayıcı iki günü de aynı kaydırmayla görsün diye. Yazma işi `M3.write_outputs` ile yapılır, yani format kaymaz.

`src/08_fullweek_polish.py` · satır 523–547 · sonuç özetinin güncellenmesi

```
        # başlık iki aşamalı hattı yansıtsın (03'ün yazdığı özet artık cilalanmış sonucu içeriyor)
        summ_path = os.path.join(OUTPUT, "results_summary.md")
        txt = open(summ_path, encoding="utf-8").read()
        txt = txt.replace("`python src/03_model.py byday`",
                          f"`python src/03_model.py byday` → "
                          f"`python src/08_fullweek_polish.py {time_limit} {lam}`")
        txt = txt.replace("## Tam veri sonucu (güne göre ayrıştırma)",
                          "## Tam veri sonucu (güne göre ayrıştırma + tam-hafta cilalaması)")
        with open(summ_path, "w", encoding="utf-8") as f:
            f.write(txt)
        with open(os.path.join(OUTPUT, "results_summary.md"), "a", encoding="utf-8") as f:
            f.write(
                "\n*Gün tablosundaki `gap` sütunu tam-hafta modelinin GLOBAL (cezalı) MIP-gap'idir "
                "(aşama-1'in gün-içi gap'i ≤ %0.1 idi).*\n"
                "\n## Tam-hafta warm-start cilalaması (08)\n\n"
                "Bu sonuç, güne-göre ayrıştırma (aşama-1) çözümünün HAFTANIN TAMAMINI\n"
                "kapsayan tek MIP'e MIP-start olarak verilip HiGHS ile iyileştirilmesiyle\n"
                f"üretildi (`python src/08_fullweek_polish.py`, ≤{time_limit}s):\n\n"
                f"- Aşama-1 (byday) SAF rekabetçilik : {byday_total:,.1f}\n"
                f"- Tam-hafta cilalanmış             : {pol_pure:,.1f}  "
                f"({pol_pure - byday_total:+,.1f})\n"
                f"- Global MIP-gap (cezalı amaç)     : {gp}\n"
                "- G düzenliliği doğrulayıcı-birebir 'ilk-gün çapalı ±Xdev' kısıtlarıyla\n"
                f"  tam modelde kuruldu ({n_star} bağ; belgeli muafiyet: "
                f"{len(all_exempt_rows)}, bkz. `output/g_band_exemptions.csv`).\n")
```

`results_summary.md` iki aşamalı hattı yansıtacak şekilde düzeltilir: komut satırı `03 byday → 08` olarak güncellenir, başlık "+ tam-hafta cilalaması" olur ve sonuna aşama-1 → aşama-2 karşılaştırması ile **global MIP-gap** eklenir.

`src/08_fullweek_polish.py` · satır 548–552 · iyileşme yoksa ne olur?

```
    else:
        why = ("A2 gece-aşan yeni ihlal" if a2x_viol else
               "anlamlı iyileşme yok" if not improved else "?")
        print(f">> output/ DEĞİŞTİRİLMEDİ ({why}) — byday çıktısı geçerli kalır.")

```

Bariyer geçilmezse `output/` dosyalarına **dokunulmaz** ve nedeni açıkça yazılır: "A2 gece-aşan yeni ihlal" veya "anlamlı iyileşme yok". byday çıktısı geçerli kalır, koşu devam eder. Yani bu adım **asla zarar veremez** — en kötü ihtimalle hiçbir şey yapmaz.

`src/08_fullweek_polish.py` · satır 553–589 · koşu raporu

```
    os.makedirs(REPORTS, exist_ok=True)
    gp = f"{res['gap']:.3%}" if res.get("gap") is not None else "hesaplanamadı"
    with open(os.path.join(REPORTS, "fullweek_polish.md"), "w", encoding="utf-8") as f:
        f.write("\n".join([
            "# Tam-Hafta Warm-Start Cilalaması — Koşu Raporu", "",
            "**Takım: Büke** · TEKNOFEST · Yapay Zeka Destekli Havayolu Optimizasyonu", "",
            f"*Üretim: {dt.datetime.now():%Y-%m-%d %H:%M} · "
            f"`python src/08_fullweek_polish.py {time_limit} {lam}`*", "",
            "Aşama-1 (güne-göre ayrıştırma) çözümü, haftanın tamamını kapsayan tek MIP'e",
            "**MIP-start** (HiGHS `setSolution`) olarak verildi; branch-and-bound bu",
            "incumbent'tan devam ederek günler-arası koordinasyonu küresel iyileştirdi.",
            "Amaç ve kısıtlar 03 ile birebir aynıdır (yönteme ek — kurala dokunulmaz).", "",
            "| | Değer |", "|---|---|",
            f"| Günler | {len(days)} ({days[0]}–{days[-1]}) |",
            f"| Pencere (Wdev) | ±{CFG['wdev_min']} dk |",
            f"| İkili değişken | {nbin:,} |",
            f"| G ilk-gün-çapalı bağ | {n_star:,} (muafiyet: byday {len(byday_exempt)}, "
            f"yeni {len(new_ex)}) |",
            f"| Aşama-1 (byday) kayıtlı toplam | {byday_total:,.1f} |",
            f"| MIP-start SAF rekabetçilik | {warm_pure:,.1f} |",
            f"| Cilalanmış SAF rekabetçilik | **{pol_pure:,.1f}** "
            f"(byday'e göre {pol_pure - byday_total:+,.1f}) |",
            f"| Baseline (mevcut tarife) | {base:,.1f} |",
            f"| İyileşme (baseline'a göre) | **{pol_pure - base:+,.1f} "
            f"({100*(pol_pure-base)/base:+.2f}%)** |",
            f"| Toplam kaydırma (tekil olay bazında) | {sh:,.0f} uçuş-dk |",
            f"| **Global MIP-gap (cezalı amaç)** | **{gp}** |",
            f"| Çözüm süresi | {res['t']:.0f} s (sınır {time_limit} s) |",
            f"| A2 gece-aşan doğrulama | {a2x_chk} tur, yeni ihlal {a2x_viol} |",
            f"| output/ güncellendi mi | {'EVET' if improved and a2x_viol == 0 else 'HAYIR'} |",
            "",
            "Not: Global MIP-gap, greedy gün-zincirinin veremediği **optimalite kanıtıdır**:",
            "tam-hafta modelinin gerçek optimumu, raporlanan cezalı amacın en fazla gap kadar",
            "üzerindedir. 'SAF rekabetçilik' resmi amaçtır ve cezasız ölçülür (docs §12).",
        ]) + "\n")
    print(f">> rapor: reports/fullweek_polish.md")
    return 0
```

`reports/fullweek_polish.md` bir tablo hâlinde her şeyi kayda geçirir: gün sayısı, pencere, ikili değişken sayısı, G bağ ve muafiyet sayıları, aşama-1 toplamı, MIP-start değeri, cilalanmış değer, baseline, iyileşme, kaydırma, **global MIP-gap**, süre, A2 doğrulama, ve `output/` güncellendi mi.

> **Global MIP-gap neden bu kadar vurgulanıyor?**
>
> Çünkü greedy bir yöntemin **veremediği** tek şey odur: "*tam-hafta modelinin gerçek optimumu, raporladığım değerin en fazla %X üzerindedir.*" Bu, "iyi bir çözüm buldum" ile "bulduğum çözümün optimale ne kadar yakın olduğunu kanıtlayabiliyorum" arasındaki farktır — ve optimizasyon yarışmalarında tam olarak bu fark değerlendirilir.

`src/08_fullweek_polish.py` · satır 592–596 · komut satırı argümanları

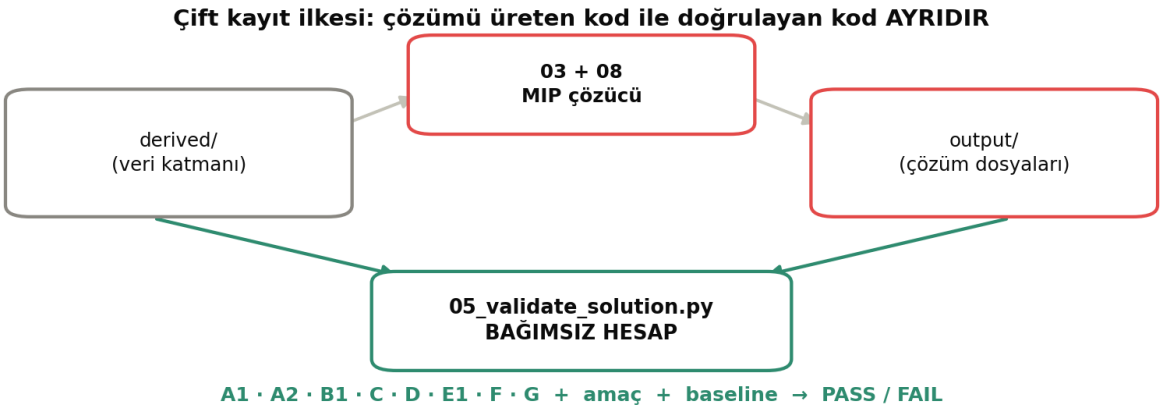
```
if __name__ == "__main__":
    tl = int(sys.argv[1]) if len(sys.argv) > 1 else 600
    lm = float(sys.argv[2]) if len(sys.argv) > 2 else None
    wd = int(sys.argv[3]) if len(sys.argv) > 3 else None
    sys.exit(main(time_limit=tl, lam=lm, wdev=wd))
```

Üç isteğe bağlı argüman: süre sınırı (saniye), λ, Wdev. `run_all.py` bunu `08_fullweek_polish.py 1800` olarak çağırır.

5 05_validate_solution.py — Bağımsız Doğrulayıcı

Bu dosya ne yapar?

`output/` altındaki çözümü, **modelin içinden hiç geçmeden**, yalnız `derived/` veri katmanı üzerinden yeniden hesaplar ve her kısıtı tek tek denetler. Bir tane bile FAIL varsa **1 ile çıkar** ve `run_all.py` koşuyu durdurur. Muhasebede buna **çift kayıt** denir: aynı sayıya iki bağımsız yoldan varmak.



Tek bir FAIL → çıkış kodu 1 → run_all.py koşuyu durdurur.

05, modelin içinden HİÇ geçmez; sabitleri (Wdev, τ, Xdev, cap) kendi dosyasında yeniden tanımlar.
Böylece modeldeki bir hata, doğrulamada aynı hatayla örtülemez.

Şekil 5.1 — Üreten kod ile doğrulayan kod ayrıdır.

	Değer
Girdi	`derived/` (veri) + `output/` (çözüm) — **`src/03_model.py` DEĞİL**
Çıktı	`reports/validation_report.md`
Çalıştırma	`python src/05_validate_solution.py`
Çıkış kodu	**0** = tüm kontroller PASS · **1** = en az bir FAIL
Bağımlılık	`pandas`, `numpy` — *Pyomo yok, çözücü yok*
Süre	birkaç saniye

5.1 Doğrulama listesi

`src/05_validate_solution.py` · satır 1–30 · hangi kontroller yapılıyor?

```
"""
BAĞIMSIZ ÇÖZÜM DOĞRULAYICI (5. adım) — MIP kodundan ayrı ikinci kayıt.

`src/03_model.py`'nin ürettiği `output/` dosyalarını, modelin İÇİNDEN geçmeden
(yalnız `derived/` veri katmanı + `output/` çıktıları üzerinden) yeniden hesaplayıp
doğrular. Amaç: jürinin, çözümün kısıt uyumunu ve amaç değerini çözücüden bağımsız
bir kod yoluyla teyit edebilmesi ("çift kayıt" ilkesi).

Kontroller (docs/model_formulation.md kısıt adlarıyla):
  A1  yeni_IST 10 dk ızgarada, |değişim| ≤ Wdev=60 dk, saat aritmetiği tutarlı
  A2  uçak rotasyonu — GERÇEK zaman ekseninde (gece-aşan dahil): mevcut tarifenin
      sağladığı her turda optimize tarife de T_OB+τ+T_IB minimumunu sağlar;
      ayrıca aynı-gün turlarda model çapası (min(brief-exact, mevcut tur)) korunur
  B1  seçilen her bağlantıda 60 ≤ boşluk ≤ 300; J = legsum + boşluk; boşluk, tarifedeki
```

```
      optimize saatlerle birebir tutarlı
  C   aktarma ödülü = Σ_{j=1..n} (1/2)^(j-1), n = sunulan bağlantı sayısı
  D   yenilen rakip: pazarın en iyi J'si ≤ T_comp olan rakip sayısı (dosyadakiyle
      karşılaştırılır; fazla-beyan = HATA, eksik-beyan = ihtiyatlı, not edilir)
  E1  |n_od − n_do| ≤ α·(n_od + n_do), α = 0.20 (her iki yönü de mevcut pazar çifti)
  F   IST kova kapasitesi: her operasyon günü + 10 dk kovada kalkış ≤ 10, varış ≤ 15
      (sabit + ayarlanabilir TÜM uçuşlar)
  G   tarife düzenliliği: aynı uçuş no+rol, ilk günkü yerleşiminin ±Xdev=15 dk bandında
      (bandın penceresinin yetişemediği günler modeldeki gibi atlanır ve raporlanır)
  Amaç  Σ ρ·(aktarma + sıralama) hem market_results üzerinden hem lookup'tan yeniden
      hesaplanır; BASELINE tümüyle bağımsız (yalnız derived/) yeniden hesaplanır ve
      results_summary.md'deki değerlerle karşılaştırılır.

Çalıştırma:  python src/05_validate_solution.py   →  reports/validation_report.md
Çıkış kodu:  0 = tüm kontroller PASS, 1 = en az bir FAIL.
"""
```

Docstring, kontrol edilecek her kısıtı `docs/model_formulation.md` ile aynı harflerle sıralar. Bu, jürinin "şu kısıt doğrulandı mı?" sorusunu tek bakışta yanıtlamasını sağlar.

src/05_validate_solution.py · satır 32–48 · import'lar — burada ne YOK'a dikkat

```python
import os
import re
import sys

import numpy as np
import pandas as pd

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
DERIVED = os.path.join(ROOT, "derived")
OUTPUT = os.path.join(ROOT, "output")
REPORTS = os.path.join(ROOT, "reports")
```

Bu 17 satırın en anlamlı yanı **içermediği** şeydir: ne `pyomo`, ne `highspy`, ne de `03_model.py`. Doğrulayıcı yalnız `pandas` ve `numpy` kullanır. Modeli kuran/çözen hiçbir kod bu dosyaya girmez — bağımsızlığın ilk ve en görünür işareti budur.

5.2 En önemli tasarım kararı: sabitlerin tekrar yazılması

src/05_validate_solution.py · satır 50–66 · 03'ten import ETMEYEN sabitler

```python
# 03_model.py CFG ile birebir aynı sabitler (bağımsızlık için burada AÇIKÇA tekrarlanır)
DELTA = 10        # dk — zaman kovası
# NOT (2026-07-13): Wdev=90 denendi (src/08_fullweek_polish.py Wdev=90) — +%5.34'e
# çıktı AMA bu doğrulayıcı gerçek bir A2 ihlali yakaladı (uçuş 2200/2201, gün 2):
# 2201'in bazı günlerdeki varışı ρ'suz (dışlanan) bir pazara (KCM-AGP) ait olduğundan
# o günkü olay 03'ün build_instance()'ında HİÇ OLUŞTURULMUYOR → rotasyon ortağı 2200
# ile A2 kısıtı o gün hiç kurulmuyor (F'de U4 ile düzeltilen açığın A2 karşılığı).
# Wdev=60'ta çözücü bu boşluğu tetiklemeyecek kadar az kaydırmıştı; Wdev=90'da tetikledi.
# GÜVENLİ olmadığından Wdev=90 İPTAL edildi, Wdev=60 sonucuna dönüldü. A2'nin rotasyon
# ortağı olayları MARKET evreninden BAĞIMSIZ (F'nin extra_use'u gibi) kurması gerekir —
# bu, dikkatli test isteyen ayrı bir düzeltmedir (bkz. reports/ içindeki not).
WDEV = 60         # dk — ayarlanabilir uçuş penceresi ±
L_GAP, U_GAP = 60, 300   # dk — bağlantı boşluğu penceresi
ALPHA = 0.20      # E1 yönsel dengesizlik eşiği
TAU = 45          # dk — minimum yer süresi (A2)
XDEV = 15         # dk — G gün-arası bant yarıçapı
CAP_DEP, CAP_ARR = 10, 15   # F kova kapasiteleri
```

> **Neden `from model03 import WDEV` yazılmadı?**
>
> Çünkü o zaman doğrulama **bağımsız olmazdı**. 03'teki bir sabit yanlış olsaydı, 05 aynı yanlış sabitle kontrol eder ve PASS verirdi — yani hata kendini doğrulardı. Sabitleri elle tekrar yazmak, ikisinin **birbirinden bağımsız iki kayıt** olmasını sağlar. Uyuşmazlık olursa doğrulama FAIL verir; bu bir kusur değil, sistemin **çalıştığının kanıtıdır**.

Kodun içine gömülmüş bir hata ayıklama günlüğü (satır 52–60)

Yukarıdaki bloğun ortasındaki dokuz satırlık yorum, bu dosyanın **gerçekten iş gördüğünün** kanıtıdır. Anlattığı olay şudur:

1. `08_fullweek_polish.py` Wdev = 90 ile denendi ve sonuç %5.34'e çıktı — cazip.
2. Doğrulayıcı **gerçek bir A2 ihlali** yakaladı: uçuş 2200/2201, gün 2.
3. Kök neden bulundu: 2201'in o günkü varışı ρ'su olmayan bir pazara (KCM–AGP) ait olduğu için `build_instance()` o olayı hiç oluşturmuyordu → rotasyon ortağıyla A2 kısıtı o gün hiç kurulmuyordu.
4. Wdev = 60'ta çözücü bu boşluğu tetikleyecek kadar kaydırma yapmamıştı; Wdev = 90 tetikledi.
5. **Karar: yüksek rakam iptal, Wdev = 60'a dönüldü.** Kök neden ise 03'te `full_role_time` evren-ankoru ile kalıcı olarak kapatıldı (bkz. 3.4).

5.3 Yardımcılar ve `Checks` sınıfı

`src/05_validate_solution.py` · satır 69–84 · okuma ve ızgara yuvarlaması

```python
def rd_derived(name, **kw):
    return pd.read_csv(os.path.join(DERIVED, name), encoding="utf-8-sig", **kw)


def rd_output(name, **kw):
    return pd.read_csv(os.path.join(OUTPUT, name), encoding="utf-8-sig", **kw)


def snap10(ts):
    """Zaman damgasını 10 dk ızgaraya yuvarla (03_model ile aynı: Python round)."""
    m = (ts - ts.normalize()).total_seconds() / 60.0
    return ts.normalize() + pd.Timedelta(minutes=int(round(m / DELTA) * DELTA))


def hhmm_of(ts):
    return f"{ts.hour:02d}:{ts.minute:02d}"
```

`snap10()` bilinçli olarak 03'teki `snap` ile **aynı yuvarlama kuralını** kullanır (Python `round`). Farklı bir yuvarlama, gerçek olmayan tutarsızlıklar üretirdi — bağımsızlık, *aynı tanımı* kullanmayı engellemez; *aynı kodu* kullanmayı engeller.

`src/05_validate_solution.py` · satır 87–102 · PASS/FAIL toplayıcı

```python
class Checks:
    """PASS/FAIL toplayıcı + markdown rapor satırları."""

    def __init__(self):
        self.rows = []          # (kısıt, kapsam, sonuç, ayrıntı)
        self.failed = False

    def add(self, name, scope, ok, detail=""):
        self.rows.append((name, scope, "PASS" if ok else "**FAIL**", detail))
        if not ok:
            self.failed = True
        print(f"  [{'PASS' if ok else 'FAIL'}] {name:<28} {scope:<34} {detail}")

    def note(self, name, scope, detail):
        self.rows.append((name, scope, "not", detail))
        print(f"  [not ] {name:<28} {scope:<34} {detail}")
```

Projedeki ikinci (ve son) sınıf. Üç görevi vardır:

- `add(name, scope, ok, detail)` — kontrolü kaydeder, ekrana basar ve `ok` yanlışsa `failed` bayrağını kaldırır.
- `note(...)` — PASS/FAIL olmayan bilgi satırları (örneğin olay eşleme sayıları) için.
- `rows` listesi — sonunda markdown tablosuna dönüşür.

5.4 Veriyi ve çözümü yükleme

`src/05_validate_solution.py` · satır 105–146 · iki katman + kaydırma sözlüğü

```python
def main():
    print(">> BAĞIMSIZ DOĞRULAMA — output/ çıktıları derived/ verisiyle yeniden hesaplanıyor\n")
    ck = Checks()

    # ------------------------------------------------------------------ veri
    tk = rd_derived("tk_connections.csv", parse_dates=["ist_arr", "ist_dep"])
    markets = rd_derived("markets.csv")
    nod = rd_derived("market_n_od.csv")
    tcomp = rd_derived("competitor_times_carrier.csv")
    rank = rd_derived("ranking_lookup.csv")
    rank.columns = ["N", "b", "s", "W"]
    rkW = {(int(r.N), int(r.b), int(r.s)): float(r.W) for r in rank.itertuples()}
    rot = rd_derived("flight_rotation.csv")
    blocks_df = rd_derived("flight_block_times.csv")
    blocks = {(int(r.flno), int(r.day), str(r.role)): float(r.block_min)
              for r in blocks_df.itertuples()}

    sched = rd_output("optimized_schedule.csv")
    mres = rd_output("market_results.csv")
    conns = rd_output("selected_connections.csv")
    # G hedefli bant-muafiyetleri (rotasyon-güvenliği önceliği; modelin belgelediği liste)
    exempt = set()
    exq = os.path.join(OUTPUT, "g_band_exemptions.csv")
    if os.path.exists(exq):
        for r in pd.read_csv(exq, encoding="utf-8-sig").itertuples():
            exempt.add((int(r.gun), int(r.ucus_no), r.rol))

    days = sorted(tk["day"].dropna().astype(int).unique())
    tk["cur_arr_snap"] = tk["ist_arr"].map(snap10)
    tk["cur_dep_snap"] = tk["ist_dep"].map(snap10)

    # tarifedeki değişimler: (gün, uçuş, rol, eski hh:mm) -> değişim dk
    shift = {}
    for r in sched.itertuples():
        shift[(int(r.gun), int(r.ucus_no), r.rol, r.eski_IST)] = int(r.degisim_dk)

    def opt_dt(flno, day, role, cur_snap_dt):
        """Olayın optimize gerçek zamanı: mevcut (snap) + tarifedeki değişim (yoksa sabit)."""
        rol_tr = "kalkış" if role == "dep" else "varış"
        d = shift.get((day, flno, rol_tr, hhmm_of(cur_snap_dt)), 0)
        return cur_snap_dt + pd.Timedelta(minutes=d)

```

Kilit yapı `shift` sözlüğü ve `opt_dt()` fonksiyonudur:

> **Optimize saati nasıl bulunuyor?**
>
> Doğrulayıcı çözücüden hiçbir şey almaz. Elinde yalnız (a) `derived/`'deki **mevcut** saatler ve (b) `optimized_schedule.csv`'deki **değişim dakikaları** vardır. `opt_dt()` bu ikisini birleştirir: `optimize = snap(mevcut) + değişim`. Tarifede kaydı olmayan bir olay **sabit** kabul edilir (değişim = 0). Yani doğrulayıcı çözümü sıfırdan yeniden inşa eder.

5.5 Kontrol A1 — ızgara, pencere ve saat aritmetiği

`src/05_validate_solution.py` · satır 147–160 · üç ayrı A1 kontrolü

```python
    # ------------------------------------------------------------------ A1
    grid_ok = (sched["degisim_dk"] % DELTA == 0).all()
    wdev_ok = (sched["degisim_dk"].abs() <= WDEV).all()
    n_bad_hhmm = 0
    for r in sched.itertuples():
        h0, m0 = map(int, r.eski_IST.split(":"))
        tot = (h0 * 60 + m0 + int(r.degisim_dk)) % 1440
        if f"{tot // 60:02d}:{tot % 60:02d}" != r.yeni_IST:
            n_bad_hhmm += 1
    ck.add("A1 ızgara (10 dk)", f"{len(sched)} uçuş hareketi", grid_ok)
    ck.add("A1 pencere (±60 dk)", f"{len(sched)} uçuş hareketi", wdev_ok,
           f"maks |değişim| = {sched['degisim_dk'].abs().max()} dk")
    ck.add("A1 saat aritmetiği", "eski+değişim = yeni", n_bad_hhmm == 0,
           f"tutarsız satır: {n_bad_hhmm}")
```

Kontrol	Ne doğrular?	Beklenen
Izgara	Her değişim 10'un katı mı?	`degisim_dk % 10 == 0`
Pencere	Hiçbir kaydırma ±60 dk'yı aşıyor mu?	`|degisim_dk| ≤ 60`
Saat aritmetiği	`eski_IST + değişim == yeni_IST` mi?	0 tutarsız satır

Üçüncü kontrol basit görünür ama önemlidir: çıktı dosyasındaki `yeni_IST` sütunu, `eski_IST` ve `degisim_dk` ile **tutarlı** olmalıdır. Bir yazma hatası bu kontrolde yakalanır.

5.6 Olay tablosunun yeniden inşası

`src/05_validate_solution.py` · satır 162–176 · varış ve kalkış olaylarının birleştirilmesi

```
    # ------------------------------------------------------------------ olay tablosu (F, A2, G için)
    # olay = (gün, uçuş, rol, snap'lenmiş mevcut zaman); bağlantılarda paylaşılır → unique
    ev = pd.concat([
        tk[["day", "FlNo1", "cur_arr_snap"]].rename(
            columns={"FlNo1": "flno", "cur_arr_snap": "cur"}).assign(role="arr"),
        tk[["day", "FlNo2", "cur_dep_snap"]].rename(
            columns={"FlNo2": "flno", "cur_dep_snap": "cur"}).assign(role="dep"),
    ]).drop_duplicates(["day", "flno", "role", "cur"]).reset_index(drop=True)
    ev["opt"] = [opt_dt(int(r.flno), int(r.day), r.role, r.cur) for r in ev.itertuples()]
    n_events = len(ev)
    n_adj = sum(1 for r in ev.itertuples()
                if (int(r.day), int(r.flno), "kalkış" if r.role == "dep" else "varış",
                    hhmm_of(r.cur)) in shift)
    ck.note("olay eşleme", f"{n_events} olay", f"{n_adj} ayarlanabilir (tarife dosyasıyla eşleşti), "
            f"{n_events - n_adj} sabit; tarife satırı: {len(sched)}")
```

`tk_connections.csv`'den varış ve kalkış olayları ayrı ayrı çıkarılıp `drop_duplicates` ile tekilleştirilir — 03'teki `events` sözlüğünün **bağımsız karşılığı**. `note` satırı kaç olayın tarife dosyasıyla eşleştiğini (ayarlanabilir) ve kaçının sabit olduğunu bildirir; bu sayılar 03'ün konsol çıktısıyla karşılaştırılabilir.

5.7 Kontrol F — kova kapasitesi

`src/05_validate_solution.py` · satır 178–188 · her gün × 10 dk kova doluluğu

```
    # ------------------------------------------------------------------ F  (kova kapasitesi)
    worst = {"dep": 0, "arr": 0}
    n_over = 0
    for (day, role), grp in ev.groupby(["day", "role"]):
        occ = grp.groupby("opt").size()
        cap = CAP_DEP if role == "dep" else CAP_ARR
        worst[role] = max(worst[role], int(occ.max()))
        n_over += int((occ > cap).sum())
    ck.add("F kova kapasitesi", f"{len(days)} gün × 10 dk kovalar", n_over == 0,
           f"aşan kova: {n_over} · en dolu kova: kalkış {worst['dep']}/{CAP_DEP}, "
           f"varış {worst['arr']}/{CAP_ARR}")
```

Kritik nokta: gruplama `opt` (optimize saat) üzerinden yapılır, `cur` üzerinden değil. Yani **çözümün ürettiği** dolulukları sayar. Ayrıca **tüm** TK uçuşları sayılır — sabitler, ayarlananlar ve amaç evreni dışındakiler. Bu, 03'ün `extra_use` mekanizmasının doğru çalışıp çalışmadığının bağımsız testidir.

5.8 Kontrol A2 — rotasyon, gerçek zaman ekseninde

`src/05_validate_solution.py` · satır 190–225 · gece-aşan turlar dahil doğrulama

```
    # ------------------------------------------------------------------ A2  (rotasyon, gerçek eksen)
    pairs = []
    for _, grp in rot.groupby("Pair"):
        for of in grp[grp["Orig"] == "IST"]["FlNo"]:
            for inf in grp[grp["Dest"] == "IST"]["FlNo"]:
                pairs.append((int(of), int(inf)))
    cur_map = {(int(r.flno), int(r.day), r.role): r.cur for r in ev.itertuples()}
    opt_map = {(int(r.flno), int(r.day), r.role): r.opt for r in ev.itertuples()}
    a2_chk = a2_new = a2_base = 0
    for of, inf in pairs:
```

```
        for d in days:
            t_ob = blocks.get((of, d, "dep"))
            dep_c, dep_o = cur_map.get((of, d, "dep")), opt_map.get((of, d, "dep"))
            if dep_c is None or dep_o is None or t_ob is None:
                continue
            pick = None
            for d2 in (d, d + 1):
                a_c = cur_map.get((inf, d2, "arr"))
                t_ib = blocks.get((inf, d2, "arr"))
                if a_c is not None and t_ib is not None and a_c > dep_c:
                    pick = (d2, a_c, t_ib)
                    break
            if pick is None:
                continue
            d2, a_c, t_ib = pick
            a_o = opt_map.get((inf, d2, "arr"))
            if a_o is None:
                continue
            a2_chk += 1
            need = pd.Timedelta(minutes=t_ob + TAU + t_ib)
            if a_c < dep_c + need:
                a2_base += 1        # mevcut tarife de sağlamıyor → eşleşme artefaktı
            elif a_o < dep_o + need:
                a2_new += 1         # optimizasyonun YENİ yarattığı ihlal
    ck.add("A2 rotasyon (gerçek eksen)", f"{a2_chk} tur (gece-aşan dahil)", a2_new == 0,
           f"YENİ ihlal: {a2_new} · mevcut tarifede de sağlanmayan (artefakt): {a2_base}")
```

Doğrulayıcının en değerli kontrolü budur, çünkü **modelin göremediği** bir şeyi görür: byday ayrıştırmasında gün h'nin kalkışı ile gün h+1'in dönüşü aynı modelde yer almaz. Burada ikisi de dosyadan okunur ve gerçek zaman ekseninde karşılaştırılır.

Dönüş örneğini **mevcut tarife** belirler (206–213): kalkıştan sonraki ilk varış — önce aynı gün, yoksa ertesi gün. Bu, uçağın gerçek eşleşmesinin en makul yorumudur.

Sonuç	Anlamı	FAIL mı?
`a2_base`	Mevcut tarife de bu turu sağlamıyor → eşleşme artefaktı	**Hayır** — ama sayılır ve raporlanır
`a2_new`	Mevcut sağlıyordu, optimizasyon bozdu	**Evet** — 0 olmalıdır

src/05_validate_solution.py · satır 227–244 · aynı-gün model çapasının korunması

```
    # aynı-gün model çapası: opt tur ≥ min(T_OB+τ+T_IB, mevcut tur)
    n_anchor = n_anchor_bad = 0
    for of, inf in pairs:
        for d in days:
            dep_c, dep_o = cur_map.get((of, d, "dep")), opt_map.get((of, d, "dep"))
            arr_c, arr_o = cur_map.get((inf, d, "arr")), opt_map.get((inf, d, "arr"))
            if None in (dep_c, dep_o, arr_c, arr_o):
                continue
            turn = (arr_c - dep_c).total_seconds() / 60.0
            if turn <= 0:
                continue
            t_ob, t_ib = blocks.get((of, d, "dep")), blocks.get((inf, d, "arr"))
            sep = min(t_ob + TAU + t_ib, turn) if (t_ob is not None and t_ib is not None) else turn
            n_anchor += 1
            if (arr_o - dep_o).total_seconds() / 60.0 < sep - 1e-6:
                n_anchor_bad += 1
    ck.add("A2 aynı-gün çapası", f"{n_anchor} tur", n_anchor_bad == 0,
           f"çapa (min(brief-exact, mevcut tur)) altına inen: {n_anchor_bad}")
```

İkinci A2 kontrolü modelin **kendi kuralını** denetler: aynı-gün turlarda optimize tur ≥ `min(brief-exact, mevcut tur)` olmalı. Yani model, kendi kurduğu kısıta gerçekten uymuş mu?

5.9 Kontrol G — düzenlilik bandı

src/05_validate_solution.py · satır 246–272 · ilk-gün çapalı ±Xdev denetimi

```
    # ---------------------------------------------------------------- G  (düzenlilik)
    tod = lambda dtv: (dtv - dtv.normalize()).total_seconds() / 60.0
    g_chk = g_bad = g_skip = g_exempt = 0
    for (flno, role), grp in ev.sort_values("day").groupby(["flno", "role"]):
        rol_tr = "kalkış" if role == "dep" else "varış"
        rows = [r for r in grp.itertuples()
```

```
                if (int(r.day), int(r.flno), rol_tr, hhmm_of(r.cur)) in shift]
        if len({r.day for r in rows}) < 2:
            continue
        c0 = None
        for r in rows:
            o_tod = tod(r.cur) + int(shift[(int(r.day), int(r.flno), rol_tr, hhmm_of(r.cur))])
            if c0 is None:
                c0 = o_tod                              # ilk günün yerleşimi = bant merkezi
                continue
            if (int(r.day), int(flno), rol_tr) in exempt:
                g_exempt += 1                           # belgeli muafiyet (rotasyon > bant)
                continue
            if abs(tod(r.cur) - c0) > WDEV + XDEV:
                g_skip += 1                             # pencere banda yetişemez → modelde atlanır
                continue
            g_chk += 1
            if abs(o_tod - c0) > XDEV + 1e-6:
                g_bad += 1
    ck.add("G düzenlilik (±15 dk bant)", f"{g_chk} gün-arası bağ", g_bad == 0,
           f"bant dışı: {g_bad} · belgeli rotasyon-önceliği muafiyeti: {g_exempt} · "
           f"yapısal atlanan (pencere yetişemez): {g_skip}")
```

Her `(uçuş, rol)` grubunda ilk günün optimize yerleşimi bant merkezi (`c0`) kabul edilir; sonraki günler `|yerleşim − c0| ≤ Xdev` sağlamalı. Üç istisna kategorisi vardır ve **hepsi ayrı ayrı sayılır**:

Kategori	Koşul	Nasıl işlenir?
Belgeli muafiyet	`(gün, uçuş, rol) ∈ g_band_exemptions.csv`	`g_exempt++`, atlanır — rotasyon güvenliği bandın önündedir
Yapısal atlama	`|tod(cur) − c0| > Wdev + Xdev`	`g_skip++`, atlanır — pencere banda **fiziksel olarak** yetişemiyor
Denetlenen	Diğer her şey	`g_chk++`; band dışıysa `g_bad++` → FAIL

> **Muafiyet listesini modelin kendisi yazıyor — bu bir açık mı?**
>
> Hayır, çünkü liste **dosyaya yazılır ve raporlanır**. Jüri `output/g_band_exemptions.csv`'yi açıp kaç muafiyet olduğunu görebilir. Alternatif — muafiyetleri hiç bildirmemek — çok daha kötü olurdu: o zaman G kısıtının nerede uygulanmadığı **bilinemezdi**. Şeffaflık, gizli gevşetmeden iyidir.

5.10 Kontrol B1 — bağlantı tutarlılığı

`src/05_validate_solution.py` · satır 274–313 · üç katmanlı çapraz kontrol

```
    # ------------------------------------------------------------------ B1 + bağlantı tutarlılığı
    tk_key = {}                                         # anahtar -> [satırlar] (anomali olaylar
    for r in tk.itertuples():                           # aynı anahtarda FARKLI saatli satır bırakabilir)
        tk_key.setdefault((int(r.day), r.market, int(r.FlNo1), int(r.FlNo2)), []).append(r)
    n_conn = len(conns)
    bad_gapwin = bad_j = bad_time = bad_gap = miss = 0
    j_opt = {}                                          # (market, gün) -> [J]
    for r in conns.itertuples():
        key = (int(r.gun), r.pazar, int(r.gelen_ucus), int(r.giden_ucus))
        rows = tk_key.get(key, [])
        if not rows:
            miss += 1
            continue
        if not (L_GAP <= r.bosluk_dk <= U_GAP):
            bad_gapwin += 1
        # saat/boşluk: anahtarın HERHANGİ bir satırı (olay kopyası) ile tutarlıysa geçerli
        matched = None
        for row in rows:
            a_opt = opt_dt(int(r.gelen_ucus), int(r.gun), "arr", row.cur_arr_snap)
            d_opt = opt_dt(int(r.giden_ucus), int(r.gun), "dep", row.cur_dep_snap)
            if hhmm_of(a_opt) == r.IST_varis and hhmm_of(d_opt) == r.IST_kalkis:
                matched = (row, a_opt, d_opt)
                break
        if matched is None:
            bad_time += 1
            bad_gap += 1
            row = rows[0]
```

```
301         else:
302             row, a_opt, d_opt = matched
303             if abs((d_opt - a_opt).total_seconds() / 60.0 - r.bosluk_dk) > 0.51:
304                 bad_gap += 1
305         if abs((row.legsum_min + r.bosluk_dk) - r.J_dk) > 0.51:
306             bad_j += 1
307         j_opt.setdefault((r.pazar, int(r.gun)), []).append(float(row.legsum_min) + float(r.bosluk_dk))
308     ck.add("B1 boşluk penceresi", f"{n_conn} seçilen bağlantı", bad_gapwin == 0,
309            f"[60,300] dışı: {bad_gapwin}")
310     ck.add("B1 J = legsum + boşluk", f"{n_conn} bağlantı", bad_j == 0 and miss == 0,
311            f"tutarsız J: {bad_j} · veri eşleşmeyen: {miss}")
312     ck.add("B1 tarife ↔ bağlantı saati", f"{n_conn} bağlantı", bad_time == 0 and bad_gap == 0,
313            f"saat uyuşmayan: {bad_time} · boşluk uyuşmayan: {bad_gap}")
```

Seçilen her bağlantı için üç şey doğrulanır:

1. **Boşluk penceresi:** `60 ≤ bosluk_dk ≤ 300`.
2. **J tutarlılığı:** `legsum_min + bosluk_dk == J_dk` (veriden okunan legsum ile).
3. **Tarife ↔ bağlantı tutarlılığı:** `selected_connections.csv`'deki `IST_varis` ve `IST_kalkis` saatleri, `optimized_schedule.csv`'deki kaydırmalardan hesaplananla **birebir** eşleşmeli.

Üçüncüsü en güçlüsüdür: iki farklı çıktı dosyasının birbiriyle çeliştiğini yakalar. Bir dosyaya doğru, diğerine yanlış yazan bir hata burada FAIL verir.

289–302: Anomali olaylar için ince bir esneklik var — aynı anahtarda farklı saatli birden çok satır olabileceğinden, **herhangi biriyle** tutarlıysa geçerli sayılır.

5.11 Kontroller C, D ve amaç

`src/05_validate_solution.py` · satır 315–364 · ödüllerin ve katkıların yeniden hesabı

```
315     # ------------------------------------------------------------------ pazar sonuçları (C, D, amaç)
316     comp_map = {}                                          # (market, gün) -> {taşıyıcı: T_comp}
317     for r in tcomp.itertuples():
318         comp_map.setdefault((r.market, int(r.day)), {})[r.carrier] = float(r.T_comp_min)
319     rho_map = markets.set_index("market")["rho"].to_dict()
320     nsel = {}                                              # (market, gün) -> seçilen bağlantı sayısı
321     for r in conns.itertuples():
322         nsel[(r.pazar, int(r.gun))] = nsel.get((r.pazar, int(r.gun)), 0) + 1
323
324     bad_cnt = bad_c = bad_rank = bad_katki = over_beat = under_beat = 0
325     tot_file = tot_recomp = 0.0
326     for r in mres.itertuples():
327         key = (r.pazar, int(r.gun))
328         n = int(r.sunulan_baglanti)
329         if n != nsel.get(key, 0):
330             bad_cnt += 1
331         conn_rew = sum(0.5 ** (j - 1) for j in range(1, n + 1))
332         if abs(conn_rew - float(r.aktarma_odul)) > 1e-3:
333             bad_c += 1
334         # D: gerçek yenilen = en iyi J ≤ T_comp olan rakipler
335         N = int(r.N_rakip)
336         beat_file = int(r.yenilen_rakip)
337         js = j_opt.get(key, [])
338         comps = comp_map.get(key, {})
339         beat_true = sum(1 for tc in comps.values() if js and min(js) <= tc)
340         if beat_file > beat_true:
341             over_beat += 1                                 # fazla-beyan → gerçek hata
342         elif beat_file < beat_true:
343             under_beat += 1                                # ihtiyatlı (çözücü θ'yı bağlamamış)
344         # sıralama ödülü lookup'tan
345         rank_rew = 0.0
346         if N >= 1 and not pd.isna(r.baslangic_sira_bod):
347             s = int(np.clip(N - beat_file, 1, N))
348             rank_rew = rkW.get((N, int(r.baslangic_sira_bod), s), 0.0)
349         if abs(rank_rew - float(r.siralama_odul)) > 1e-3:
350             bad_rank += 1
351         rho = float(rho_map.get(r.pazar, 0.0))
352         katki = rho * (conn_rew + rank_rew)
353         if abs(katki - float(r.rho_katki)) > 0.5 + rho * 2e-3:
354             bad_katki += 1
355         tot_file += float(r.rho_katki)
356         tot_recomp += katki
357     ck.add("C aktarma ödülü", f"{len(mres)} pazar-gün", bad_c == 0 and bad_cnt == 0,
358            f"ödül tutarsız: {bad_c} · sayaç tutarsız: {bad_cnt}")
```

```
    ck.add("D yenilen rakip", f"{len(mres)} pazar-gün", over_beat == 0,
           f"fazla-beyan: {over_beat} · ihtiyatlı (eksik-beyan): {under_beat}")
    ck.add("D sıralama ödülü (lookup)", f"{len(mres)} pazar-gün", bad_rank == 0,
           f"lookup uyuşmayan: {bad_rank}")
    ck.add("Amaç ρ-katkı", f"{len(mres)} pazar-gün", bad_katki == 0,
           f"katkı tutarsız: {bad_katki}")
```

Her pazar-gün satırı için beş şey yeniden hesaplanır ve dosyadakiyle karşılaştırılır:

Kontrol	Yeniden hesap	Hata koşulu
Bağlantı sayısı	`selected_connections.csv`'den sayılan n = `market_results`'taki n mi?	`bad_cnt`
C aktarma ödülü	`Σ (½)^(j−1)`, j = 1..n	`bad_c`
D yenilen rakip	Pazarın en iyi optimize J'si ≤ T_comp olan rakip sayısı	`over_beat` (fazla-beyan)
D sıralama ödülü	lookup'tan `W(N, b_od, s)`, `s = clip(N−beat, 1, N)`	`bad_rank`
Amaç katkısı	`ρ × (aktarma + sıralama)`	`bad_katki`

Fazla-beyan ile eksik-beyan neden farklı muamele görüyor?

Fazla-beyan (`over_beat`) = dosyada yazandan az rakip gerçekten yenilmiş → amaç değeri **şişirilmiş** → gerçek hata, FAIL.

Eksik-beyan (`under_beat`) = gerçekte yenilmiş ama çözücü β göstergesini bağlamamış → amaç değeri **olduğundan küçük** görünüyor → hata değil, ihtiyatlılık. Sadece not edilir.

Bu asimetri bilinçlidir: kendi lehimize olan sapmalar hata sayılır, aleyhimize olanlar sayılmaz.

5.12 Kontrol E1

`src/05_validate_solution.py` · satır 366–380 · yön-çifti dengesi

```
    # ---------------------------------------------------------------- E1  (yön dengesi)
    n_by_key = {(r.pazar, int(r.gun)): int(r.sunulan_baglanti) for r in mres.itertuples()}
    e1_chk = e1_bad = 0
    seen = set()
    for (mk, d), n_od in n_by_key.items():
        o, dd = mk.split("-")
        rkey = (f"{dd}-{o}", d)
        if rkey in n_by_key and ((mk, d), rkey) not in seen and (rkey, (mk, d)) not in seen:
            seen.add(((mk, d), rkey))
            n_do = n_by_key[rkey]
            e1_chk += 1
            if abs(n_od - n_do) > ALPHA * (n_od + n_do) + 1e-6:
                e1_bad += 1
    ck.add("E1 yön dengesi (α=0.20)", f"{e1_chk} yön-çifti", e1_bad == 0,
           f"dengesiz çift: {e1_bad}")
```

`market_results.csv`'den her yön ve tersi eşleştirilir, `|n_od − n_do| ≤ 0.20·(n_od + n_do)` denetlenir. `seen` kümesi her çiftin bir kez sayılmasını sağlar.

5.13 Baseline'ın tamamen sıfırdan hesaplanması

`src/05_validate_solution.py` · satır 382–406 · output/'a hiç bakmadan baseline

```
    # ---------------------------------------------------------------- BASELINE (tam bağımsız)
    mk_rho_ok = set(markets[(markets["has_tk"]) & (~markets["excluded_no_rho"])
                            & (markets["rho"].notna())]["market"])
    nod_map = nod.set_index(["market", "day"])["N_od"].to_dict()
    base_tot = 0.0
    n_base_md = 0
    for (mk, d), grp in tk.groupby(["market", "day"]):
        if mk not in mk_rho_ok:
            continue
        n_base_md += 1
        comps = comp_map.get((mk, int(d)), {})
        N = int(nod_map.get((mk, int(d)), len(comps)))
```

```
        off = grp[grp["gap_min"].between(L_GAP, U_GAP)]
        n = len(off)
        conn_rew = sum(0.5 ** (j - 1) for j in range(1, n + 1))
        beaten0 = 0
        if n and comps:
            j0 = float((off["legsum_min"] + off["gap_min"]).min())
            beaten0 = sum(1 for tc in comps.values() if j0 <= tc)
        rank_rew = 0.0
        if N >= 1 and len(grp):
            b = int(np.clip(N - beaten0, 1, N))
            s = int(np.clip(N - beaten0, 1, N))
            rank_rew = rkW.get((N, b, s), 0.0)
        base_tot += float(rho_map.get(mk, 0.0)) * (conn_rew + rank_rew)
```

Bu blok `output/` klasörüne **hiç bakmaz**. Yalnız `derived/`'den, mevcut tarifenin amaç değerini baştan hesaplar: pencere içindeki bağlantılar, azalan getiri ödülü, en iyi J'nin yendiği rakipler, sıralama ödülü.

403–405 satırı dikkat çekicidir: baseline için `b == s` olarak alınır — yani mevcut tarife lookup tablosunun **köşegenine** oturur ve "değişim yok" nötr referansını alır. Bu, 03'teki `b_od` tanımının bağımsız teyididir.

`src/05_validate_solution.py` · satır 408–431 · raporlanan sayılarla karşılaştırma

```
    # results_summary.md'deki raporlanan sayılarla karşılaştır
    rep_opt = rep_base = None
    summ_path = os.path.join(OUTPUT, "results_summary.md")
    if os.path.exists(summ_path):
        txt = open(summ_path, encoding="utf-8").read()
        m1 = re.search(r"OPTIMIZE amaç[^:]*:\*{0,2}\s*([\d,]+)", txt)
        m2 = re.search(r"BASELINE[^:]*:\*{0,2}\s*([\d,]+)", txt)
        rep_opt = float(m1.group(1).replace(",", "")) if m1 else None
        rep_base = float(m2.group(1).replace(",", "")) if m2 else None
    ck.add("Amaç toplamı (OPT)", "market_results ⇔ bağımsız hesap",
           abs(tot_file - tot_recomp) < 5.0,
           f"dosya Σ={tot_file:,.0f} · yeniden hesap={tot_recomp:,.0f}"
           + (f" · raporlanan={rep_opt:,.0f}" if rep_opt else ""))
    if rep_opt is not None:
        ck.add("Amaç ⇔ results_summary", "raporlanan OPT", abs(tot_file - rep_opt) < 5.0,
               f"|Δ| = {abs(tot_file - rep_opt):.1f}")
    ck.add("BASELINE bağımsız hesap", f"{n_base_md} pazar-gün",
           rep_base is None or abs(base_tot - rep_base) < 5.0,
           f"yeniden hesap={base_tot:,.0f}"
           + (f" · raporlanan={rep_base:,.0f} · |Δ|={abs(base_tot - rep_base):.1f}"
              if rep_base is not None else ""))
    if rep_base and rep_opt:
        ck.note("İYİLEŞME", "bağımsız teyit",
                f"Δ = {tot_recomp - base_tot:+,.0f}  ({100 * (tot_recomp - base_tot) / base_tot:+.2f}%)")
```

`results_summary.md` dosyası **regex ile** okunur ve raporlanan OPTIMIZE/BASELINE sayıları çekilir. Üç karşılaştırma yapılır:

- `market_results` toplamı ↔ bağımsız yeniden hesap (fark < 5.0 olmalı)
- `market_results` toplamı ↔ `results_summary.md`'de **raporlanan** OPT
- Bağımsız baseline ↔ raporlanan BASELINE

> **Bu üçlü kontrol neyi kapatıyor?**
>
> Bir sonucun üç yerde birden yanlış olması gerekir ki fark edilmesin: çözüm dosyalarında, özet raporunda ve bağımsız hesapta. Pratikte bu, "raporda yazan sayı ile dosyadaki veri farklı" tipi hataları **imkânsız** hâle getirir.

5.14 Raporun yazılması ve çıkış kodu

`src/05_validate_solution.py` · satır 433–466 · markdown tablo ve genel sonuç

```
    # ---------------------------------------------------------------- rapor
    os.makedirs(REPORTS, exist_ok=True)
    stamp = pd.Timestamp.now().strftime("%Y-%m-%d %H:%M")
    lines = [
        "# Bağımsız Doğrulama Raporu",
        "",
        "**Takım: Büke** · TEKNOFEST · Yapay Zeka Destekli Havayolu Optimizasyonu",
        "",
```

```
        f"*Üretim: {stamp} · `python src/05_validate_solution.py`*",
        "",
        "`output/` çözüm dosyaları, MIP kodundan (`src/03_model.py` + `src/08_fullweek_polish.py`)",
        "geçmeden yalnız `derived/` veri katmanı üzerinden yeniden hesaplanarak doğrulanmıştır",
        "(çift kayıt). Kısıt adları `docs/model_formulation.md` ile aynıdır.",
        "",
        "| Kontrol | Kapsam | Sonuç | Ayrıntı |",
        "|---|---|---|---|",
    ]
    for name, scope, res, det in ck.rows:
        lines.append(f"| {name} | {scope} | {res} | {det} |")
    lines += [
        "",
        f"**Genel sonuç: {'TÜM KONTROLLER GEÇTİ' if not ck.failed else 'FAIL VAR — incele!'}**",
        "",
        "- *ihtiyatlı (eksik-beyan)*: çözücünün β göstergesini bağlamadığı, gerçekte yenilen ama",
        "  raporda sayılmayan rakip — amaç değerini ŞİŞİRMEZ (yalnız olduğundan küçük gösterir),",
        "  bu yüzden hata değildir; fazla-beyan ise hata sayılır (0 olmalı).",
        "- *eşleşme artefaktı (A2)*: mevcut yayınlanan tarifenin kendisinin de sağlamadığı",
        "  rotasyon yorumları (o gün o çift tek uçakla dönülmüyordur); ihlal sayılmaz.",
    ]
    with open(os.path.join(REPORTS, "validation_report.md"), "w", encoding="utf-8") as f:
        f.write("\n".join(lines) + "\n")
    print(f"\n>> rapor: reports/validation_report.md — "
          f"{'TÜM KONTROLLER GEÇTİ' if not ck.failed else 'FAIL VAR'}")
    return 1 if ck.failed else 0
```

Tüm kontroller bir markdown tablosuna dökülür ve `reports/validation_report.md` yazılır. Rapor sonunda iki kavram açıkça tanımlanır (ihtiyatlı eksik-beyan ve A2 eşleşme artefaktı) — okuyucu bu terimlerle raporda karşılaştığında ne demek olduğunu bilir.

src/05_validate_solution.py · satır 469–470 · sys.exit ile çıkış kodu

```
if __name__ == "__main__":
    sys.exit(main())
```

`sys.exit(main())` — tek satırın gücü

`main()` FAIL varsa 1 döndürür. `sys.exit(1)` ise işletim sistemine "bu betik başarısız oldu" der. `run_all.py` bu kodu okur ve **koşuyu durdurur**. Yani doğrulama yalnızca bir rapor değil, boru hattının içine gömülmüş bir **kalite kapısıdır**: doğrulanmamış bir çözüm için figür bile üretilmez.

6 06_make_figures.py — Rapor Figürleri

Bu dosya ne yapar?

`output/` çıktılarını okuyup teknik rapora girecek **dört PNG figürü** üretir. Hiçbir hesap yapmaz, hiçbir dosyayı değiştirmez — yalnız görselleştirir. Eksik girdi varsa o figürü atlar ve nedenini yazar (koşu durmaz).

	Değer
Girdi	`output/results_summary.md`, `optimized_schedule.csv`, `market_results.csv`, `sweep_results.csv` (ops.)
Çıktı	`docs/figures/*.png` — 4 dosya
Çalıştırma	`python src/06_make_figures.py`
Bağımlılık	`matplotlib`, `pandas`

6.1 Kurulum ve palet

`src/06_make_figures.py` · satır 1–32 · docstring, Agg arka ucu ve yollar

```
"""
RAPOR GÖRSELLERİ (6. adım) — output/ çıktılarından teknik rapor figürleri (PNG).

  fig1_gunluk_iyilesme.png    gün-bazlı Δ amaç (OPT − baseline) çubukları
  fig2_kaydirma_dagilimi.png  kaydırılan uçuşların dakika dağılımı (histogram)
  fig3_lambda_pareto.png      λ ödünleşimi: kaydırma ↔ rekabetçilik kazanımı
                              (output/sweep_results.csv varsa; yoksa atlanır)
  fig4_siralama_iyilesme.png  pazar-günlerde sıralama değişimi (b_od − son sıra)

Çalıştırma:  python src/06_make_figures.py   →  docs/figures/*.png
Stil: tek/iki renk (doğrulanmış palet), çekinik ızgara, doğrudan etiket; başlıklar Türkçe.
"""

import os
import re
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import pandas as pd

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)
OUT = os.path.join(ROOT, "output")
FIGDIR = os.path.join(ROOT, "docs", "figures")
os.makedirs(FIGDIR, exist_ok=True)
```

Docstring dört figürün ne olduğunu tek bakışta verir ve stil tercihini de yazar: *tek/iki renk, çekinik ızgara, doğrudan etiket*. Yani figürlerin görünümü de belgelenmiş bir karardır, rastgele değil.

`matplotlib.use("Agg")` satırı `pyplot` import'undan **önce** gelmelidir: Agg, ekran gerektirmeyen dosyaya-çizim arka ucudur. Bu olmadan betik ekransız bir sunucuda veya CI ortamında çöker.

`src/06_make_figures.py` · satır 34–48 · renk paleti ve global stil

```
# doğrulanmış referans palet (aydınlık yüzey) — tek hue: mavi; kutupluluk: mavi/kırmızı
BLUE, RED = "#2a78d6", "#e34948"
INK, INK2, MUTED = "#0b0b0b", "#52514e", "#898781"
GRID, AXIS = "#e1e0d9", "#c3c2b7"

plt.rcParams.update({
```

```
    "figure.dpi": 200, "savefig.dpi": 200, "figure.facecolor": "white",
    "axes.facecolor": "white", "font.family": "DejaVu Sans",
    "axes.edgecolor": AXIS, "axes.labelcolor": INK2, "axes.titlecolor": INK,
    "xtick.color": MUTED, "ytick.color": MUTED, "text.color": INK,
    "axes.grid": True, "grid.color": GRID, "grid.linewidth": 0.7,
    "axes.axisbelow": True, "font.size": 9,
})

tr = lambda v: f"{v:,.0f}".replace(",", ".")     # 123456 -> '123.456'
```

Palet bilinçli olarak **dardır**: tek ana renk (mavi) ve kutupluluk için (iyileşme/gerileme) ikinci bir renk (kırmızı). Gri tonları metin hiyerarşisi içindir.

- `figure.dpi: 200` — baskı kalitesi.
- `axes.grid: True` + `grid.color` açık gri + `axes.axisbelow: True` — ızgara **verinin arkasında** kalır, çekinik durur.
- `font.family: DejaVu Sans` — Türkçe karakterleri (ı, ş, ğ, İ) eksiksiz destekleyen, matplotlib ile birlikte gelen font.
- `tr()` lambda'sı (48) — `123456` → `123.456` dönüşümü, Türkçe binlik ayracı.

`src/06_make_figures.py` · satır 51–56 · style() — ortak temizlik

```
def style(ax, ygrid_only=True):
    for s in ("top", "right"):
        ax.spines[s].set_visible(False)
    if ygrid_only:
        ax.grid(axis="x", visible=False)
    ax.tick_params(length=0)
```

Üst ve sağ çerçeve çizgileri kaldırılır, dikey ızgara kapatılır, tick çizgileri sıfırlanır. Küçük bir fonksiyon ama dört figürün de aynı görünmesini sağlar.

6.2 Figür 1 — Günlük iyileşme

`src/06_make_figures.py` · satır 59–82 · results_summary.md'den regex ile okuma

```
def fig1_daily():
    """Gün-bazlı iyileşme: results_summary.md gün tablosundan (kendi ürettiğimiz format)."""
    txt = open(os.path.join(OUT, "results_summary.md"), encoding="utf-8").read()
    rows = re.findall(r"^\|\s*(\d+)\s*\|\s*[\d.,]+\s*\|\s*[\d.,]+\s*\|\s*([\d.,]+)\s*\|"
                      r"\s*([\d.,]+)\s*\|\s*([+\-][\d.,]+)\s*\|", txt, re.M)
    if not rows:
        print("[atla] fig1: results_summary.md gün tablosu bulunamadı")
        return
    days = [int(r[0]) for r in rows]
    delta = [float(r[3].replace(",", "")) for r in rows]
    fig, ax = plt.subplots(figsize=(6.4, 2.9), layout="constrained")
    ax.bar(days, delta, width=0.62, color=BLUE, edgecolor="white", linewidth=0.8)
    for d, v in zip(days, delta):
        ax.annotate(f"+{tr(v)}", (d, v), ha="center", va="bottom", fontsize=8, color=INK2,
                    xytext=(0, 2), textcoords="offset points")
    ax.set_title("Günlük rekabetçilik iyileşmesi (OPT − mevcut tarife)", loc="left", pad=10)
    ax.set_xlabel("Operasyon günü")
    ax.set_ylabel("Δ amaç (gelir-ağırlıklı)")
    ax.set_xticks(days)
    ax.set_ylim(0, max(delta) * 1.18)
    style(ax)
    fig.savefig(os.path.join(FIGDIR, "fig1_gunluk_iyilesme.png"))
    plt.close(fig)
    print("[ok] fig1_gunluk_iyilesme.png")
```

İlginç bir tercih: veriyi CSV'den değil, **kendi ürettiğimiz markdown tablosundan** regex ile okur. Bunun mantığı, figürün raporda yazan sayılarla **birebir aynı** olmasını garanti etmektir — iki farklı kaynaktan okunsa aralarında sapma olabilirdi. Tablo bulunamazsa figür atlanır ve neden yazılır.

6.3 Figür 2 — Kaydırma dağılımı

`src/06_make_figures.py` · satır 85–104 · histogram ve özet metni

```
def fig2_shift_hist():
```

```
    sched = pd.read_csv(os.path.join(OUT, "optimized_schedule.csv"), encoding="utf-8-sig")
    moved = sched.loc[sched["degisim_dk"] != 0, "degisim_dk"]
    n_all = len(sched)
    fig, ax = plt.subplots(figsize=(6.4, 2.9), layout="constrained")
    bins = [b - 5 for b in range(-60, 80, 10)]
    ax.hist(moved, bins=bins, color=BLUE, edgecolor="white", linewidth=0.8)
    ax.axvline(0, color=AXIS, linewidth=1)
    ax.set_title("Kaydırılan uçuş hareketlerinin dakika dağılımı", loc="left", pad=26)
    ax.set_xlabel("Saat değişimi (dk; − erken, + geç)")
    ax.set_ylabel("Uçuş hareketi")
    ax.set_xticks(range(-60, 70, 20))
    txt = (f"{tr(len(moved))} hareket kaydırıldı (%{100 * len(moved) / n_all:.0f}); "
           f"{tr(n_all - len(moved))} tanesi (%{100 * (n_all - len(moved)) / n_all:.0f}) yerinde kaldı · "
           f"ort. |kayma| {moved.abs().mean():.0f} dk")
    ax.text(0, 1.03, txt, transform=ax.transAxes, fontsize=8, color=MUTED, va="bottom")
    style(ax)
    fig.savefig(os.path.join(FIGDIR, "fig2_kaydirma_dagilimi.png"))
    plt.close(fig)
    print("[ok] fig2_kaydirma_dagilimi.png")
```

`bins = [b - 5 for b in range(-60, 80, 10)]` — kutucuklar 10'ar dakikalık değerlerin **etrafına ortalanır** (−65, −55, …), böylece bir çubuk tam olarak bir kaydırma değerini temsil eder. Başlığın altındaki metin en önemli sayıyı verir: uçuşların yüzde kaçı **hiç kımıldamadı**.

6.4 Figür 3 — λ Pareto eğrisi

`src/06_make_figures.py` · satır 107–137 · ödünleşim eğrisi ve seçilen noktanın vurgulanması

```
def fig3_pareto():
    p = os.path.join(OUT, "sweep_results.csv")
    if not os.path.exists(p):
        print("[atla] fig3: output/sweep_results.csv yok (once: python src/07_sensitivity.py)")
        return
    sw = pd.read_csv(p, encoding="utf-8-sig").sort_values("lam")
    fig, ax = plt.subplots(figsize=(6.4, 3.1), layout="constrained")
    ax.plot(sw["shift_dk"] / 1000, sw["delta"] / 1000, color=AXIS, linewidth=1.2, zorder=1)
    sel = sw["lam"] == 0.3
    ax.scatter(sw.loc[~sel, "shift_dk"] / 1000, sw.loc[~sel, "delta"] / 1000,
               s=42, facecolor="white", edgecolor=BLUE, linewidth=1.6, zorder=2)
    ax.scatter(sw.loc[sel, "shift_dk"] / 1000, sw.loc[sel, "delta"] / 1000,
               s=64, color=BLUE, zorder=3)
    offs = {0.0: (-8, 8, "right"), 0.1: (8, 8, "left"), 0.3: (10, -3, "left"),
            0.7: (-10, -4, "right"), 1.5: (8, -14, "left")}
    for r in sw.itertuples():
        chosen = r.lam == 0.3
        dx, dy, ha = offs.get(float(r.lam), (8, 6, "left"))
        ax.annotate(("λ=" + (f"{r.lam:g}" if not chosen else "0.3 (seçilen)")),
                    (r.shift_dk / 1000, r.delta / 1000),
                    xytext=(dx, dy), textcoords="offset points", ha=ha,
                    fontsize=8, color=(INK if chosen else INK2),
                    fontweight=("bold" if chosen else "normal"))
    ax.margins(y=0.12)
    ax.set_title("Tarife-değişim cezası λ: kaydırma ⇔ kazanım ödünleşimi", loc="left", pad=10)
    ax.set_xlabel("Toplam kaydırma (bin uçuş-dk)")
    ax.set_ylabel("Δ rekabetçilik (bin)")
    style(ax, ygrid_only=False)
    fig.savefig(os.path.join(FIGDIR, "fig3_lambda_pareto.png"))
    plt.close(fig)
    print("[ok] fig3_lambda_pareto.png")
```

`sweep_results.csv` yoksa (yani 07 koşulmadıysa) figür atlanır. Görsel dil ödünleşimi doğrudan anlatır: her nokta bir λ değeridir, x ekseni toplam kaydırma, y ekseni rekabetçilik kazancı. **Seçilen λ = 0.3 dolu daire ve kalın etiketle** vurgulanır; diğerleri içi boş dairedir. `offs` sözlüğü etiketlerin üst üste binmesini engeller.

6.5 Figür 4 — Sıralama iyileşmesi

`src/06_make_figures.py` · satır 140–164 · üç renkli değişim dağılımı

```
def fig4_rank():
    mr = pd.read_csv(os.path.join(OUT, "market_results.csv"), encoding="utf-8-sig")
    mr = mr[(mr["N_rakip"] >= 1) & mr["baslangic_sira_bod"].notna() & mr["son_sira"].notna()]
    dr = (mr["baslangic_sira_bod"] - mr["son_sira"]).astype(int)   # + = sıralama yükseldi
    counts = dr.value_counts().sort_index()
    colors = [RED if k < 0 else (AXIS if k == 0 else BLUE) for k in counts.index]
    fig, ax = plt.subplots(figsize=(6.4, 2.9), layout="constrained")
```

```
    ax.bar(counts.index, counts.values, width=0.62, color=colors,
           edgecolor="white", linewidth=0.8)
    for k, v in counts.items():
        ax.annotate(tr(v), (k, v), ha="center", va="bottom", fontsize=8, color=INK2,
                    xytext=(0, 2), textcoords="offset points")
    n_up = int((dr > 0).sum())
    ax.set_title("Pazar-günlerde sıralama değişimi (başlangıç b − son sıra)", loc="left", pad=26)
    ax.text(0, 1.03, f"{tr(n_up)} pazar-günde sıralama yükseldi (mavi); "
            f"{tr(int((dr == 0).sum()))} değişmedi (gri); {tr(int((dr < 0).sum()))} geriledi (kırmızı)",
            transform=ax.transAxes, fontsize=8, color=MUTED, va="bottom")
    ax.set_xlabel("Sıralama değişimi (basamak; + = iyileşme)")
    ax.set_ylabel("Pazar-gün")
    ax.set_xticks(range(int(counts.index.min()), int(counts.index.max()) + 1))
    ax.set_ylim(0, counts.max() * 1.15)
    style(ax)
    fig.savefig(os.path.join(FIGDIR, "fig4_siralama_iyilesme.png"))
    plt.close(fig)
    print("[ok] fig4_siralama_iyilesme.png")
```

`b_od − son_sıra` farkı hesaplanır: pozitif = sıralama yükseldi. Renkler doğrudan anlam taşır — mavi iyileşme, gri değişmedi, kırmızı geriledi. Başlık altındaki metin üç sayıyı birden verir, okuyucu çubukları tek tek saymak zorunda kalmaz.

src/06_make_figures.py · satır 167–172 · dört figürü sırayla üret

```
if __name__ == "__main__":
    fig1_daily()
    fig2_shift_hist()
    fig3_pareto()
    fig4_rank()
    print(f">> figürler: {os.path.relpath(FIGDIR, ROOT)}/")
```

7 07_sensitivity.py — Duyarlılık Analizi

Bu dosya ne yapar?

Modeli üç ana parametrede (λ, Wdev, τ) **11 kez** koşturur ve tepkisinin makul ve sürekli olduğunu belgeler. **Teslim edilen çözümü değiştirmez** (`write=False`) — bu bir analiz aracıdır, bir üretim adımı değil.

	Değer
Girdi	`derived/`
Çıktı	`reports/sensitivity_analysis.md` + `output/sweep_results.csv`
Çalıştırma	`python src/07_sensitivity.py` (veya `python run_all.py --with-sensitivity`)
Süre	~15–30 dk (11 tam-veri koşusu)

7.1 Neden duyarlılık analizi?

Bir optimizasyon sonucu tek başına bir sayıdır. "Bu sayı ne kadar sağlam?" sorusunun cevabı ise parametreleri oynatıp modelin tepkisine bakmakla verilir. Beklenen davranış: **sürekli ve yönü açıklanabilir** tepki. Küçük bir parametre değişikliğinde sonuç sıçrıyorsa, modelde ya bir kırılganlık ya bir hata vardır.

7.2 03'ü modül olarak yüklemek

`src/07_sensitivity.py` · satır 1–35 · docstring ve importlib ile dinamik yükleme

```
"""
DUYARLILIK ANALİZİ (7. adım) — λ-taraması + pencere (Wdev) ve MCT (τ) duyarlılığı.

Teslim edilen çözümü DEĞİŞTİRMEZ (write=False); modelin ana parametrelere makul ve
sürekli tepki verdiğini belgeler (jüri kriteri: model doğruluğu & analiz derinliği).

  * λ ∈ {0, 0.1, 0.3, 0.7, 1.5}   tarife-değişim cezası (Pareto ödünleşimi)
  * Wdev ∈ {30, 60, 90} dk        ayarlanabilir uçuş penceresi (±)
  * τ ∈ {30, 45, 60} dk           minimum yer süresi (A2 rotasyonu)

Çalıştırma:  python src/07_sensitivity.py
Çıktı:       reports/sensitivity_analysis.md + output/sweep_results.csv
Süre:        ~15–30 dk (11 tam-veri koşusu; HiGHS deterministik, gap ≤ %0.1)
"""

import importlib.util
import os
import sys

import pandas as pd

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)

spec = importlib.util.spec_from_file_location("model03", os.path.join(HERE, "03_model.py"))
m3 = importlib.util.module_from_spec(spec)
spec.loader.exec_module(m3)

LAM, GAP, TL = 0.3, 0.001, 150
BASE = {"wdev_min": 60, "tau_min": 45}
```

Docstring taranan üç parametreyi, süre beklentisini ve en önemlisi şu cümleyi içerir: **"Teslim edilen çözümü DEĞİŞTİRMEZ (write=False)"**. Analiz aracı ile üretim adımı arasındaki sınır baştan çizilmiştir.

08 ile aynı teknik: rakamla başlayan dosya adı normal `import` ile alınamaz. `m3` referansı üzerinden `m3.CFG`, `m3.run_byday`, `m3.main_sweep` doğrudan kullanılır.

`src/07_sensitivity.py` · satır 38–40 · reset_cfg — her varyanttan önce temiz sayfa

```
38 def reset_cfg():
39     m3.CFG.update(BASE)
40
```

Bu üç satır neden kritik?

`m3.CFG` **paylaşılan, değiştirilebilir** bir sözlüktür. Bir varyant `wdev_min`'i 90 yaparsa, sonraki varyant da 90 ile koşar ve sonuçlar birbirine karışır. `reset_cfg()` her koşudan önce temel değerlere (`BASE`) döner. Paylaşılan durumla çalışan kodda bu tür "sıfırlama" alışkanlığı hata önlemenin en ucuz yoludur.

7.3 Üç tarama

`src/07_sensitivity.py` · satır 42–72 · λ, Wdev ve τ koşuları

```
42 def main():
43     # ---------------- λ-taraması (Pareto) ----------------
44     reset_cfg()
45     print(">> [1/3] λ-taraması (5 koşu)")
46     sweep = m3.main_sweep()                                  # [0, 0.1, 0.3, 0.7, 1.5]
47     sw = pd.DataFrame([{
48         "lam": s["lam"], "delta": s["opt"] - s["base"], "dpct": s["dpct"],
49         "moved": s["moved"], "movedpct": s["movedpct"], "shift_dk": s["shift"],
50     } for s in sweep])
51     sw.to_csv(os.path.join(ROOT, "output", "sweep_results.csv"),
52               index=False, encoding="utf-8-sig")
53
54     # ---------------- Wdev ve τ varyantları ----------------
55     variants = []
56     print("\n>> [2/3] pencere (Wdev) duyarlılığı")
57     for wd in (30, 90):
58         reset_cfg()
59         m3.CFG["wdev_min"] = wd
60         s = m3.run_byday(lam=LAM, gap=GAP, time_limit=TL, write=False, verbose=False)
61         variants.append(("Wdev", wd, s))
62         print(f"   Wdev=±{wd}dk: Δ={s['opt'] - s['base']:>+9,.0f} ({s['dpct']:+.2f}%)  "
63               f"kaydırılan %{s['movedpct']:.0f}")
64     print("\n>> [3/3] minimum yer süresi (τ) duyarlılığı")
65     for tau in (30, 60):
66         reset_cfg()
67         m3.CFG["tau_min"] = tau
68         s = m3.run_byday(lam=LAM, gap=GAP, time_limit=TL, write=False, verbose=False)
69         variants.append(("tau", tau, s))
70         print(f"   τ={tau}dk: Δ={s['opt'] - s['base']:>+9,.0f} ({s['dpct']:+.2f}%)  "
71               f"kaydırılan %{s['movedpct']:.0f}")
72     reset_cfg()
```

Tarama	Değerler	Neyi ölçer?	Beklenen yön
λ	0, 0.1, 0.3, 0.7, 1.5	Kazanç ↔ kaydırma ödünleşimi (Pareto)	λ↑ → kaydırma↓; λ=0 dejenere
Wdev	30, 60, 90 dk	Serbestlik alanının etkisi	Wdev↑ → daha çok seçenek → kazanç↑
τ	30, 45, 60 dk	Minimum yer süresinin (MCT) etkisi	τ↑ → rotasyon sıkışır → kazanç↓

λ taraması `m3.main_sweep()` ile yapılır ve sonucu `sweep_results.csv`'ye yazılır (figür 3'ün girdisi). Wdev ve τ varyantları ise doğrudan `run_byday(write=False)` ile koşturulur.

7.4 Raporun dürüst yorumu

`src/07_sensitivity.py` · satır 74–98 · raporun başlığı ve λ tablosu

```
    # ---------------- rapor ----------------
    base_row = sw[sw["lam"] == LAM].iloc[0]
    L = [
        "# Duyarlılık Analizi",
        "",
        "**Takım: Büke** · TEKNOFEST · Yapay Zeka Destekli Havayolu Optimizasyonu",
        "",
        f"*Üretim: {pd.Timestamp.now():%Y-%m-%d %H:%M} · `python src/07_sensitivity.py` · "
        f"tüm koşular tam veri, gap ≤ %0.1, deterministik*",
        "",
        "Teslim edilen çözüm **λ=0.3, Wdev=±60, τ=45** iledir. Bu analiz çözümü değiştirmez;",
        "modelin parametrelere tepkisinin makul/sürekli olduğunu belgeler. Tüm koşular",
        "**aşama-1 (byday)** çözücüsüyledir; teslim edilen nihai sonuç, bu parametrelerle",
        "byday + `src/08_fullweek_polish.py` (tam-hafta MIP-start cilalaması) iki aşamalı",
        "hattan üretilir (bkz. `docs/model_formulation.md` §13).",
        "",
        "## 1. Tarife-değişim cezası λ (Pareto ödünleşimi)",
        "",
        "| λ | Δ rekabetçilik | Δ % | kaydırılan uçuş | toplam kaydırma (dk) |",
        "|---:|---:|---:|---:|---:|",
    ]
    for r in sw.itertuples():
        sel = " **(seçilen; aşama-1 ölçümü)**" if r.lam == LAM else ""
        L.append(f"| {r.lam}{sel} | {r.delta:+,.0f} | {r.dpct:+.2f}% | "
                 f"%{r.movedpct:.0f} ({r.moved:,}) | {r.shift_dk:,.0f} |")
```

Rapor başında teslim edilen yapılandırma (λ=0.3, Wdev=±60, τ=45) açıkça yazılır ve bu tablodaki rakamların **aşama-1 (byday) ölçümü** olduğu belirtilir — nihai sonuç iki aşamalı hattan gelir.

`src/07_sensitivity.py` · satır 99–133 · sonuçların ve kısıtlarının birlikte yazılması

```
    L += [
        "",
        "λ=0 (cezasız) hem düşük kazanım verir hem uçuşların ~%90+'ını kaydırır (dejenerasyon);",
        "λ>0 simetriyi kırar: kazanım artar, kaydırma ~%25'e düşer. λ∈[0.1,1.5] kararlı bant.",
        "",
        "## 2. Pencere genişliği Wdev ve minimum yer süresi τ (λ=0.3)",
        "",
        "| Varyant | Δ rekabetçilik | Δ % | kaydırılan | not |",
        "|---|---:|---:|---:|---|",
        f"| **Wdev=±60, τ=45 (aşama-1 referansı)** | {base_row.delta:+,.0f} | {base_row.dpct:+.2f}% | "
        f"%{base_row.movedpct:.0f} | referans (byday ölçümü) |",
    ]
    notes = {("Wdev", 30): "dar pencere → daha az serbestlik (beklenen yön)",
             ("Wdev", 90): "DİKKAT: problem büyür; zaman-limitli ölçüm (yorum gerekir)",
             ("tau", 30): "kısa MCT → rotasyon gevşer (beklenen yön; yalnız analiz)",
             ("tau", 60): "uzun MCT; artefakt-sınıflandırma nedeniyle monoton olmayabilir"}
    for kind, val, s in variants:
        tag = f"Wdev=±{val}, τ=45" if kind == "Wdev" else f"Wdev=±60, τ={val}"
        L.append(f"| {tag} | {s['opt'] - s['base']:+,.0f} | {s['dpct']:+.2f}% | "
                 f"%{s['movedpct']:.0f} | {notes[(kind, val)]} |")
    L += [
        "",
        "Not: Wdev=±90'da olay başına aday sayısı artıp problem büyüdüğünden 150 s/gün limitinde",
        "çözücü kalitesi düşebilir (ölçülen fark serbestlik etkisi olmayabilir); τ'da artefakt-",
        "sınıflandırma sınırı kaydığı için tepki monoton olmayabilir. Sonuçları raporlarken bu iki",
        "etkeni açıkça belirtin (bkz. reports/sensitivity_analysis.md'nin elden geçirilmiş yorumu).",
    ]
    with open(os.path.join(ROOT, "reports", "sensitivity_analysis.md"), "w",
              encoding="utf-8") as f:
        f.write("\n".join(L) + "\n")
    print("\n>> yazıldı: reports/sensitivity_analysis.md + output/sweep_results.csv")


if __name__ == "__main__":
    main()
```

Raporun en değerli kısmı, sonuçları **kayıtsız şartsız sunmamasıdır**. İki uyarı açıkça yazılır:

- **Wdev = ±90 için:** olay başına aday sayısı 13'ten 19'a çıkar, problem büyür ve 150 s/gün sınırında çözücü kalitesi düşebilir. Ölçülen fark, *serbestlik etkisi olmayabilir* — zaman limiti etkisi olabilir.
- **τ için:** artefakt-sınıflandırma sınırı kaydığı için tepki monoton olmayabilir (bkz. A2'deki artefakt tanımı).

Bu neden bir güç göstergesi?

Bir sonucun **hangi koşullarda geçerli olmadığını** bilmek, sonucu bilmek kadar değerlidir. Rapor "Wdev=90 daha iyi" demek yerine "bu ölçüm zaman-limitlidir, yorum gerekir" der. Nitekim Wdev=90 denemesi tam da bu ihtiyat sayesinde sorgulanmış ve 05 doğrulayıcısı gerçek bir A2 ihlali bulmuştur (bkz. Bölüm 5.2).

Rapor ayrıca teslim edilen sonucun **iki aşamalı hattan** (byday + 08) geldiğini, bu tablodaki rakamların ise **yalnız aşama-1 ölçümü** olduğunu açıkça belirtir — iki farklı sayının karıştırılmasını önler.

8 04_export_docs.py — Doküman Dışa Aktarma

Bu dosya ne yapar?

Markdown kaynaklarını teslim biçimlerine çevirir: `.md → .docx` (pandoc ile) ve `.docx → .pdf` (Microsoft Word COM otomasyonu ile). Araçlar yoksa **atlar, çökmez** — teslimdeki hazır dosyalar geçerli kalır.

`src/04_export_docs.py` · satır 1–30 · docstring, kaynaklar ve UTF-8 konsol

```
# -*- coding: utf-8 -*-
"""
TEKNOFEST Havayolu Optimizasyonu — Teslim Dokümanı Dışa Aktarma (4. adım)

Markdown kaynaklarından teslim biçimlerini üretir:
  .md → .docx  (pandoc / pypandoc)
  .docx → .pdf (Microsoft Word COM otomasyonu; Word yüklü değilse atlanır ve
                'Word'de açıp Farklı Kaydet → PDF' notu basılır)

Çalıştırma:  python src/04_export_docs.py   (proje kökünden)
Brief §5: model dokümanı PDF ister; teknik rapor ve sonuç özeti de aynı akışla üretilir.
"""

import os
import subprocess
import sys

try:
    sys.stdout.reconfigure(encoding="utf-8")
except Exception:
    pass

HERE = os.path.dirname(os.path.abspath(__file__))
ROOT = os.path.dirname(HERE)

DOCS = [
    os.path.join(ROOT, "docs", "model_formulation.md"),
    os.path.join(ROOT, "reports", "technical_report.md"),
    os.path.join(ROOT, "output", "results_summary.md"),
]
```

Üç doküman çevrilir: model formülasyonu, teknik rapor ve sonuç özeti. Brief yalnızca model dokümanının PDF olmasını ister; diğer ikisi de aynı akışla üretilir.

`src/04_export_docs.py` · satır 33–42 · md_to_docx — pandoc çağrısı

```
def md_to_docx(md_path):
    import pypandoc  # yoksa main() bunu yakalar ve adım atlanır (koşu durmaz)
    docx_path = os.path.splitext(md_path)[0] + ".docx"
    # --resource-path: md içindeki göreli görsel yolları (docs/figures/…) proje kökünden
    # ve dosyanın kendi klasöründen çözülsün (pandoc varsayılanı yalnız CWD'dir)
    res = os.pathsep.join([ROOT, os.path.dirname(md_path)])
    pypandoc.convert_file(md_path, "docx", outputfile=docx_path,
                          extra_args=["--resource-path", res])
    print(f"[docx] {os.path.relpath(docx_path, ROOT)}")
    return docx_path
```

`--resource-path` neden gerekli?

Markdown içindeki görsel yolları göreli yazılmıştır (`docs/figures/...`). Pandoc varsayılan olarak bunları yalnız **çalışma dizinine** göre arar. `--resource-path` ile hem proje kökü hem de dosyanın kendi klasörü arama yoluna eklenir; böylece figürler betik nereden çağrılırsa çağrılsın bulunur.

`src/04_export_docs.py` · satır 45–62 · docx_to_pdf — Word COM üzerinden

```
def docx_to_pdf(docx_path):
    """Word COM üzerinden PDF (wdFormatPDF=17). PowerShell ile — ek Python bağımlılığı yok."""
    pdf_path = os.path.splitext(docx_path)[0] + ".pdf"
    ps = (
        "$ErrorActionPreference='Stop';"
        "$word = New-Object -ComObject Word.Application;"
```

```
        "$word.Visible = $false;"
        f"$doc = $word.Documents.Open('{docx_path}');"
        f"$doc.SaveAs('{pdf_path}', 17);"
        "$doc.Close($false); $word.Quit()"
    )
    r = subprocess.run(["powershell", "-NoProfile", "-Command", ps],
                       capture_output=True, text=True)
    if r.returncode == 0 and os.path.exists(pdf_path):
        print(f"[pdf ] {os.path.relpath(pdf_path, ROOT)}")
    else:
        print(f"[pdf ] ÜRETİLEMEDİ ({os.path.basename(docx_path)}): Word COM hatası — "
              f"dosyayı Word'de açıp 'Farklı Kaydet → PDF' ile dışa aktarın.\n{r.stderr.strip()}")
```

PDF üretimi için ek bir Python kütüphanesi kurmak yerine, **zaten yüklü olan Word** kullanılır. PowerShell üzerinden COM nesnesi açılır, dosya görünmez modda açılıp `SaveAs(..., 17)` ile kaydedilir (17 = `wdFormatPDF`). Başarısız olursa kullanıcıya elle nasıl yapacağı söylenir — sessiz başarısızlık yok.

src/04_export_docs.py · satır 65–79 · main — zarif atlama

```
def main():
    try:
        import pypandoc  # noqa: F401 — varlık kontrolü
        pypandoc.get_pandoc_version()
    except Exception as e:
        print(f"[atla] pypandoc/pandoc bulunamadı ({e}) — docx/pdf üretimi atlandı; "
              "teslimdeki hazır .docx/.pdf dosyaları geçerlidir. "
              "(Kurulum: pip install pypandoc_binary)")
        return
    for md in DOCS:
        if not os.path.exists(md):
            print(f"[atla] bulunamadı: {md}")
            continue
        docx = md_to_docx(md)
        docx_to_pdf(docx)
```

İlk iş pandoc var mı diye bakmaktır. Yoksa **açıklayıcı bir mesajla** çıkar: "docx/pdf üretimi atlandı; teslimdeki hazır dosyalar geçerlidir. (Kurulum: pip install pypandoc_binary)". Bu, `run_all.py`'nin son adımının isteğe bağlı bir araç yüzünden tüm koşuyu başarısız göstermesini engeller.

9 run_all.py — Uçtan Uca Koşu

Bu dosya ne yapar?

Tek komutla teslimin tamamını sıfırdan üretir. Her adımı **ayrı bir süreçte** çalıştırır; bir adım başarısız olursa akış açık bir hata mesajıyla durur ve o adımın çıkış kodunu döndürür.

`run_all.py` · satır 1–12 · kullanım ve süre beklentisi

```
"""
TEK KOMUTLA UÇTAN UCA KOŞU — teslimin tamamını sıfırdan üretir. (Takım: Büke)

  python run_all.py                    veri sağlığı → hazırlama → model (byday λ=0.3)
                                       → tam-hafta warm-start cilalaması (≤1800 s)
                                       → bağımsız doğrulama → figürler → docx/pdf
                                       (toplam ~35–40 dk; ~30 dk'sı cilalama adımı)
  python run_all.py --with-sensitivity ayrıca λ/Wdev/τ duyarlılık taraması (~15–30 dk)

Her adım ayrı süreçte koşar; bir adım başarısız olursa akış AÇIK hata ile durur
(bağımsız doğrulayıcının FAIL'i de koşuyu durdurur — çözüm doğrulanmadan teslim edilmez).
"""
```

`run_all.py` · satır 14–36 · adım listesi

```
import os
import subprocess
import sys
import time

ROOT = os.path.dirname(os.path.abspath(__file__))
SRC = os.path.join(ROOT, "src")

STEPS = [
    ("Veri sağlık kontrolü", ["01_data_health_check.py"]),
    ("Veri hazırlama (derived/)", ["02_data_prep.py"]),
    ("MIP modeli — tam veri, güne göre (λ=0.3)", ["03_model.py", "byday"]),
    # 08: byday çözümünü tam-hafta MIP'ine MIP-start verip küresel iyileştirir;
    # iyileşme yoksa byday çıktısını korur (koşuyu durdurmaz). Süre: 1800 s.
    ("Tam-hafta warm-start cilalaması", ["08_fullweek_polish.py", "1800"]),
    ("Bağımsız çözüm doğrulama", ["05_validate_solution.py"]),
]
if "--with-sensitivity" in sys.argv:
    STEPS.append(("Duyarlılık analizi (λ, Wdev, τ)", ["07_sensitivity.py"]))
STEPS += [
    ("Rapor figürleri", ["06_make_figures.py"]),
    ("Teslim dokümanları (.docx/.pdf)", ["04_export_docs.py"]),
]
```

Adımlar bir liste olarak tanımlanır: `(insan-okunur ad, [betik, argümanlar…])`. Bu yapı sayesinde yeni bir adım eklemek tek satırdır. Duyarlılık analizi ise **isteğe bağlıdır** (31–32): 15–30 dakika sürdüğü için varsayılan koşuya dahil değildir, `--with-sensitivity` bayrağıyla eklenir.

#	Adım	Komut	Yaklaşık süre
1	Veri sağlık kontrolü	`01_data_health_check.py`	saniyeler
2	Veri hazırlama	`02_data_prep.py`	~15 sn
3	MIP modeli (byday)	`03_model.py byday`	~5–10 dk
4	Tam-hafta cilalama	`08_fullweek_polish.py 1800`	~30 dk
5	Bağımsız doğrulama	`05_validate_solution.py`	saniyeler
(6)	*Duyarlılık (ops.)*	`07_sensitivity.py`	15–30 dk
6/7	Rapor figürleri	`06_make_figures.py`	saniyeler
7/8	Teslim dokümanları	`04_export_docs.py`	saniyeler

`run_all.py` · satır 39–52 · adımların çalıştırılması

```
39 def main():
40     env = dict(os.environ, PYTHONIOENCODING="utf-8")
41     t_all = time.time()
42     for i, (name, cmd) in enumerate(STEPS, 1):
43         print(f"\n{'=' * 74}\n[{i}/{len(STEPS)}] {name}  —  python src/{' '.join(cmd)}\n{'=' * 74}")
44         t = time.time()
45         r = subprocess.run([sys.executable, os.path.join(SRC, cmd[0]), *cmd[1:]],
46                            cwd=ROOT, env=env)
47         if r.returncode != 0:
48             print(f"\n!! ADIM BAŞARISIZ ({name}, çıkış kodu {r.returncode}) — koşu durduruldu.")
49             sys.exit(r.returncode)
50         print(f"[{i}/{len(STEPS)}] tamam ({time.time() - t:.0f}s)")
51     print(f"\n>> TÜM ADIMLAR TAMAM ({(time.time() - t_all) / 60:.1f} dk). "
52           f"Teslim çıktıları: output/ · docs/ · reports/")
```

Üç tasarım tercihi:

- **Ayrı süreç (`subprocess.run`)** — her adım temiz bir Python yorumlayıcısında koşar. Bir adımın bıraktığı global durum (örneğin mutasyona uğramış bir CFG) diğerini etkileyemez.
- **`PYTHONIOENCODING="utf-8"`** — alt süreçlerin Türkçe çıktısı Windows konsolunda bozulmasın diye ortam değişkeni olarak geçirilir.
- **`sys.exit(r.returncode)`** — hata durumunda çıkış kodu **aynen dışarı aktarılır**. 05'in FAIL'i (kod 1) böylece tüm koşuyu durdurur; bu, doğrulamanın gerçek bir kalite kapısı olmasını sağlayan mekanizmadır.

"Doğrulanmamış çözüm teslim edilmez"

Docstring'in ikinci paragrafı bir mühendislik taahhüdüdür: bağımsız doğrulayıcının FAIL'i koşuyu durdurur. Yani `output/` klasöründe bir sonuç varsa ve `run_all.py` başarıyla bittiyse, o sonuç **kesinlikle** 12 kontrolden geçmiştir.

A Ek A — Hızlı Başvuru

A.1 Komutlar

Ne yapmak istiyorum?	Komut
Her şeyi sıfırdan üret	`python run_all.py`
Duyarlılık analiziyle birlikte	`python run_all.py --with-sensitivity`
Sadece veri sağlığını gör	`python src/01_data_health_check.py`
Sadece veri katmanını yenile	`python src/02_data_prep.py`
Tam veri çöz (aşama-1)	`python src/03_model.py byday`
Farklı λ ile çöz	`python src/03_model.py byday 0.7`
λ ödünleşim tablosu	`python src/03_model.py sweep`
Küçük alt-örnekte tüm kısıtları dene	`python src/03_model.py`
Tam-hafta cilalama (aşama-2)	`python src/08_fullweek_polish.py 1800`
Çözümü doğrula	`python src/05_validate_solution.py`
Figürleri yenile	`python src/06_make_figures.py`
Dokümanları DOCX/PDF yap	`python src/04_export_docs.py`

A.2 Sabitler ve nerede tanımlı oldukları

Sabit	Değer	Anlamı	03'te	05'te
Δ (`delta_min` / `DELTA`)	10 dk	Zaman ızgarası adımı	satır 74	satır 51
Wdev (`wdev_min` / `WDEV`)	±60 dk	Uçuş kaydırma penceresi	75	61
ℓ, u (`L_min`,`U_min` / `L_GAP`,`U_GAP`)	60, 300 dk	Bağlantı boşluğu penceresi	76	62
α (`alpha` / `ALPHA`)	0.20	E1 yönsel dengesizlik toleransı	80	63
τ (`tau_min` / `TAU`)	45 dk	Minimum yer süresi	81	64
Γ (`gamma_min`)	30 dk	E2 süre farkı eşiği	82	—
Xdev (`xdev_min` / `XDEV`)	15 dk	Günler-arası düzenlilik bandı	83	65
cap_dep / cap_arr	10 / 15	Kova başına kalkış / varış kapasitesi	95–96	66
λ (`DEFAULT_LAMBDA`)	0.3	Tarife-değişim cezası ağırlığı	976	—

05'teki sabitler bilinçli olarak 03'ten import edilmez — çift kayıt ilkesi (bkz. Bölüm 5.2).

A.3 Çıktı dosyaları sözlüğü

Dosya	Sütunlar / içerik	Kim üretir?
`output/optimized_schedule.csv`	`gun, ucus_no, rol, eski_IST, yeni_IST, degisim_dk, G_gevsetildi`	03 (byday) veya 08
`output/market_results.csv`	`gun, pazar, rho, N_rakip, sunulan_baglanti, aday_baglanti, yenilen_rakip, baslangic_sira_bod, son_sira, aktarma_odul, siralama_odul, rho_katki`	03 / 08
`output/selected_connections.csv`	`gun, pazar, gelen_ucus, giden_ucus, IST_varis, IST_kalkis, bosluk_dk, J_dk, yendigi_rakip`	03 / 08

Dosya	Sütunlar / içerik	Kim üretir?
output/g_band_exemptions.csv	gun, ucus_no, rol — belgeli G muafiyetleri (boşsa da yazılır)	03 / 08
output/results_summary.md	Sonuç özeti, gün tablosu, notlar, çıktı listesi	03 / 08
output/sweep_results.csv	lam, delta, dpct, moved, movedpct, shift_dk	07
reports/data_health_report.md	Veri sağlık raporu	01
reports/validation_report.md	12 kontrolün PASS/FAIL tablosu	05
reports/fullweek_polish.md	Cilalama koşu raporu + global MIP-gap	08
reports/sensitivity_analysis.md	λ / Wdev / τ duyarlılık tabloları	07
derived/transform_log.md	Yapılan tüm veri dönüşümlerinin kaydı	02

A.4 Fonksiyon dizini — "şunu nerede bulurum?"

Aradığım şey	Dosya · fonksiyon	Satır
Bağlantı penceresi kısıtı (B1)	03_model.py · build_model	386–393
Azalan getiri ödülü (C)	03_model.py · build_model	402–418
Rakip yenme mantığı (D)	03_model.py · build_model	420–475
Uçak rotasyonu (A2)	03_model.py · build_model	492–566
Hub kapasitesi (F)	03_model.py · build_model	686–725
Tarife düzenliliği (G)	03_model.py · build_model	590–647
Amaç fonksiyonu	03_model.py · build_model	727–748
Baseline hesabı	03_model.py · current_baseline	813–828
Gün-gün çözüm döngüsü	03_model.py · run_byday	1027–1199
Gece-aşan koruma sınırları	03_model.py · compute_overnight_guards	979–1024
Çıktı dosyalarının yazımı	03_model.py · write_outputs	1270–1347
Warm-start noktasının kurulması	08_fullweek_polish.py · set_start_values	209–291
Excel süre ayrıştırma	02_data_prep.py · dur_min	72–98
Hafta normalizasyonu	02_data_prep.py · main	190–212
Doğrulama kontrolleri	05_validate_solution.py · main	147–431

B Ek B — Kodla İlgili Muhtemel Sorular

Aşağıdaki sorular, koda bakan birinin ilk soracağı türden sorulardır. Cevaplar doğrudan kodun ilgili satırlarına dayanır.

1. Neden Pyomo + HiGHS? Gurobi/CPLEX neden kullanılmadı?

HiGHS **açık kaynaktır ve lisans gerektirmez** — jüri kodu kendi makinesinde `pip install` sonrası doğrudan koşturabilir. Pyomo ise modeli çözücüden bağımsız yazmayı sağlar: `CFG["solver"]` değiştirilerek başka bir çözücüye geçilebilir. Tekrar-üretilebilirlik, marjinal hız kazancından önemli görüldü.

2. `threads: 1` performansı düşürmüyor mu?

Evet, düşürür. Ama çok çekirdekli MIP çözümü **deterministik değildir** — aynı girdiyle farklı çalışmalarda farklı çözümler dönebilir. Yarışma teslimi için "jüri koştuğunda aynı sayıyı görmeli" kısıtı hız kaygısının önüne konmuştur (`03_model.py` satır 922).

3. Model neden haftanın tamamını tek seferde çözmüyor?

Denendi ve olmadı: tam-hafta modelinde ~102.000 ikili değişken var; HiGHS makul sürede tek bir uygun çözüm bile bulamıyor (`03_model.py` satır 1030). Çözüm iki aşamalı: byday hızlı ve garanti bir çözüm üretir, `08_fullweek_polish.py` onu tam modele **başlangıç noktası** olarak verip küresel iyileştirme ve global MIP-gap elde eder.

4. Bir kısıtı kapattınız (E2) — bu modeli zayıflatmıyor mu?

E2 küçük ölçekte **açık ve doğrulanmıştır**; yalnız tam ölçekte kapatılmıştır ve gerekçesi ölçülmüştür: çekirdek + E2 tek başına çözülemiyor, çekirdek + A2/G/F/E1 hepsi hızlı (`03_model.py` satır 27–28, 110–112). Kapatma kararı gizlenmemiş, `results_summary.md`'nin notlar bölümünde açıkça ilan edilmiştir.

5. B2 kapalıysa model uygun bağlantıları sunmayabilir — bu bir kayıp değil mi?

Amaç fonksiyonu zaten sunmayı **ödüllendirir** (C ödülü), dolayısıyla model kârlı olan her bağlantıyı kendiliğinden sunar. B2'yi zorlamak ise E1 yönsel dengesiyle çatışıp modeli **infeasible** yapıyor (bisection ile doğrulandı, satır 86–89). Yani B2 açık olsaydı hiç çözüm alamazdık.

6. λ = 0.3 nereden geldi?

Taramadan. `03_model.py sweep` beş λ değerini koşturur; $\lambda \in [0.1, 1.5]$ aralığında kazanç düz bir plato çizer (%4.2–4.3) ve λ = 0.3 bu platonun dizidir: en yüksek kazanç, en düşük kaydırma. Sonuçlar `output/sweep_results.csv` ve Figür 3'te.

7. "Artefakt" dediğiniz A2 ihlalleri gerçek ihlal olabilir mi?

Artefakt olarak sayılan turlar, **mevcut yayınlanan tarifenin kendisinin de sağlamadığı** turlardır — yani o gün o çift zaten tek uçakla dönülmüyor. Optimizasyon bunları yaratmadı, sadece devraldı. Kod bu ikisini asla karıştırmaz: `a2x_base` (artefakt) ve `a2x_viol` (YENİ ihlal) ayrı sayılır ve **ikisi de** raporlanır. YENİ ihlal sayısı 0 olmalıdır ve 08'de bu bir yeniden yazma bariyeridir.

8. G muafiyetlerini modelin kendisi yazıyor — bu kendi kendini aklama değil mi?

Muafiyet listesi diske yazılır (`output/g_band_exemptions.csv`), raporlarda sayısıyla birlikte anılır ve doğrulayıcı onu okuyup ayrıca sayar. Alternatif — muafiyetleri hiç bildirmemek — G'nin nerede uygulanmadığını **bilinemez** kılardı. Şeffaf gevşetme, gizli gevşetmeden iyidir.

9. Doğrulayıcı modelden bağımsız derken ne kadar bağımsız?

`05_validate_solution.py` ne Pyomo import eder ne de `03_model.py`'ye dokunur. Sabitleri (Wdev, τ, Xdev, kapasiteler) kendi dosyasında **yeniden yazar**. Girdisi yalnız `derived/` (veri) ve `output/` (çözüm dosyaları)dır. Baseline'ı bile sıfırdan hesaplar. Tek ortaklık, aynı *tanımları* kullanmasıdır — aynı *kodu* değil.

10. Bu doğrulayıcı hiç gerçek hata yakaladı mı?

Evet. `05_validate_solution.py` satır 52–60'taki not, Wdev = 90 denemesinde yakalanan gerçek bir A2 ihlalini kaydeder (uçuş 2200/2201, gün 2). Kök neden bulundu (amaç evreni dışındaki rotasyon ortağı için A2 kısıtının hiç kurulmaması), yüksek rakam **iptal edildi** ve 03'te `full_role_time` evren-ankoru ile kalıcı olarak düzeltildi.

11. Sonuçlar başka bir makinede aynı çıkar mı?

Evet, tasarım gereği: tek iş parçacığı, sabit rasgele tohum (`random_seed: 0`), sabit MIP-gap ve süre sınırları, veriden türetilen (hardcode edilmeyen) gün listesi. Tek değişken, süre sınırına takılan koşuların makine hızına bağlı olmasıdır — bu yüzden gap ve süre sınırları raporlarda **açıkça** yazılır.

12. Gizli test verisi farklı gelirse kod çalışır mı?

Kod bunun için hazırlanmıştır:

- Gün listesi veriden okunur, hardcode edilmez (`all_operation_days()`).
- Dosya adı farklıysa `glob` yedeğiyle aranır (`_find_conn_file()`).
- Beklenen sütun yoksa **anlaşılır** hata verir (`need()` — 02, satır 113–122).
- `ElapsedTime` sütunları yoksa eski şemaya (minimal de-wrap) otomatik düşer.
- `flight_block_times.csv` yoksa A2 ihtiyatlı moda geçer.
- Tek bir günün çözümü hata verirse o gün atlanır, koşu devam eder (`run_byday`, satır 1085–1088).

Kapanış

Bu projenin kodu yalnız bir sonuç üretmiyor; **ürettiği sonucun neden güvenilir olduğunu da üretiyor**. Sağlık raporu, dönüşüm logu, bağımsız doğrulama, duyarlılık analizi ve global MIP-gap — hepsi aynı amaca hizmet eder: bir sayı söylediğinizde, arkasında durabilmek.